

FROM C TO C++ · PRODUCTION-READY REFERENCE

C++ 使用大全

面向 C 语言学习者的 14 讲系统进阶

覆盖 14 大主题:数据类型 → 运算符 → 流程控制 → 函数 → 内存管理 →
面向对象 → 文件I/O → 网络 → 并发异常 → 泛型与元编程 →
反射 → 注解装饰器 → 模块系统 → 类型系统进阶

第1章 数据类型与变量 · 第2章 运算符与表达式 · 第3章 流程控制 · 第4章 函数与方法
第5章 内存管理 · 第6章 面向对象 · 第7章 文件I/O · 第8章 网络通信
第9章 并发与异常 · 第10章 泛型与元编程 · 第11章 反射与动态特性
第12章 注解与装饰器 · 第13章 模块系统 · 第14章 类型系统进阶

知识导图 × C 语言对照 × 例题精讲 × 章末实验

第1章 数据类型与变量

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 C++ 全部内建类型,说出与 C 类型的——对应及差异(long 在 Windows 上是 32 位!)
- 理解四种初始化写法与 auto 推导,搞清楚"未初始化"在 C++ 里意味着什么
- 理解 const 与 constexpr 的本质区别,用编译期常量替代 #define
- 掌握 static_cast 等显式转换,理解列表初始化为何拒绝窄化
- 会用 std::string、std::vector、std::map 替代 C 的 char[] 与裸数组,会用 enum class
- C 对照:知道 C 中未初始化、有符号溢出等 UB 在 C++ 中同样存在,学会主动规避

💡 从 C 到 C++:本章主题如何迁移

如果你能熟练写出 `struct Node { int data; struct Node* next; }` 这样的 C 代码,那么 C++ 的数据类型对你来说九成是熟面孔:int、double、char、指针,字节数与语义几乎不变。C++ 是 C 的超集,语法向下兼容,但它在类型体系上做了三件大事。

第一,类型更安全。C 的 enum 常量泄漏到全局、能隐式变成整数;C++11 的 enum class 把名字关进作用域,拒绝隐式转换。C 的 `(int*)&` 想怎么强转就怎么强转;C++ 提供 static_cast 等四种语义明确的转换,把"窄化转换"列为编译错误。

第二,标准库补齐了 C 的短板。C 的 char[] 要靠 strlen/strcpy 手工维护长度,数组传参退化为指针;C++ 的 std::string 自己管理内存与长度, std::vector 是自动扩容的"安全数组", std::map 直接提供键值对。生产环境里,C 风格字符串与裸数组几乎只出现在遗留代码中。

第三,编译期能力大幅增强。C 的 #define MAX 100 只是文本替换;C++ 的 constexpr 变量与函数在编译期完成计算与类型检查。读完本章,你应该默认使用现代写法。

📦 前置知识自查

- C 的内建类型:char/int/long/float/double 的字节数与取值范围
- sizeof 运算符、#define 宏常量、typedef
- 字符串字面量与转义序列、strcpy/strlen/strcmp 家族
- 数组与指针的关系、数组传参退化
- 隐式类型转换与整型提升、C 风格强转 `(int)x`

🚀 本章学习路线

1. 第一阶段(熟悉):通读 §1.1–§1.4,重点类型对照表与初始化规则,编译运行"类型大小实测"程序
2. 第二阶段(重点):§1.5–§1.7 是最值钱的部分——转换规则、`std::string`、`std::vector`,边读边把 C 写法翻译成 C++
3. 第三阶段(收尾):§1.8 枚举与别名,做自测题与章末实验
4. 验收标准:能默写"类型对照表",能说清 `int a;` 与 `int a{};` 的区别,能徒手写出 `std::string` + `std::vector` 的小程序

本章知识导图

- §1.1 类型总览:C→C++ 类型对照;bool;char 三兄弟;固定宽度整型
- §1.2 变量:四种初始化、默认 vs 值初始化、auto 推导
- §1.3 字面量:进制、后缀、分隔符、原始字符串、编码前缀
- §1.4 常量:const 的局限、constexpr、constexpr/constexpr
- §1.5 类型转换:隐式规则、列表初始化禁窄化、四种 cast
- §1.6 字符串:char[] 痛点 → std::string 常用 API
- §1.7 复合类型:std::array、std::vector、std::map、结构化绑定
- §1.8 枚举与别名:enum class、using

§1.1 类型总览:从 C 类型表到 C++ 类型表

C++ 的内建类型与 C 几乎一一对应。注意:标准不规定精确字节数,只保证大小关系,下表以最常见的 64 位平台为例,程序应随时用 `sizeof` 查询。

C 的写法	C++ 的写法	典型字节(64 位)	说明
int	int	4	语义完全相同
long	long	Win:4;Linux:8	最大陷阱:Msvc 恒 32 位
unsigned	unsigned	4	等价 unsigned int
float/double	float/double	4/8	IEEE 754
无	bool	1	_Bool 现代化身
char	char	1	符号性实现定义
size_t	std::size_t	8	有符号对应 ptrdiff_t
long long	long long	8	跨平台 64 位用它

1.1.1 bool:C 没有的"真正的布尔"

C 直到 C99 才有 `_Bool`,且本质上是整数。C++ 的 `bool` 是独立类型,只有 `true` / `false` 两个值,任何非零值赋给它都会变成 `true`。`sizeof(bool)` 通常是 1,不是 1 位。生产习惯:逻辑判断用 `bool`,不要用 `int` 充当标志位。

1.1.2 char 的三大陷阱

- 符号性未定义: `char` 是 `signed` 还是 `unsigned` 由编译器决定,跨平台读写二进制必须显式用 `signed char / unsigned char` 存字节
- `char` 不是"字符":汉字在 UTF-8 下占 3 字节, `char` 存不下一个汉字;文本处理用 `std::string`
- `char` 参与算术先提升为 `int`: `char c = 100 + 100;` 在有符号 `char` 上可能溢出,与 C 完全一致

1.1.3 固定宽度整型:C++11 的礼物

`<cstdint>` 提供 `std::int8_t/16_t/32_t/64_t` 及无符号版本。生产建议:网络协议、二进制文件、位域计算一律用固定宽度类型,普通计数用 `int` 或 `std::size_t`。

⚠ 易错点 1:long 在 Windows 与 Linux 上字节数不同

Windows(MSVC,LLP64)上 `long` 是 4 字节,64 位 Linux(GCC,LP64)上是 8 字节。同样的 `long x = 3000000000L;` 在 Windows 上溢出、在 Linux 上正常。生产规则:需要确定 64 位时写 `long long` 或 `std::int64_t`,不要写 `long`。

示例 1-1 内建类型大小实测

```
#include <iostream>
#include <cstdint>

int main() {
    std::cout << "long      = " << sizeof(long)      << "\n"; // Windows 上是 4!
    std::cout << "long long = " << sizeof(long long) << "\n";
    std::cout << "size_t   = " << sizeof(std::size_t) << "\n";
    std::cout << "int64_t  = " << sizeof(std::int64_t) << "\n";
    return 0;
}
```

§1.2 变量:声明、初始化与 auto

1.2.1 四种初始化写法

C 只有 `int x = 0;` 一种写法,C++ 有四种,含义各不相同:

C 的写法	C++ 的写法	含义
<code>int x = 0;</code>	<code>int x = 0;</code>	拷贝初始化,兼容 C
无	<code>int x(0);</code>	直接初始化
无	<code>int x{0};</code>	列表初始化,禁止窄化,推荐
无	<code>int x{};</code>	值初始化,归零

生产建议:标量与结构体一律用 `{}`,同时杜绝"忘记初始化"与"窄化截断"两类问题。

1.2.2 未初始化:C 的 UB 文化在 C++ 中的延续

C 里 `int x; printf("%d", x);` 是未定义行为(UB)。C++ 规矩相同:内置类型默认初始化后值不确定,读取即 UB;类类型调用默认构造函数;而 `int x{};` 是值初始化,保证得到 0。全局与静态变量无论哪种写法都零初始化。

⚠ 易错点 2:auto 推导会剥掉 const 和引用

`const int a = 1; auto b = a;` 中 `b` 是 `int`, 不是 `const int` ——`auto` 按值推导, 拷贝了一份。同理 `int& r = x; auto r2 = r;` 得到 `int`。需要保留修饰时写 `const auto& c = a;`。

示例 1-2 变量初始化与 auto 推导

```
#include <iostream>

struct Student {           // 类类型:有默认构造函数
    int id;                 // 无初始化器:值不确定(等价于 C)
    double score = 0.0;    // C++11 默认成员初始化器
};

int main() {
    int a;                  // 默认初始化:值不确定,别读
    int b = 0;              // 拷贝初始化
    int d{0};               // 列表初始化
    int e{};                // 值初始化:确定得到 0

    auto f = 3.14;          // double
    const auto h = 42;      // const int
    std::cout << b << d << e << f << h << "\n";

    Student s1;             // id 不确定,score 是 0.0
    Student s2{};           // 值初始化:id 也归零,推荐
    std::cout << s1.score << s2.id << "\n";
    return 0;
}
```

1.2.3 auto:类型推导的两面性

`auto` 不是 "动态类型", 变量在编译期就已确定类型。它最适合类型名冗长时 (`std::map<std::string, std::vector<int>>::iterator` 写起来痛不欲生)。生产约定:公共接口显式写类型,局部变量放心用 `auto`,推导可读性差时宁可写全。

§1.3 字面量:C++ 的扩展

示例 1-3 字面量全家福

```
#include <iostream>

int main() {
    int dec = 42;           // 十进制
    int hex = 0x2A;         // 十六进制
    int bin = 0b101010;     // C++14 起支持二进制
    int sep = 1'000'000;     // C++14 数字分隔符
    long long ll = 42LL;     // long long 后缀

    double pi = 3.14159;    // 默认 double
    float f = 3.14f;        // float 后缀 f

    char ch = 'A';
    auto s = "hello";        // const char[6], 以 '\0' 结尾
    auto raw = R"(C:\Users\dev\x)"; // 原始字符串: 免转义
    std::cout << dec << hex << bin << sep << ll << pi << f
              << ch << s << raw << "\n";
    return 0;
}
```

原始字符串是生产利器:写正则、JSON、Windows 路径时免双重转义。语法 `R"(内容)"`, 内容里出现 `)` 时可加定界符 `R"xx(内容)xx"`。编码前缀: `u8` 是 UTF-8(`char8_t`, C++20 起), `u` 是 UTF-16, `U` 是 UTF-32。生产规范:文件存 UTF-8、字符串写 `u8` 前缀、全程用 `std::string` 承载。

§1.4 常量:const、constexpr 与编译期计算

1.4.1 const 的局限与 C++ 的修正

C 的 `const int N = 100; int a[N];` 在标准 C 里不能保证编译通过;C 的 `const` 本质是"运行期只读变量"。C++ 修正了这一点:C++ 的 `const` 整型常量是编译期常量,可用于数组大小、模板参数。但 `const` 只在编译期检查,不保证对象不在别处被改(`const int*` 可指向可变对象)。

1.4.2 constexpr:真正的编译期常量

C++11 引入 `constexpr`, C++14 起 `constexpr` 函数里允许循环与分支。它比 `const` 强:值必须在编译期确定。生产替代关系: `#define MAX 100` → `constexpr int kMax = 100;` 后者有类型、有作用域、不污染预处理器。

示例 1-4 `const` 与 `constexpr` 的分工

```
#include <iostream>

constexpr int kMaxItems = 100;           // 替代 #define MAX_ITEMS 100
constexpr int kSquare(int n) { return n * n; } // constexpr 函数

int main() {
    int arr[kMaxItems];                  // 编译期大小,可以
    int arr2[kSquare(10)];               // constexpr 函数也能定数组大小

    int n = 0;
    std::cin >> n;
    const int runtime = n;               // const:运行期只读
    // constexpr int bad = n;           // 编译错误:n 不是编译期常量
    constexpr int kFixed = 2 + 3;       // 正确
    return 0;
}
```

1.4.3 C++20 的 consteval 与 constexpr

constexpr 强制函数只能在编译期求值; **constexpr** 要求静态变量用常量初始化。生产上 **constexpr** 已足够 90% 场景。

§1.5 类型转换:隐式、显式与窄化

1.5.1 隐式转换:与 C 完全一致的规则

整型提升、有符号/无符号混合、double 吸收 float——C 的规则原样继承,C 的坑也全部保留:int 与 unsigned 比较时 int 被转成无符号;有符号溢出仍是 UB。写作时主动避免混用有符号与无符号, `std::vector<T>::size()` 返回 `size_t`, 与 int 比较前先想清楚。

1.5.2 列表初始化拒绝窄化

`int x{300.5};` 是编译错误——double 到 int 是窄化转换。这是 C++ 对 C 最有价值的改进之一:把危险的隐式转换从运行期事故变成编译期错误。而 `int x = 300.5;` 仍能编译(兼容 C),只是截断成 300。

1.5.3 四种显式转换

转换	用途
<code>static_cast</code>	有明确定义的转换,日常 95% 用它
<code>const_cast</code>	去掉 const;能不用就不用
<code>reinterpret_cast</code>	重新解释二进制位;危险,审慎
<code>dynamic_cast</code>	多态运行时向下转型(第6章),有 RTTI 开销

C 风格强转 `(int)x` 是组合拳,编译器不拦任何错误。生产规范:代码中禁止 C 风格强转,统一用四种具名转换,便于 grep 与 review。

示例 1-5 类型转换的正确姿势

```
#include <iostream>

int main() {
    int i = 300;
    char c = static_cast<char>(i);      // 显式转换:我知道可能截断
    double d = i;                      // 隐式:int → double,无损
    i = static_cast<int>(d);            // double → int,丢弃小数

    // int x{300.5};                    // 编译错误:拒绝窄化
    int y{300};                        // 正确:无损放入

    unsigned u = 1u;                  // 有符号/无符号混用陷阱
    int s = -1;
    std::cout << (s < u) << "\n";     // 输出 0!s 被转成 unsigned

    char a = 100, b = 100;             // 整型提升:char 先转 int 再运算
    char sum = static_cast<char>(a + b); // a+b 是 int,等于 200,再转回

    std::cout << sum << y << "\n";
    return 0;
}
```

§1.6 字符串:从 char[] 到 std::string

💡 C 字符串的三大痛点,C++ 的三种解法

C 里字符串是"以 `\0` 结尾的 char 数组": `strcpy` 不知道目标容量,拼长了缓冲区溢出; `strlen` 每次 $O(n)$ 扫描;很多人错误地拿 `==` 去比指针。 `std::string` 一次性解决:内存自动扩容、 `size()` 是 $O(1)$ 、 `==` 比较内容。生产铁律:禁止用 C 字符串函数操作裸缓冲区。

C 的写法	C++ 的写法	差异
<code>strlen</code>	<code>s.size()</code>	$O(1)$ vs $O(n)$
<code>strcmp</code>	<code>a == b</code>	直接比内容
<code>strcpy</code>	<code>s2 = s;</code>	自动管容量
<code>strcat</code>	<code>s2 += s;</code>	自动扩容
<code>strstr</code>	<code>s.find(t)</code>	返回下标,失败 npos

示例 1-6 std::string 常用操作

```

#include <iostream>
#include <string>

int main() {
    std::string s = "hello";
    s += " world";           // 拼接,自动管理容量
    std::cout << s.size() << "\n"; // 0(1),不是 strlen 的 O(n)
    std::cout << (s == "hello world") << "\n"; // 内容比较,不是指针!
    std::cout << s.substr(0, 5) << "\n";       // 子串 "hello"
    std::cout << s.find("world") << "\n";       // 下标 6;找不到返回 npos
    s.replace(6, 5, "C++"); // 替换 [6, 11)
    const char* cstr = s.c_str(); // 与 C 接口互操作:只读 C 字符串

    std::string msg = "count=" + std::to_string(42); // 数值转字符串
    int n = std::stoi("42"); // 字符串转数值

    // 只读视图 string_view(C++17):零拷贝,只做函数参数
    std::string_view sv = "只读数据";
    std::cout << n << msg << cstr << sv.size() << "\n";
    return 0;
}

```

工程建议:编码与性能

编码:现代项目的字符串策略是"内部统一 UTF-8 的 `std::string`,只在 I/O 边界做编码转换"。不要用 `std::wstring` 处理业务数据——Windows 上是 UTF-16,Linux 上是 UTF-32,行为不一致。性能:小字符串优化 (SSO)让小串(约 15 字符内)不分配堆内存;拼接大量小串用 `reserve()` 预分配。

§1.7 复合类型入门:array、vector 与 map

1.7.1 `std::array`:知道长度的固定数组

C 的数组传参退化为指针,函数里拿不到长度——这是 C 数组最大的设计缺陷。`std::array<T, N>` 是值语义的数组,能 `size()`、`at()` 越界检查、能整体拷贝赋值,性能与 C 数组相同。

1.7.2 `std::vector`:生产环境的默认容器

`std::vector<T>` 对应 C 里"malloc + 手工 realloc"的场景,但自动管理:满了翻倍扩容,析构自动释放。注意:扩容会使迭代器与引用失效;删除元素用 `erase-remove` 惯用法(第3章详述)。

示例 1-7 array、vector 与 map 入门

```

#include <iostream>
#include <array>
#include <vector>
#include <map>

int main() {
    // at() 越界抛异常;[] 越界是 UB(和 C 一样)
    std::array<int, 5> a{1, 2, 3, 4, 5};
    std::cout << a.size() << a.at(0) << "\n";

    std::vector<int> v;           // 动态数组,自动扩容
    v.push_back(10);
    v.push_back(20);
    v.emplace_back(30);         // 原地构造,免拷贝
    v[1] = 99;
    std::cout << v.size() << v.front() << v.back() << "\n";

    for (int x : v) {           // 范围 for:现代遍历正统写法
        std::cout << x << " ";   // 10 99 30
    }

    std::map<std::string, int> scores; // 红黑树,按键有序
    scores["张三"] = 90;           // 键不存在则插入
    std::cout << scores["张三"] << scores.size() << "\n";

    for (const auto& [name, score] : scores) { // 结构化绑定(C++17)
        std::cout << name << ":" << score << " ";
    }
    return 0;
}

```

⚠ 易错点 3:vector 越界与迭代器失效

对 `std::vector` 用 `[]` 越界访问是未定义行为——它不会检查,和 C 数组一样可能悄悄踩坏内存。需要安全访问用 `v.at(i)` (越界抛 `std::out_of_range`),需要性能用 `[]` 并先保证下标合法。另外,`push_back` 导致扩容时,之前保存的迭代器、指针、引用全部失效——这是 C 里不存在的“新坑”,第3章专门讲。

§1.8 枚举与类型别名

1.8.1 从 C 的 enum 到 enum class

C 的 `enum Color { RED };` 把 `RED` 泄漏到全局作用域,且枚举值能隐式转 `int`、`int` 也能隐式转回枚举——类型形同虚设。C++11 的 `enum class` 是真正的强类型:名字在作用域内,不隐式转 `int`,还能指定底层类型(默认 `int`,可改成 `char` 省空间)。

示例 1-8 强类型枚举与类型别名

```

#include <iostream>
#include <vector>

// C 的写法:enum Color { RED, GREEN, BLUE }; // 名字泄漏进全局
// C++11 强类型枚举:作用域隔离,不隐式转整数
enum class Color : int { RED, GREEN, BLUE };
enum class Status : char { OK = 0, WARN = 1, ERR = 2 };

int main() {
    Color c = Color::GREEN;
    // int n = c;           // 编译错误:不能隐式转整数
    int n = static_cast<int>(c); // 必须显式转换

    // C 用 typedef,现代 C++ 用 using(可用于模板,更可读)
    using Byte = unsigned char;
    using IntList = std::vector<int>;

    IntList v{1, 2, 3};
    Byte b = 200;
    std::cout << static_cast<int>(b) << v.size() << "\n";
    return 0;
}

```

1.8.2 别名不是新类型

`using Id = int;` 只是别名, `Id` 与 `int` 完全互通。若需要"防混淆的新类型"(学号与工号都是 `int` 但不应混用),需要包装结构体等手段——第14章"新类型模式"会讲。

本章速查表

主题	速查内容
固定 64 位整型	<code>std::int64_t(<cstdint>)</code> ;跨平台别用 <code>long</code>
布尔	<code>bool</code> : <code>true/false</code> , <code>sizeof</code> 为 1
char 三兄弟	<code>char</code> 符号性未定义; <code>signed/unsigned char</code> 存字节
未初始化	内置类型默认初始化值不确定(UB); <code>int x{};</code> 得 0
推荐初始化	一律 <code>{}</code> ;拒绝窄化
auto 陷阱	<code>auto</code> 剥掉 <code>const</code> /引用;要保留写 <code>const auto&</code>
字面量扩展	<code>0b1010</code> 、 <code>1'000'000</code> 、 <code>R"(...)"</code>
常量	<code>#define</code> → <code>constexpr</code> ;运行期只读用 <code>const</code>
窄化转换	<code>int x{300.5};</code> 编译错误;= 赋值截断
具名转换	<code>static_cast</code> 为主; <code>dynamic_cast</code> 多态下行
字符串	<code>std::string</code> : <code>size()</code> $O(1)$ 、 <code>==</code> 比内容、 <code>c_str()</code> 对接 C
动态数组	<code>std::vector</code> : <code>push_back/emplace_back</code> 、 <code>at()</code> 越界检查
键值对	<code>std::map</code> :有序、 <code>operator[]</code> 不存在即插入
强类型枚举	<code>enum class</code> :作用域隔离、显式转 <code>int</code>

最小必会清单

- 能说出 `int/long/long long/size_t` 在你平台上的字节数,并解释为何用固定宽度类型
- 能解释 `int a;`、`int a{;};`、`int a{0};` 三者的行为差异
- 能说清 `auto` 何时丢掉 `const` 与引用,并正确写出 `const auto&`
- 能写出二进制字面量、数字分隔符、原始字符串的语法
- 能区分 `const` 与 `constexpr`,知道数组大小需要哪种
- 能解释"列表初始化拒绝窄化"并举例,能说出四种具名转换的用途
- 能徒手写出 `std::string` 的拼接、查找、子串、替换代码
- 能用 `std::vector` 与 `std::map` 完成增删改查,知道 `at()` 与 `[]` 的区别
- 能写出 `enum class` 并说出它与 C 枚举的三个区别

自测题

Q 在 64 位 Windows 上, `sizeof(long)` 与 `sizeof(long long)` 分别是多少?跨平台需要 64 位整数时应该写什么?

✓ 参考答案

Windows(LLP64)上 `long` 是 4 字节, `long long` 是 8 字节。需要确定 64 位时写 `std::int64_t`。

Q 下面哪些行是编译错误?为什么? `double d = 3.14; int a{3.14}; int b = 3.14; int c{3};`

✓ 参考答案

`int a{3.14};` 是编译错误——列表初始化拒绝窄化。`int b = 3.14;` 能编译但截断为 3(兼容 C)。`int c{3};` 与 `double d = 3.14;` 正确。

Q `unsigned u = 1; int s = -1;` 表达式 `s < u` 的结果是什么?请解释。

✓ 参考答案

结果是 `false(0)`。混合比较时 `int` 被转换为 `unsigned`, -1 变成 4294967295,不小于 1。生产代码应避免有符号/无符号混比。

Q 写出下列代码中 a 到 d 的推导类型: `int x = 5; const int& r = x; auto a = r; auto& b = r; const auto c = r; auto* d = &x;`

✓ 参考答案

a 是 `int` (拷贝,剥掉引用与 `const`); b 是 `const int&`; c 是 `const int`; d 是 `int*`。

Q 用 `std::string` 把 `"apple,banana,cherry"` 按逗号切分,打印每段并统计总字符数。写出完整程序。

✅ 参考答案

```
#include <iostream>
#include <string>
#include <vector>
#include <sstream>

int main() {
    std::string line = "apple,banana,cherry";
    std::vector<std::string> parts;
    std::string token;
    std::istringstream ss(line);
    while (std::getline(ss, token, ',')) {
        parts.push_back(token);
    }
    std::size_t total = 0;
    for (const auto& p : parts) {
        std::cout << p << "\n";
        total += p.size();
    }
    std::cout << "总字符数: " << total << "\n";
    return 0;
}
```

章末实验题:学生信息登记与成绩统计

实验 1:学生信息登记与成绩统计

实验目标:写一个命令程序,登记学生信息并统计成绩,把全章的类型、初始化、常量、转换、字符串、容器、枚举、别名知识点全部用上。

实验要求:

- 任务 1:定义 `struct Student`,含学号(int)、姓名(std::string)、性别(bool)、班级(char)、成绩(double)(覆盖:结构体、内置类型、字符串)
- 任务 2:用 `constexpr` 定义及格线等常量,不用 `#define` (覆盖:编译期常量)
- 任务 3:用 `enum class Grade` 表示成绩等级,函数按分数返回等级(覆盖:强类型枚举)
- 任务 4:用 `std::vector` 存学生、`std::map` 统计各等级人数(覆盖:容器、range-for、结构化绑定)
- 任务 5:一律用 `{}` 初始化;算平均分、及格率时显式 `static_cast` (覆盖:列表初始化、类型转换)
- 任务 6:格式化输出总数、平均分、及格率、各等级人数(覆盖:to_string、setprecision)
- 任务 7:用 `auto` / `const auto&` 推导遍历变量,说出每处的推导类型(覆盖:auto 规则)

实验环境:任意 C++17 编译器(Visual Studio 2022 / MinGW g++ / clang++)。编译命令:g++ -std=c++17 main.cpp -o main

第一步 搭骨架:头文件、常量、枚举、结构体,编译通过后再加函数

第二步 `classify()` 函数与登记代码, `push_back` 传花括号初始化的临时对象

第三步 计与格式化输出;平均分先把 `size()` 转成 `double` 再除,避免整数除法;运行验证与手算一致

✅ 参考答案要点(第一部分:常量、枚举、结构体与函数)

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <iomanip>    // 输出格式化

// 任务 2:constexpr 常量,替代 #define
constexpr double kPassScore = 60.0;

// 任务 3:强类型枚举
enum class Grade { EXCELLENT, GOOD, PASS, FAIL };

Grade classify(double score) {           // 按分数返回等级
    if (score >= 90.0) return Grade::EXCELLENT;
    if (score >= 75.0) return Grade::GOOD;
    if (score >= kPassScore) return Grade::PASS;
    return Grade::FAIL;
}

// 任务 1:学生结构体
struct Student {
    int id;                // 学号
    std::string name;      // 姓名
    bool male;             // 性别:true=男
    char classCode;        // 班级代号 'A'/'B'
    double score;          // 成绩
};

// 附加任务:按学号查找,返回下标,未找到返回 -1
int findById(const std::vector<Student>& list, int targetId) {
    for (std::size_t i = 0; i < list.size(); ++i) {
        if (list[i].id == targetId) return static_cast<int>(i);
    }
    return -1;
}
```

✅ 参考答案要点(第二部分:main 主流程)

```
int main() {
    // 任务 4:vector 登记学生,{}-列表初始化
    std::vector<Student> students;
    students.push_back({1001, "张三", true, 'A', 92.5});
    students.push_back({1002, "李四", false, 'A', 78.0});
    students.push_back({1003, "王五", true, 'B', 55.5});
    students.push_back({1004, "赵六", false, 'B', 87.0});

    // 任务 5:统计,显式转换避免整数除法
    std::map<Grade, int> gradeCount;    // 等级 -> 人数
    double sum = 0.0;
    int passed = 0;
    for (const auto& s : students) {    // 任务 7:const auto& 避免拷贝
        sum += s.score;
        if (s.score >= kPassScore) ++passed;
        ++gradeCount[classify(s.score)];
    }
    double avg = sum / static_cast<double>(students.size());
    double passRate = static_cast<double>(passed) / students.size() * 100.0;

    // 任务 6:格式化输出
    std::cout << std::fixed << std::setprecision(2);
    std::cout << "总人数: " << students.size() << "\n";
    std::cout << "平均分: " << avg << "\n";
    std::cout << "及格率: " << passRate << "%\n";
    for (const auto& [grade, cnt] : gradeCount) {    // 结构化绑定遍历 map
        std::cout << "等级 " << static_cast<int>(grade) << " 人数: "
            << cnt << "\n";
    }

    // 附加任务:查找并打印
    int pos = findById(students, 1002);
    if (pos >= 0) std::cout << "学号 1002 是 " << students[pos].name << "\n";
    return 0;
}
```

设计说明:全程序零 `#define`、零 C 风格强转、零裸数组;先 `std::fixed` 再 `setprecision(2)` 保证两位小数。除法写 `sum / static_cast<double>(students.size())` 而非 `sum / students.size()`,是显式表达"我要浮点除法"——后者通常也能工作,但隐式转换意图不明,生产规范要求显式。

下一章预告:第2章 运算符与表达式——从 C 的运算符体系出发,讲清 C++ 的有符号溢出 UB、短路求值、优先级与求值顺序,以及 C++20 的三路比较运算符 `<=>`。

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

本章学习目标

- 掌握 C++ 全部运算符,说出与 C 的对应关系
- 理解溢出:有符号 UB、unsigned 回绕、整数除零也是 UB
- 理解短路、三目、复合赋值的真实语义
- 掌握位运算生产场景:标志位、掩码、快速乘除 2 的幂
- 掌握浮点比较的正确姿势,理解 NaN 的坑
- 会用 C++20 三路比较与 auto 比较,写出可读的比较逻辑

从 C 到 C++:本章主题如何迁移

运算符是 C++ 从 C 继承最彻底的部分:算术、位、逻辑、比较、赋值几乎 1:1 保留,坑也全部保留:有符号溢出仍是 UB,求值顺序仍未规定。C++20 引入三路比较,把 strcmp 式的"负数/零/正数"约定提升为语言级语义。

学完本章你会形成两个习惯:把"会不会是 UB"当作写每行表达式的默认提问;把复杂比较交给三路比较与标准库。这正是 C 工程师进阶 C++ 的分水岭。

前置知识自查

- C 的算术运算符、除法截断与取模符号规则
- C 的位运算符、逻辑运算符与短路求值
- sizeof、逗号表达式、三目运算符 ?:
- C 的运算符优先级表(乘法高于加法、移位低于算术)
- 头文件与基本 I/O

本章学习路线

1. 第一阶段(熟悉):§2.1-§2.2 算术与自增自减,理解"UB 文化"在运算符上的体现
2. 第二阶段(重点):§2.3-§2.5 位运算、短路、比较,生产代码出错率最高的区域
3. 第三阶段(进阶):§2.6-§2.8 三路比较、优先级表、求值顺序,做自测题与章末实验
4. 验收标准:能背优先级口诀,能解释 `i++ + i++` 是 UB,能写标志位权限代码

本章知识导图

- §2.1 算术:溢出(UB vs 回绕)、除法截断、负数取模
- §2.2 自增自减:前置/后置语义、++ 是表达式
- §2.3 位运算:全套运算符、标志位、有符号右移
- §2.4 逻辑与短路:&& || !、短路保证、& 与 && 之别
- §2.5 比较:值比较、浮点相等、NaN
- §2.6 三路比较:C++20 的 <=> 与 auto 比较
- §2.7 优先级与求值顺序:优先级表、未定序 UB
- §2.8 三目与复合赋值?: 类型规则、+= 只求值一次

§2.1 算术运算:溢出、除法与取模

C++ 的算术运算符 + - * / % 与 C 完全一致,行为不变。下面这张对照表是本章最重要的一张表。

C 的写法/行为	C++ 的写法/行为	差异
有符号溢出:UB	有符号溢出:UB	完全相同,编译器可做任意优化
unsigned 回绕	unsigned 回绕	相同,按模 2^N 运算,有定义
整数除法向零截断	整数除法向零截断	相同(C99 起);负数 $-7/2 == -3$
取模符号同被除数	取模符号同被除数	相同; $-7 \% 3 == -1$
整数除零:UB	整数除零:UB	相同,不是异常,程序行为未定义
无幂运算符	无幂运算符	相同;2 的幂用 <<,其他用 std::pow

有符号溢出是 UB,编译器认为"这不可能发生"并据此优化:优化器可能把 $x + 1 > x$ 直接判定为恒真。生产规则:可能溢出的运算先检查边界或用 int64_t 兜底,并用 UBSan 扫描。unsigned 回绕定义明确: $0xFFFFFFFF + 1$ 得到 0,协议序号、校验和都依赖它。

⚠ 易错点 1:整数除零是 UB,不是异常

C/C++ 的整数除零不会抛异常、不会返回 0,行为未定义——常见结果是程序崩溃,但标准不保证任何结果。浮点除零则安全:得到 inf 或 NaN。生产规则:除数来自外部输入时先判零再运算。

示例 2-1 溢出、回绕与除法取模

```

#include <iostream>
#include <climits>

int main() {
    // unsigned 回绕:有定义,按模 2^32 计算
    unsigned u = 0xFFFFFFFFu;
    std::cout << "回绕: " << (u + 1) << "\n";      // 0

    // 有符号溢出:UB,勿依赖
    int imax = INT_MAX;
    std::cout << "有符号溢出(UB,勿依赖): " << (imax + 1) << "\n";

    // 除法向零截断,取模符号同被除数
    std::cout << "-7 / 2 = " << (-7 / 2) << "\n";    // -3
    std::cout << "-7 % 3 = " << (-7 % 3) << "\n";    // -1
    std::cout << "7 % -3 = " << (7 % -3) << "\n";    // 1

    // 整数除零:UB,仅注释展示,勿执行
    // int bad = 100 / 0;
    return 0;
}

```

注意与 Python 的区别:Python 的 % 结果与除数同号(-7 % 3 == 2),C/C++ 与被除数同号,跨语言移植取模代码要显式处理。

§2.2 自增自减:前置与后置

C 的写法	C++ 的写法	差异
i++ 后置,返回旧值	i++ 后置,返回旧值	完全相同
++i 前置,返回新值	++i 前置,返回新值	完全相同
++ 是表达式	++ 是表达式	相同;Go 中 ++ 是语句,不可嵌套
i++++ 不合法	i++++ 不合法	相同,后置自增的结果不是左值

自增自减与 C 完全一致,记住三点:后置返回旧值的拷贝;后置结果不是左值,i++ = 0 不合法;单独成句时前后缀等价。生产习惯:循环统一写 ++i——类类型上后置自增会构造临时对象,习惯统一可避免将来泛型化踩坑。

示例 2-2 前置与后置的区别

```

#include <iostream>

int main() {
    int i = 0;
    int a = i++;      // a = 0, 然后 i = 1(后置先取旧值)
    int b = ++i;      // i = 2, 然后 b = 2(前置先自增)
    std::cout << "a = " << a << ", b = " << b << ", i = " << i << "\n";

    // 自增结果不是左值,以下写法不合法
    // i++ = 5;
    return 0;
}

```

§2.3 位运算:全套运算符与生产场景

C++ 提供与 C 相同的六种位运算:& | ^ ~ << >>。与 Java/JS 不同,C++ 没有 >>>:无符号类型右移天然逻辑移位(高位补零);有符号类型 C++20 起明确规定是算术移位,C++17 及以前是实现定义,但主流编译器早已统一为算术移位。

C 的写法	C++ 的写法	差异
& ^ ~ << >>	& ^ ~ << >>	运算符完全相同
有符号右移:实现定义	有符号右移:C++20 定为算术移位	C++20 起语义确定
无 >>>	无 >>>	与 Java/JS 不同;无符号右移用无符号类型
位运算结果类型	位运算结果类型	相同:整型提升后运算,无符号溢出回绕

⚠ 易错点 2:有符号类型的移位陷阱

对负数右移是算术移位(向下取整的除法):-8 >> 1 == -4。负数左移曾是 UB,即便 C++20 已定义其结果,位运算也只对无符号类型做。~x == -(x+1),~0 == -1,别把 ~ 当逻辑非。

生产场景:标志位与权限集合是位运算最常见的用途——| 组合、& 判断、^ 切换、~ 掩码。还常用于哈希、位图、协议解析与快速乘除 2 的幂。注意:位运算不做短路,两个操作数都会求值。

示例 2-3 权限标志位的实现

```
#include <iostream>

// 权限位:每比特一个权限(C 的老办法,C++ 依然常用)
constexpr unsigned kRead    = 1u << 0;    // 0b001
constexpr unsigned kWrite   = 1u << 1;    // 0b010
constexpr unsigned kExecute = 1u << 2;    // 0b100

int main() {
    unsigned perm = kRead | kWrite;        // 组合权限
    std::cout << (perm & kRead) << "\n";    // 非零:有读权限
    std::cout << (perm & kExecute) << "\n"; // 0:无执行权限

    perm ^= kWrite;                        // 切换:去掉写权限
    std::cout << (perm & kWrite) << "\n";    // 0

    perm |= kExecute;                      // 加执行权限
    perm &= ~kRead;                        // 掩码去掉读权限
    std::cout << (perm & (kExecute | kRead)) << "\n"; // 只余执行位

    // 移位即乘除 2 的幂:x<<2==x*4,x>>1==x/2(无符号)
    unsigned x = 7;
    std::cout << (x << 2) << " " << (x >> 1) << "\n"; // 28 3
    return 0;
}
```

§2.4 逻辑运算与短路求值

&& 与 || 的短路语义与 C 完全一致:左侧决定结果时,右侧不再求值。短路是语言保证而非优化,它让"指针非空再解引用"安全成立。小差异:逻辑表达式结果类型是 bool。

C 的写法	C++ 的写法	差异
a && b 结果是 int	a && b 结果是 bool	C++ 结果类型是 bool
短路求值	短路求值	完全相同,语言保证
if (p) 指针非零即真	if (p) 指针非零即真	相同;条件上下文允许指针与整数
& 与 && 是不同运算符	& 与 && 是不同运算符	相同;常被误用

⚠ 易错点 3:用 & 代替 && 会丢失短路

if (p && p->x) 安全,if (p & p->x) 是灾难:& 先求值两侧,空指针解引用直接 UB。a & b 的结果是数值而非真值,评审中把 & 写成 && 是高频错误;两者优先级不同(& 高于 &&),混用要加括号。

示例 2-4 短路求值的两种场景

```
#include <iostream>

int main() {
    int n = 0;
    // 短路:左侧 false,右侧 10/n 不执行,不会除零
    bool safe = (n != 0) && (10 / n > 1);
    std::cout << "safe = " << safe << "\n";    // 0

    // || 左侧为 true 时右侧不求值:常用于给默认值
    const char* env = nullptr;
    const char* value = env ? env : "默认值";    // 判空兜底
    std::cout << value << "\n";

    // 副作用:短路使右侧副作用不发生
    int cnt = 0;
    bool r = (n != 0) || (++cnt != 0);    // 右侧求值,cnt 变 1
    bool r2 = (n == 0) || (++cnt != 0);    // 左侧已 true,右侧不求值
    std::cout << "cnt = " << cnt << ", r = " << r << ", r2 = " << r2 << "\n";
    return 0;
}
```

§2.5 比较运算与浮点相等

== != < <= > >= 都是值比较,与 C 相同:数组名比较的是地址,结构体不能整体比较(C++20 可自动生成,见下节)。浮点相等是经典陷阱:0.1 + 0.2 != 0.3;NaN 不等于任何值,包括它自己。

C 的写法	C++ 的写法	差异
<code>a == b</code> 值比较	<code>a == b</code> 值比较	完全相同
NaN 不等于任何值	NaN 不等于任何值	相同;判断 NaN 用 <code>isnan()</code>
浮点相等直接 <code>==</code>	浮点相等直接 <code>==</code>	相同;都需要 <code>epsilon</code> 或整数化
结构体不能 <code>==</code>	结构体不能 <code>==(C++20 可自动生成)</code>	C++20 用 <code><=></code> 或默认比较

⚠ 易错点 4:浮点相等比较

if (`a == b`) 对浮点数几乎总是错的:`0.1 + 0.2 == 0.3` 是 `false`。正确姿势:误差允许时用阈值比较;能整数化就整数化(金额用分、距离用毫米)。生产规范:浮点相等比较必须写注释说明误差来源。

示例 2-5 浮点比较的正确姿势

```
#include <iostream>
#include <cmath>

// 误差阈值按数量级缩放,不用绝对 1e-9
bool nearlyEqual(double a, double b, double eps = 1e-9) {
    return std::fabs(a - b) <= eps * std::max({1.0, std::fabs(a), std::fabs(b)});
}

int main() {
    double x = 0.1 + 0.2;
    std::cout << (x == 0.3) << "\n";           // 0:经典陷阱
    std::cout << nearlyEqual(x, 0.3) << "\n"; // 1:阈值比较
    std::cout << (x < 0.3) << "\n";           // 1:0.30000000000000004

    // NaN 不等于任何值,包括它自己
    double nan = std::nan("");
    std::cout << (nan == nan) << "\n";         // 0
    std::cout << std::isnan(nan) << "\n";     // 1:判断 NaN 用 isnan

    // 能整数化就整数化:金额用"分"存整数
    long long cents = 125;                     // 1.25 元
    std::cout << (cents / 100.0) << "\n";     // 展示时才除 100
    return 0;
}
```

§2.6 C++20 三路比较运算符:告别 strcmp 时代

C 时代比较两个对象要靠 `strcmp`/自己的 `compare` 函数,返回负数/零/正数,调用方再写 if (`cmp < 0`)。C++20 的三路比较运算符 `<=>`(星船运算符)把"三态比较"提升为语言语义:`a <=> b` 返回序类型(`strong_ordering` / `partial_ordering` / `weak_ordering`),结果要么是序值要么是相等值,零比较的正确性由语言保证。

C 的写法	C++ 的写法	差异
strcmp(a, b) 返回 <0/0/>0	a <=> b 返回序类型	C++ 把约定变成类型
手写 6 个比较函数	= default 自动生成 6 个	C++20 声明式比较
用 memcmp 比较结构体	结构体直接 == <	C++ 按成员逐字段比较
无	float 比较得 partial_ordering	NaN 参与时是"不可比较",安全

示例 2-6 三路比较自动生成全部比较运算符

```
#include <iostream>
#include <compare>          // C++20:std::strong_ordering 等

// = default:自动生成六个比较运算符
struct Version {
    int major;
    int minor;
    auto operator<=>(const Version&) const = default;
};

int main() {
    Version v1{2, 10};
    Version v2{2, 9};

    // 直接比较,不用再写 compare
    std::cout << (v1 < v2) << (v1 == v2) << (v1 > v2) << "\n"; // 0 0 1

    // 三路比较的返回:序类型,而非 int
    std::strong_ordering ord = v1 <=> v2;
    std::cout << (ord < 0) << (ord == 0) << (ord > 0) << "\n"; // 0 0 1

    // 浮点三路比较:partial_ordering,NaN 参与时不可比较
    double a = 0.3, b = 0.1 + 0.2;
    std::partial_ordering po = a <=> b;
    std::cout << (po == std::partial_ordering::equivalent) << "\n"; // 0
    return 0;
}
```

生产建议:C++20 项目中,按字典序比较的纯数据类直接声明 `auto operator<=> 并 = default`,六个比较运算符自动一致。C++17 及以下用 `friend` 函数或宏生成。注意:裸数组不能自动比较,`std::array` 可以;含指针成员的结构体比较的是地址。

§2.7 优先级、结合性与求值顺序

C++ 的优先级表与 C 相同,口诀:括号后缀、一元、乘除加减、移位、关系相等、位与异或或、逻辑与或、三目、赋值、逗号。

优先级	运算符(同格内结合性从左到右)
1(最高)	() 下标 . -> ++ -- (后缀) ::
2	! ~ + - * & (一元) ++ -- (前缀) (类型)强转 sizeof new
3	* / %
4	+ -
5	<< >>
6	< <= > >=
7	== !=
8	&
9	^
10	
11	&&
12	
13	?:
14	= += -= *= /= %= <<= >>= &= = ^=
15(最低)	, 逗号运算符

优先级决定"谁先结合",求值顺序决定"谁先算出值",两者无关。C++ 与 C 相同:操作数与函数实参的求值顺序大多未规定。i++ + i++ 两次自增读写同一变量且无顺序点,是 UB;f(a(), b()) 的实参调用顺序也未规定(Java、C# 则从左到右)。生产规则:一条语句只做一件事,同一变量在一个表达式里最多一个副作用。



易错点 5:未定序求值是 UB 的重灾区

printf("%d %d", i, i++); 在 C 里常见,在 C++ 里同样是 UB:实参求值顺序未规定。即使输出"看起来稳定",编译器也可任意重排。修复:拆成两条语句,先自增再传值。

示例 2-7 求值顺序:UB 与正确写法

```

#include <iostream>

int sideEffect(int v) {
    std::cout << "调用 sideEffect(" << v << ")\n";
    return v;
}

int main() {
    // UB:两次自增未定序,程序行为未定义
    // int x = 1;
    // int r = x++ + x++;

    // 同样危险:实参求值顺序未规定
    // std::cout << sideEffect(1) + sideEffect(2) << "\n";

    // 正确:拆成语句,顺序明确
    int x = 1;
    int a = x++;
    int b = x++;
    std::cout << "a + b = " << (a + b) << "\n";    // 1 + 2 = 3

    int s = sideEffect(1);
    s += sideEffect(2);
    std::cout << "s = " << s << "\n";
    return 0;
}

```

§2.8 三目运算符与复合赋值

?: 与复合赋值都从 C 原样继承。三目是表达式,分支类型要一致或可隐式转换,长表达式务必加括号。复合赋值的要点:左操作数只求值一次—— $a[i++] += 1$ 安全,而 $a[i++] = a[i++] + 1$ 求值两次,是未定义行为。能用复合赋值就不用展开写法。

示例 2-8 三目与复合赋值

```
#include <iostream>

int main() {
    // 三目:两个分支类型一致时是表达式
    int score = 61;
    const char* verdict = (score >= 60) ? "及格" : "不及格";
    std::cout << verdict << "\n";

    // 复合赋值:左操作数只求值一次
    int arr[3] = {10, 20, 30};
    int i = 0;
    arr[i++] += 5;      // 只对 i++ 求值一次:arr[0] = 15,i = 1
    std::cout << arr[0] << " " << i << "\n";

    // 三目可嵌套但可读性差,生产不这么写
    int grade = 85;
    const char* level = grade >= 90 ? "优" : grade >= 75 ? "良" : "差";
    std::cout << level << "\n";
    return 0;
}
```

本章速查表

主题	速查内容
有符号溢出	UB;先检查边界或 int64_t;UBSan 检测
unsigned 回绕	有定义,模 2^N ;序号、校验和放心用
整数除零	UB,不是异常;浮点除零得 inf/NaN
除法/取模	向零截断;取模符号同被除数($-7\%3=-1$)
自增自减	$++i$ 返回新值, $i++$ 返回旧值;结果非左值
位运算	$\& ^ \sim < < > >$;无 $> > >$;右移用无符号
标志位	$ $ 组合、 $\&$ 判断、 $^$ 切换、 $\sim$ 掩码;常量 $1u < < n$
短路	$\&\& $ 保证短路;别用 $\&$ 代替 $\&\&$
逻辑结果	$\&\& !$ 返回 bool;条件上下文允许指针
浮点相等	禁止 $==$;用相对误差阈值或整数化
NaN	不等于任何值;用 std::isnan 判断
三路比较	C++20:operator<=> = default 生成 6 个比较
优先级	后缀>一元>乘除>加减>移位>关系>相等> $\&>^> >\&\&> >?:>$ 赋值>逗号
求值顺序	操作数顺序未规定;一表达式内同变量只一个副作用
复合赋值	左操作数只求值一次, $a[i++] += 1$ 安全

最小必会清单

- 能解释有符号溢出是 UB、unsigned 回绕,并给出检测手段
- 能说出 $-7 / 2$ 与 $-7 \% 3$ 的值及与 Python 取模的差异

- 能写出权限标志位的组合/判断/切换/掩码代码
- 能说明短路求值的语言保证与 `&` 和 `&&` 的差别
- 能写出浮点比较的正确函数,能解释 NaN 的 `==` 行为
- 能用 `<=> = default` 让结构体获得全部比较运算符
- 能背出优先级口诀,能解释 `i++ + i++` 为何是 UB
- 能说明复合赋值为何只求值一次

自测题

Q `int i = 1; 表达式 i++ + i++ 的值是多少?`

✓ 参考答案

没有答案——这是未定义行为:两次自增作用于同一变量且未定序,编译器可任意重排。正确回答是"UB,不能这么写"。

Q 写出 `-7 / 2`、`-7 % 3`、`7 % -3` 在 C++ 中的结果,并说明取模符号规则。

✓ 参考答案

分别是 -3、-1、1。整数除法向零截断,取模符号与被除数一致。Python 中 `-7 % 3 == 2`(与除数同号),移植时要注意。

Q `bool a = 1 && 1; bool b = 1 & 1; bool c = (2 < 1) || (1 < 2);` 三个变量分别是多少?`&` 与 `&&` 的关键区别是什么?

✓ 参考答案

三者都是 true。区别:`&&` 短路且结果为 bool;`&` 不短路、结果是整数。指针判空后用 `&` 会解引用空指针。

Q `double a = 0.1 + 0.2;` 判断 `a == 0.3` 会得到什么?给出两种正确的比较方案。

✓ 参考答案

得到 false:0.1 与 0.2 在二进制浮点中都是近似值。方案一:相对误差阈值比较;方案二:整数化——金额用分、距离用毫米,全程整数运算,展示时才转浮点。

Q 给 Version 结构体(含 major、minor 两个 int 成员)声明比较运算符,使 `v1 < v2`、`v1 == v2`、`v1 > v2` 都能编译。

✓ 参考答案

结构体内写 `auto operator<=>(const Version&) const = default;` 一个声明自动生成六个比较运算符,按成员声明顺序逐字段比较。

章末实验题:表达式求值计算器

实验 2:表达式求值计算器

实验目标:写一个命令行程序:读入表达式,用"中缀转后缀(调度场算法)+ 栈求值"输出结果,覆盖全章运算符知识点。

实验要求:

- 任务 1:词法器把输入切成数字与运算符,支持 $+$ $-$ $*$ $/$ $\%$ $\&$ $|$ $^$ $\sim$ $<<$ $>>$ $\&\&$ $||$ $!$ $==$ $!=$ $<$ $<=$ $>$ $>=$ 与括号(覆盖:算术/位/逻辑/比较/一元运算符)
- 任务 2:用 `std::map` 存优先级表,数字越大越优先(覆盖:优先级、`map`)
- 任务 3:调度场算法转后缀,处理括号与一元负号(覆盖:结合性;后缀消除未定序求值)
- 任务 4:用 `std::vector` 当栈做后缀求值(覆盖:栈、`int64_t` 与回绕)
- 任务 5:main 演示短路、三目、自增、负数取模与回绕,解释 $++$ 与 $?:$ 为何不进求值器(覆盖:短路/三目/自增/取模/溢出)
- 任务 6:手动验证: $3 + 4 * 2$ 得 11; $(1 < 4) - 6$ 得 10; $-7 \% 3$ 得 -1
- 任务 7:用 `-Wall -Wextra` 编译,全程零警告

实验环境:任意 C++17 编译器。编译命令:`g++ -std=c++17 -Wall -Wextra calc.cpp -o calc`

第一步 写词法器与优先级表,编译通过后再写调度场

第二步 写求值器,先支持 $+$ $-$ $*$ $/$ 与括号,验证无误后加位运算与逻辑运算

第三步 后写 main 演示,运行算例与手算核对

✅ 参考答案(第一部分:优先级表与词法器)

```
// 表达式求值计算器(第一部分:词法器)
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <cctype>

// 任务 2:优先级表,数字越大越优先;NEG 是最高优先级的一元负号
std::map<std::string, int> kPrec = {
    {"|", 1}, {"&&", 2},
    {"|", 3}, {"^", 4}, {"&", 5},
    {"==", 6}, {"!=", 6},
    {"<", 7}, {"<=", 7}, {">", 7}, {">=", 7},
    {"<<", 8}, {">>", 8},
    {"+", 9}, {"-", 9},
    {"*", 10}, {"/", 10}, {"%", 10},
    {"NEG", 100}, {"!", 100}, {"~", 100},
};

bool isOp(const std::string& t) { return kPrec.count(t) > 0; }

// 任务 1:词法器;'!' '~' 与"需要操作数处的 -"作一元前缀运算符
std::vector<std::string> tokenize(const std::string& expr) {
    std::vector<std::string> out;
    for (std::size_t i = 0; i < expr.size(); ) {
        char c = expr[i];
        if (std::isspace(static_cast<unsigned char>(c))) { ++i; continue; }
        if (std::isdigit(static_cast<unsigned char>(c))) {
            std::size_t j = i;
            while (j < expr.size() &&
                std::isdigit(static_cast<unsigned char>(expr[j]))) ++j;
            out.push_back(expr.substr(i, j - i));
            i = j;
            continue;
        }
        std::string two = expr.substr(i, 2);
        if (isOp(two)) { out.push_back(two); i += 2; continue; }
        if (c == '-' && (out.empty() || out.back() == "(" || isOp(out.back()))) {
            out.push_back("NEG"); ++i; continue; // 一元负号
        }
        out.push_back(std::string(1, c)); // 单字符运算符或括号
        ++i;
    }
    return out;
}
```

✅ 参考答案(第二部分:调度场转换与栈求值)

```
// 第二部分:中缀转后缀(调度场算法)
// 任务 3:调度场算法。后缀表达式的运算顺序确定,天然消除未定序 UB
std::vector<std::string> toRpn(const std::vector<std::string>& tokens) {
    std::vector<std::string> out, ops;
    for (const std::string& t : tokens) {
        if (t == "(") { ops.push_back(t); continue; }
        if (t == ")") {
            while (!ops.empty() && ops.back() != "(") {
                out.push_back(ops.back());
                ops.pop_back();
            }
            if (!ops.empty()) ops.pop_back(); // 弹出 '('
            continue;
        }
        if (isOp(t)) {
            if (t == "NEG" || t == "!" || t == "~") {
                ops.push_back(t); // 前缀运算符:直接压栈
            } else {
                while (!ops.empty() && ops.back() != "(" &&
                    kPrec[ops.back()] >= kPrec[t]) {
                    out.push_back(ops.back());
                    ops.pop_back();
                }
                ops.push_back(t); // 同优先级左结合:先弹后压
            }
            continue;
        }
        out.push_back(t); // 数字
    }
    while (!ops.empty()) { out.push_back(ops.back()); ops.pop_back(); }
    return out;
}
```

✅ 参考答案(第三部分:栈求值)

```
// 任务 4:栈求值。int64_t 计算,位运算按无符号语义回绕
long long applyOp(long long a, long long b, const std::string& op) {
    switch (op[0]) {
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        case '/': return b == 0 ? 0 : a / b;    // 除零防御,生产应报错
        case '%': return b == 0 ? 0 : a % b;    // 取模符号同被除数
    }
    if (op == "&") return a & b;
    if (op == "|") return a | b;
    if (op == "^") return a ^ b;
    if (op == "<<") return a << b;
    if (op == ">>") return a >> b;
    if (op == "&&") return (a != 0 && b != 0) ? 1 : 0;
    if (op == "||") return (a != 0 || b != 0) ? 1 : 0;
    if (op == "==") return a == b;
    if (op == "!=") return a != b;
    if (op == "<") return a < b;
    if (op == "<=") return a <= b;
    if (op == ">") return a > b;
    if (op == ">=") return a >= b;
    return 0;
}

long long evalRpn(const std::vector<std::string>& rpn) {
    std::vector<long long> st;
    for (const std::string& t : rpn) {
        if (!isOp(t)) { st.push_back(std::stoll(t)); continue; }
        if (t == "NEG") { st.back() = -st.back(); continue; }
        if (t == "!") { st.back() = st.back() == 0 ? 1 : 0; continue; }
        if (t == "~") { st.back() = ~st.back(); continue; }
        long long b = st.back(); st.pop_back();
        long long a = st.back(); st.pop_back();
        st.push_back(applyOp(a, b, t));
    }
    return st.back();
}
```

✅ 参考答案(第四部分:main 主流程与知识点演示)

```
// 第四部分:main 主流程
int main() {
    std::string line;
    std::cout << "输入表达式(如 3 + 4 * 2): ";
    std::getline(std::cin, line);

    std::vector<std::string> tokens = tokenize(line);
    std::vector<std::string> rpn = toRpn(tokens);
    std::cout << "后缀: ";
    for (const std::string& t : rpn) std::cout << t << " ";
    std::cout << "\n结果: " << evalRpn(rpn) << "\n";

    // 任务 5:短路演示(&& 左侧为假,右侧不求值)
    int n = 0;
    bool safe = (n != 0) && (10 / n > 1);
    std::cout << "短路演示: " << safe << "\n";

    // 任务 5:负数取模与回绕(与 C 完全一致)
    std::cout << "-7 % 3 = " << (-7 % 3) << "\n";
    unsigned mx = 0xFFFFFFFFu;
    std::cout << "unsigned 回绕: " << (mx + 1) << "\n";

    // 任务 5:三目与前置自增(求值器语法不含?: 与 ++:
    // 前者需三操作数,后者是赋值语义,工程求值器同样如此)
    for (int i = 0; i < 3; ++i) {
        std::cout << "第" << i << "次:" << (i == 2 ? "末位" : "普通") << " ";
    }
    std::cout << "\n";
    return 0;
}
```

设计说明:调度场算法在转后缀时"按优先级出栈",最终运算顺序完全确定——这是生产环境规避求值顺序 UB 的惯用手法。求值用 long long;除零做了防御返回 0,生产代码应改为报错退出。

下一章预告:第3章 流程控制——if/else 的条件类型要求、switch 与 C 的差别、范围 for 循环、循环中的迭代器失效,以及 goto 在 C++ 中的正确归宿。

第3章 流程控制

建议学时:3-5 小时 | 难度:★★☆☆☆ | 读者背景:已掌握 C 语言

本章学习目标

- 掌握 if/else、switch、三种循环与 C 的异同,特别是条件类型与悬挂 else
- 掌握范围 for 循环与结构化绑定,理解它的底层是迭代器
- 理解迭代器失效的四种场景,掌握 erase-remove 惯用法
- 掌握 break/continue/goto 的语义与限制,理解 [[fallthrough]] 属性
- 掌握 C++17 的 if/switch 初始化语句,写出作用域更小的代码

从 C 到 C++:本章主题如何迁移

流程控制是 C 工程师最熟悉的地盘:if、for、while、switch、goto,语法几乎原样保留。C++ 的改动集中在三处:新增范围 for 让遍历容器的代码短了一半;新增 if/switch 初始化语句让"临时变量只在分支内可见";以及容器时代带来的新坑——迭代器失效,这是裸数组时代根本不存在的概念。另外,switch 的 case 穿透在 C++ 里有了官方标注方式 [[fallthrough]],代码评审不再靠猜。

学完本章,你的循环代码应该默认写成"范围 for + const auto&"或"迭代器 + 下标"两种风格,并且能清楚地解释为什么不能在遍历时往 vector 里插入元素。

前置知识自查

- C 的 if/else、switch、for/while/do-while 与 break/continue/goto
- C 的悬挂 else 规则与 switch 穿透行为
- 第1章的 std::vector 与 std::map 基本操作
- 第2章的运算符优先级与短路求值

本章学习路线

1. 第一阶段(熟悉):§3.1-§3.2 条件与分支,重点理解条件类型要求与 case 穿透
2. 第二阶段(重点):§3.3-§3.4 三种循环与范围 for,把遍历习惯改过来
3. 第三阶段(难点):§3.5 迭代器失效,这是容器时代的必修课
4. 验收标准:能解释 vector 扩容为何使迭代器失效,能写出 erase-remove 惯用法

本章知识导图

- §3.1 if/else:条件类型、悬挂 else、if 初始化语句
- §3.2 switch:case 穿透、[[fallthrough]]、switch 初始化语句
- §3.3 三种经典循环:for/while/do-while
- §3.4 范围 for:底层迭代器、const auto&、结构化绑定
- §3.5 迭代器失效:扩容、删除、erase-remove 惯用法
- §3.6 break/continue/goto 与 [[fallthrough]]

§3.1 if/else:条件与分支

if/else 的语法与 C 完全一致,包括悬挂 else 规则:else 与最近的未配对 if 结合。看代码时先数花括号再读逻辑,否则 else 会配对到想不到的 if 上。与 C 相同,C++ 的条件上下文接受数值与指针(非零即真),与 Go、Rust 要求条件必须是 bool 不同——这也是 C 家族容易写出 if (x = 1) 的原因:赋值表达式的值就是被赋的值,恒为真,编译器通常只给警告。生产规范:单行 if 也要写花括号;条件里禁止赋值;判定"相等"永远写 x == 1,并把常量放左边(1 == x)可让误写赋值直接编译失败。

C 的写法	C++ 的写法	差异
条件允许数值/指针	条件允许数值/指针	相同;Go/Rust 必须 bool
悬挂 else 就近配对	悬挂 else 就近配对	完全相同
无	if (init; cond) 初始化语句	C++17 新增,变量限分支内
if (x = 1) 是赋值	if (x = 1) 是赋值	相同;想写比较先写 x == 1

C++17 的 if 初始化语句是生产中常用的小改进:if (auto it = m.find(k); it != m.end()) 把查找与判断写在一起,变量作用域限定在分支内,不会泄漏到函数体。注意初始化语句与条件之间用分号分隔,条件可以是任意表达式。同样的写法也适用于 while 循环:while (auto ch = getchar(); ch != EOF) 在每次迭代重新求值条件。习惯这个写法后,你会发现大量"临时变量+判断"的两行代码能压缩成一行,且不会忘记清理。

示例 3-1 条件与 if 初始化语句

```

#include <iostream>
#include <string>

int main() {
    int score = 85;
    if (score >= 90) {
        std::cout << "优秀\n";
    } else if (score >= 60) {
        std::cout << "及格\n";        // 悬挂 else 与 C 相同:就近配对
    } else {
        std::cout << "不及格\n";
    }

    // C++17:if 初始化语句,变量只活在分支里
    if (int pass = score - 60; pass >= 0) {
        std::cout << "超出及格线 " << pass << " 分\n";
    }
    // 此处 pass 不可见

    // 指针直接作条件:非零即真(与 C 相同)
    std::string* p = nullptr;
    if (p) {
        std::cout << "非空\n";
    } else {
        std::cout << "空指针\n";
    }
    return 0;
}

```

§3.2 switch:case 穿透与 fallthrough

switch 的规则与 C 相同:case 标签必须是整型常量表达式(枚举可以),执行到 break 才退出,否则向下穿透。C++ 与 C 的三点差异:第一,switch 内不能跨过带初始化的变量声明——case 2: 里 int x = 0; 而 case 1: 跳到 case 2 会越过初始化,是编译错误,解决办法是给 case 加花括号;第二,C++17 起可用 `[[fallthrough]]` 显式标注"故意穿透",替代 C 时代的裸注释,配合 `-Weimplicit-fallthrough` 编译器还能帮你抓漏写的 break;第三,C++17 起 switch 同样支持初始化语句。

工程习惯:case 少于 5 个时 switch 清晰可读,再多建议换查找表(数组或 `std::map` 按值取结果)——第4章讲函数指针后会给出具体写法。每个 switch 都写 default 分支,枚举的 switch 尤其要处理将来新增枚举值的情况,编译器配合 `-Wswitch` 可以警告漏掉的枚举分支。

C 的写法	C++ 的写法	差异
case 穿透,裸注释	case 穿透, <code>[[fallthrough]]</code> 标注	C++17 属性,编译器可检查
case 里可声明变量	case 里声明变量需加花括号	C++ 更严格:禁止跨初始化跳转
无	switch (init; 表达式)	C++17 新增

示例 3-2 switch 与 `[[fallthrough]]`

```

#include <iostream>

const char* weekday(int day) {
    switch (day) {                // case 必须是整型常量表达式
        case 1: return "周一";
        case 2: return "周二";
        case 3: return "周三";
        case 4: return "周四";
        case 5: return "周五";
        case 6:
        case 7: return "周末";    // 隐式穿透:case 6 落入 case 7
        default: return "非法";
    }
}

int main() {
    // 穿透演示:6 和 7 都得到"周末"
    for (int d = 1; d <= 7; ++d) {
        std::cout << d << ":" << weekday(d) << " ";
    }
    std::cout << "\n";

    // [[fallthrough]]:显式声明故意穿透(C++17)
    int x = 2;
    switch (x) {
        case 1:
            std::cout << "一";
            [[fallthrough]];      // 告诉编译器与读者:不是忘了 break
        case 2:
            std::cout << "二\n";
            break;
        default:
            std::cout << "其他\n";
    }

    // case 内声明变量必须加花括号,否则编译错误(注释展示)
    // switch (x) {
    //     case 1: int v = 0; break;    // 错误:可跨过初始化
    //     case 2: break;
    // }
    return 0;
}

```

⚠ 易错点 3:do-while 的分号与 while 条件副作用

do-while 的 while (条件); 结尾分号不能丢——丢了分号,后面的语句会被当成循环体的一部分,行为完全错乱。另一个高频错误:在 while 条件里写带副作用的表达式,如 while (getline(cin, line)) 没问题,但 while (i++ < n && ...) 这类写法在维护时极易出错,条件里的变量每次迭代都在变,调试困难。生产规范:条件里只做判断,副作用放到循环体内显式写。

§3.3 三种经典循环

for/while/do-while 与 C 完全相同,包括初始化与迭代子句里的逗号表达式。生产习惯:循环变量用 int 还是 std::size_t 要区分——遍历容器下标建议 std::size_t(避免与 size() 的无符号比较警告),计数次数明确小时用 int;死循环统一写 for(;;),不要写 while(1);do-while 至少执行一次,适合菜单与输入重试;循环体内不变的表达式提到循环外,如 size() 在迭代中不变时可缓存,但注意缓存与"每次重新取"在容器被修改时结果不同,先想清楚再优化。

示例 3-3 三种循环与死循环

```
#include <iostream>

int main() {
    // for:与 C 完全相同的三段式
    int sum = 0;
    for (int i = 1; i <= 100; ++i) {
        sum += i;
    }
    std::cout << sum << "\n";    // 5050

    // while:先判后做
    int n = 5, cnt = 0;
    while (n > 0) { n >>= 1; ++cnt; }
    std::cout << cnt << "\n";    // 3

    // do-while:至少执行一次,菜单程序的标配
    int choice = 0;
    do {
        std::cout << "选 1-3: ";
        std::cin >> choice;
    } while (choice < 1 || choice > 3);
    std::cout << "你选了 " << choice << "\n";

    // 死循环惯用法:for(;;)
    int k = 0;
    for (;;) {
        if (++k >= 3) break;
    }
    std::cout << k << "\n";    // 3
    return 0;
}
```

§3.4 范围 for:现代遍历的正统

范围 for 是 C++11 对循环的最大改进:for (const auto& x : 容器) 在底层展开为迭代器循环,代码量与出错率同时下降。三个要点:第一,只读遍历用 const auto&,修改遍历用 auto&,按值 auto 会拷贝元素(对 std::string 这类带堆内存的类型是真实开销);第二,范围 for 内部持有迭代器,遍历中修改容器结构(插入、删除)会使迭代器失效,行为未定义;第三,遍历 std::map 时用结构化绑定 for (const auto& [k, v] : m)。什么时候不用范围 for:需要下标参与算法(如冒泡排序)、需要反向遍历、需要跳过或提前删除元素时,回退到传统循环。

示例 3-4 范围 for 与结构化绑定

```

#include <iostream>
#include <vector>
#include <map>

int main() {
    std::vector<int> v{10, 20, 30};

    // 只读遍历:const auto& 零拷贝
    for (const auto& x : v) {
        std::cout << x << " ";
    }
    std::cout << "\n";

    // 修改遍历:auto&
    for (auto& x : v) {
        x *= 2;
    }

    // 结构化绑定遍历 map(C++17)
    std::map<std::string, int> scores{{"张三", 90}, {"李四", 75}};
    for (const auto& [name, score] : scores) {
        std::cout << name << ":" << score << " ";
    }
    std::cout << "\n";

    // 范围 for 的底层:传统迭代器循环
    for (auto it = v.begin(); it != v.end(); ++it) {
        std::cout << *it << " ";    // 20 40 60
    }
    std::cout << "\n";
    return 0;
}

```

§3.5 迭代器失效:C 数组没有的新坑

裸数组的下标访问在删除元素后依然“可用”(虽然语义错乱),但 `std::vector` 的迭代器在扩容或删除后直接失效——解引用失效迭代器是未定义行为。这是容器时代的新坑,C 里没有对应物。四种典型场景:vector 扩容使全部迭代器失效;vector erase 使被删位置及之后的迭代器失效;map/unordered_map 删除只使被删迭代器失效(其他仍有效,所以 map 可以安全地边遍历边删除,用 `it = m.erase(it)` 的写法);任何容器的析构使全部迭代器失效。失效迭代器与悬挂指针是同一类问题:值还在内存里,但所有权已转移,任何使用都是 UB。

⚠ 易错点 1:遍历时删除元素

`for (auto it = v.begin(); it != v.end(); ++it) { if (*it == 2) v.erase(it); }` 是错误的:erase 之后 `it` 失效,++it 是 UB。正确姿势是 erase-remove 惯用法:`v.erase(std::remove_if(v.begin(), v.end(), 条件), v.end())`——先集中移除,再一次 erase,整个过程中迭代器由算法内部安全处理。

示例 3-5 迭代器失效与 erase-remove 惯用法

```

#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    // 场景一:扩容使迭代器失效
    std::vector<int> v{1, 2, 3};
    auto it = v.begin();
    v.push_back(4);           // 可能扩容,it 已失效
    // std::cout << *it;      // 不要用:UB

    // 生产习惯:先 reserve 预分配,遍历中插入不失效
    v.reserve(100);
    it = v.begin();
    v.push_back(5);
    std::cout << *it << "\n";    // 1:容量充足,迭代器有效

    // 场景二:删除元素,用 erase-remove 惯用法
    std::vector<int> w{1, 2, 3, 4, 5};
    w.erase(std::remove_if(w.begin(), w.end(),
                           [](int x) { return x >= 3; }),
            w.end());
    for (int x : w) std::cout << x << " ";    // 1 2
    std::cout << "\n";
    return 0;
}

```

⚠ 易错点 2:范围 for 里插入元素

for (int x : v) { v.push_back(1); } 是 UB:范围 for 的底层迭代器在扩容后失效,++it 越界。若确实需要边遍历边插入,改用带下标的循环并在插入后重新获取 size(),或先收集再统一处理。

§3.6 break/continue/goto 与收尾控制

break 跳出最近一层循环或 switch;continue 跳过本次迭代进入下一次;goto 可以跳出任意深度的嵌套(但不能跳出函数),代价是代码结构被破坏,生产代码只在错误清理等少数场景使用。C++ 对 goto 有一处限制:不允许跳到带初始化变量的作用域之前——这与 switch 的 case 限制同理,都是为了防止跳过初始化。需要跳出多层循环时,优先考虑把循环抽成函数用 return 退出,可读性远好于 goto;读到别人的 goto 代码时,先确认标签位置与跳转方向,不要急着重构。

最后提醒复杂度:嵌套循环的层数要心里有数,三重循环在数据量过万时基本不可用。写循环前先估计数据规模与每层迭代次数,再把可提前退出的条件提到最外层——这是工程思维的第一课,也是第10章讲算法复杂度的铺垫。另外,循环内的分支会破坏流水线预测,热点循环里把最可能的分支写在前面、把 if 提成查表或开关,往往比微调比较写法收益更大;排查性能问题时先看循环,再谈其他。

C 的写法	C++ 的写法	差异
break/continue/goto	break/continue/goto	语义完全相同
goto 可跳任意标签	goto 不可跳过带初始化的变量	C++ 增加编译期检查
无	[[fallthrough]]、[[nodiscard]]	C++ 用属性标注意图

示例 3-6 break、continue 与 goto

```

#include <iostream>

int main() {
    // continue:跳过本次迭代
    for (int i = 1; i <= 10; ++i) {
        if (i % 2 == 0) continue;    // 跳过偶数
        std::cout << i << " ";      // 1 3 5 7 9
    }
    std::cout << "\n";

    // break:跳出最近一层循环
    int total = 0;
    for (int i = 1; ; ++i) {
        total += i;
        if (total > 20) break;
    }
    std::cout << total << "\n";      // 21

    // goto:跳出双层嵌套(生产代码慎用)
    int depth = 0;
    for (int i = 0; i < 3; ++i) {
        for (int j = 0; j < 3; ++j) {
            if (i == 1 && j == 1) goto found;
            ++depth;
        }
    }
found:
    std::cout << "depth = " << depth << "\n";    // 3

    // goto 不能跳过带初始化的变量:编译错误(注释展示)
    // goto skip;
    // int bad = 0;
    // skip:
    return 0;
}

```

本章速查表

主题	速查内容
条件类型	数值/指针非零即真(与 C 相同);Go/Rust 必须 bool
悬挂 else	else 就近配对;if (x = 1) 是赋值,不是比较
if 初始化	C++17:if (auto it = m.find(k); it != m.end())
switch	case 需整型常量;忘 break 会穿透;加 [[fallthrough]] 标注
switch 变量	case 内声明变量必须加花括号
三种循环	for/while/do-while 与 C 相同;菜单用 do-while
死循环	写 for(;;),不写 while(1)
范围 for	只读 const auto&,修改 auto&,按值会拷贝
遍历 map	for (const auto& [k, v] : m) 结构化绑定
迭代器失效	扩容/erase 后全部或部分失效,解引用是 UB
删除元素	erase-remove 惯用法;遍历中不插入、不删除
break/continue	与 C 相同;continue 跳过本次迭代
goto	可跳多层嵌套;不可跳过带初始化的变量
工程规范	循环里先 reserve;范围 for 内不改容器结构

最小必会清单

- 能解释条件类型的 C/C++ 语义与 Go/Rust 的差异,能识别 if (x = 1) 的错误
- 能写出 if 初始化语句与 switch 初始化语句的正确语法
- 能说出 case 穿透的规则与 [[fallthrough]] 的用途
- 能写出三种循环,解释 do-while 与 while 的区别
- 能写出范围 for 的三种形式:const auto&、auto&、结构化绑定
- 能解释 vector 扩容为什么使迭代器失效,并给出两种规避方式
- 能徒手写出 erase-remove 惯用法(含 lambda 条件)
- 能解释 goto 的限制:不能跳过带初始化的变量

自测题

Q `int x = 0; if (x = 1) { ... }` 会进入分支吗?为什么这个写法危险?

✓ 参考答案

会进入:`x = 1` 是赋值表达式,值为 1,非零即真。危险在于本意可能是 `x == 1`,误写后行为完全不同且编译器只给警告。生产规范:条件里禁止赋值,或用 `if (int y = ...; y)` 的初始化语句写法。

Q `switch` 里 `case 1: int v = 0; break;` 为什么是编译错误?如何修复?

✅ 参考答案

case 穿透可能从其他标签跳进 case 1 的作用域,越过 v 的初始化,违背"变量必须初始化后才能使用"的规则。修复:把声明包进花括号 case 1: { int v = 0; break; }。

Q for (const auto& [name, score] : scores) 中 const auto& 的作用是什么?去掉会怎样?

✅ 参考答案

const auto& 绑定到 map 元素的引用且只读,零拷贝。去掉 const 会允许修改;写成 auto 会拷贝整个 pair,对大数据容器是性能浪费。

Q 下面的代码哪里错了?for (int x : v) { if (x == 2) v.push_back(3); }

✅ 参考答案

遍历中 push_back 可能触发扩容,范围 for 底层的迭代器随之失效,++it 是未定义行为。修复:先收集要插入的值,遍历结束后统一插入;或改用下标循环并每次重新取 size()。

Q 写出删除 vector<int> 中所有偶数元素的惯用法(要求一行内完成逻辑)。

✅ 参考答案

v.erase(std::remove_if(v.begin(), v.end(), [](int x) { return x % 2 == 0; }), v.end()); remove_if 把满足条件的元素移到尾部并返回新逻辑结尾,erase 一次性删除,全程迭代器安全。

Q 为什么可以在遍历 std::map 时删除当前元素,却不能在遍历 std::vector 时删除当前元素?

✅ 参考答案

map 是节点型容器,删除只使被删元素的迭代器失效,前一个迭代器依然有效,所以惯用法是 it = m.erase(it);(erase 返回下一个迭代器)。vector 是连续存储,erase 会使被删位置及之后全部迭代器失效,遍历中删除必然踩空。因此 vector 的删除一律走 erase-remove 或先收集后统一处理。

章末实验题:学生成绩单菜单程序

实验 3:学生成绩单菜单程序

实验目标:写一个命令行菜单程序:录入学生成绩、统计平均分、删除不及格学生,把全章流程控制知识点全部用上。

实验要求:

- 任务 1:菜单循环用 do-while + switch,退出用 goto 跳出(覆盖:do-while、switch、default、goto)
- 任务 2:录入分支:用 if 检查成绩范围,越界用 continue 重进菜单(覆盖:if、continue)
- 任务 3:统计分支:范围 for 累加,发现极低分用 break 提前终止(覆盖:范围 for、break、setprecision)
- 任务 4:gradeOf() 用 switch 判等级,case 10/9 共享分支(覆盖:case 穿透)
- 任务 5:dropFailures() 用 erase-remove 惯用法删除不及格者(覆盖:迭代器失效的正确处理)
- 任务 6:手动验证:录入 5 人,统计,删除不及格,再统计(覆盖:完整流程)
- 任务 7:用 -Wall -Wextra 编译,零警告;每处 switch 都有 default(覆盖:工程规范)

实验环境:任意 C++17 编译器。编译命令:g++ -std=c++17 -Wall -Wextra gradebook.cpp -o gradebook

第一步 写结构体与菜单骨架(do-while + switch),编译通过再加统计

第二步 加 gradeOf 与 dropFailures,注意 case 花括号与 erase-remove 的写法

第三步 后补 goto 退出与边界验证,运行三组数据核对结果

参考答案(第一部分:数据结构)

```
// 学生成绩单菜单程序(第一部分:数据结构)
#include <iostream>
#include <string>
#include <vector>
#include <iomanip>
#include <algorithm>

struct Student {
    std::string name;    // 姓名
    double score;        // 成绩
};
```

✅ 参考答案(第二部分:判级与删除)

```
// 第二部分:判级与删除(覆盖:switch、case 穿透、erase-remove)
const char* gradeOf(double score) {
    switch (static_cast<int>(score) / 10) {    // 任务 4:switch 判等级
        case 10:
        case 9: return "优";    // case 穿透:10 与 9 共享分支
        case 8: return "良";
        case 7:
        case 6: return "及格";
        default: return "不及格";
    }
}

// 任务 5:删除不及格学生。
// 错误示范:for (auto it = list.begin(); it != list.end(); ++it)
//           if (it->score < 60) list.erase(it);    // it 失效,UB
// 正确做法:erase-remove 惯用法(lambda 详见第10章,这里当回调用)
void dropFailures(std::vector<Student>& list) {
    list.erase(std::remove_if(list.begin(), list.end(),
                               [](const Student& s) {
                                   return s.score < 60.0;
                               }),
               list.end());
}
```

✅ 参考答案(第三部分:main 菜单循环)

```
// 第三部分:main 菜单循环(覆盖:do-while、switch、break、continue、goto)
int main() {
    std::vector<Student> students;
    int choice = 0;
    do {                                     // 任务 1:菜单至少显示一次
        std::cout << "1.录入 2.统计 3.退出: ";
        std::cin >> choice;
        switch (choice) {
            case 1: {
                std::string name;
                double score;
                std::cin >> name >> score;
                if (score < 0.0 || score > 100.0) {    // 任务 2
                    std::cout << "成绩越界,忽略\n";
                    continue;                        // continue:回到菜单循环
                }
                students.push_back({name, score});
                break;
            }
            case 2: {
                if (students.empty()) { std::cout << "暂无数据\n"; break; }
                double sum = 0.0;
                int cnt = 0;
                for (const auto& s : students) {      // 任务 3:范围 for
                    if (s.score < 30.0) {
                        std::cout << "发现极低分,中止统计\n";
                        break;                        // break:跳出范围 for
                    }
                    sum += s.score;
                    ++cnt;
                }
                std::cout << std::fixed << std::setprecision(1);
                if (cnt > 0) std::cout << "平均分: " << sum / cnt << "\n";
                std::cout << "人数: " << students.size() << "\n";
                for (const auto& s : students) {      // 打印每人等级
                    std::cout << s.name << ": " << gradeOf(s.score) << "\n";
                }
                break;
            }
            case 3:
                std::cout << "再见\n";
                goto done;                          // goto:跳出循环与 switch
            default:
                std::cout << "无效选项\n";
                break;
        }
    } while (choice != 3);
done:
    std::cout << "程序结束\n";
    return 0;
}
```

设计说明:case 1 与 case 2 用花括号包住声明(switch 内声明变量的规则);continue 在 switch 内跳转的是外层 do-while;goto done 演示了跳出嵌套的正确场合——退出多层结构。dropFailures 放在统计前调用可演示完整流程:录入、删除、再统计。

下一章预告:第4章 函数与方法——从 C 的函数指针时代讲起,覆盖引用参数、默认参数、重载、const 成员函数、lambda 与函数对象,以及传值/传引用/传指针的三方对比。

本章目标

掌握 C++ 函数相对 C 函数的全部增强:引用参数、默认参数、函数重载、const 成员函数、函数对象、lambda 与 `std::function`;理解三种传参方式的取舍,能够在工程中写出接口清晰、复用性强的函数。

前置知识

会写 C 函数(返回值、形参、递归);理解指针与取址;了解第1章的引用概念与第3章的循环。本章代码均基于 C++17。

\$4.1 声明与定义分离	\$4.2 传值、传引用、传指针	\$4.3 默认参数与重载
\$4.4 const 成员函数	\$4.5 函数指针与函数对象	\$4.6 lambda 表达式

\$4.1 从 C 函数到 C++ 函数:声明与定义

C 程序员对函数已经很熟悉:返回类型、形参列表、局部变量、递归调用。C++ 的普通函数与 C 几乎完全兼容,但增加了三样 C 没有的东西——引用参数、默认参数、函数重载——并为后续的成员函数与 lambda 提供了基础。先看最基本的一条纪律:声明与定义分离。

声明只告诉编译器函数的存在与签名,定义给出函数体。工程上声明放在头文件,定义放在源文件:多个源文件包含同一个头文件时,各自编译,互不干扰,最后在链接阶段把符号合并。这与 C 的做法一致,但 C++ 的类型检查更严格:重载函数的调用必须与某个签名精确匹配或可转换匹配,否则直接编译失败,而不是 C 那种"名字对上就链接"的宽松模型。

这里有一个 C 程序员必须理解的概念:名字修饰。C++ 编译器会把函数名与参数类型编码成底层符号,例如 `int f(double)` 会被修饰为内部符号,两个参数不同的同名函数因此可以并存——这就是重载的底层机制。代价是 C 语言不认这些符号,所以调用 C 库或让 C 代码链接 C++ 代码时,要用 `extern "C"` 关闭修饰。这是多语言项目中的高频知识点,第13章讲模块系统时会再见到它。

清单 4-1 声明与定义分离,以及 `extern "C"`

```
// 声明:只给签名,放头文件 math_util.h
int add(int a, int b);
double avg(const double* data, int n);
extern "C" int c_version(void); // 关闭名字修饰,供 C 代码调用

// 定义:给出函数体,放源文件 math_util.cpp
#include "math_util.h"
int add(int a, int b) { return a + b; }

double avg(const double* data, int n) {
    double sum = 0.0;
    for (int i = 0; i < n; ++i) sum += data[i];
    return n > 0 ? sum / n : 0.0;
}

int c_version(void) { return 42; } // 被 C 程序以普通函数名调用
```

\$4.2 传值、传引用、传指针

C 里向函数传数据只有两条路:传值(拷贝)或传指针(传地址)。C++ 增加了一条路:传引用。引用是变量的别名,声明时必须绑定对象,永远不为空;对引用的修改就是对实参的修改。底层实现与指针一样都是地址,但语法上直接写变量名,并且编译器保证它非空,不需要判空分支。

三种方式的适用场景可以总结成一句话:小对象传值,大对象传 `const` 引用,要修改实参就传引用,允许为空(表示"无")就传指针。所谓大对象,指 `std::string`、`std::vector` 这类内部持有一块堆内存的类型——按值传递会触发一次拷贝,拷贝一个百万元素的 `vector` 就是全量复制,代价高昂;传 `const` 引用只传递一个地址,零拷贝且保证不被修改。

C 的写法	C++ 的写法	说明
void f(int x)	void f(int x)	传值,修改形参不影响实参,两者相同
void f(int* p)	void f(int* p)	传指针,可为空,调用方需判空;两者相同
无	void f(int& x)	传引用,别名语义,非空保证,修改即修改实参
void f(const int* p)	void f(const int& x)	只读传参,C 用 const 指针,C++ 首选 const 引用

还有一个 C 程序员容易忽略的细节:const 引用可以绑定临时对象,并且会把临时对象的生命周期延长到引用本身失效为止;普通引用不能绑定临时对象。这保证了 `avg({1.0, 2.0, 3.0}, 3)` 这类传临时数组的调用是安全的。反过来,把临时对象绑定给普通引用、或者让函数返回局部对象的引用或指针,都会制造悬空引用——对象在函数返回后已经销毁,访问它是未定义行为。编译器通常只给出警告,不会拒绝编译,这是 C 系语言最经典的一类 bug。

清单 4-2 三种传参方式对比

```
#include <iostream>
#include <string>

void byValue(int x) { x = 100; }           // 修改形参,不影响实参
void byRef(int& x) { x = 100; }           // 修改引用,即修改实参
void byPtr(int* p) { if (p) *p = 100; }   // 指针可为空,务必判空

double total(const std::vector<double>& v) { // const 引用:只读零拷贝
    double s = 0.0;
    for (double x : v) s += x;
    return s;
}

int main() {
    int a = 1, b = 1, c = 1;
    byValue(a);                          // a 仍为 1
    byRef(b);                             // b 变为 100
    byPtr(&c);                             // c 变为 100
    byPtr(nullptr);                       // 安全:内部判空
    std::cout << a << " " << b << " " << c << "\n"; // 1 100 100
    return 0;
}
```

⚠ 易错点 1:返回局部对象的引用或指针

函数返回局部变量时,对象在函数退出时销毁,引用与指针随即悬空。正确做法是按值返回:现代编译器普遍实现了返回值优化(RVO)和移动语义,大对象按值返回不会产生多余拷贝,放心使用。

§4.3 默认参数与函数重载

默认参数让调用方可以省略部分实参。两条硬规则:第一,一旦某个参数有默认值,它右边的所有参数都必须有默认值,这就是靠右原则——否则省略实参时无法确定省略到哪一位;第二,默认值写在声明处(头文件),定义处不能再写,否则编译器报“重复默认实参”错误。头文件的读者一眼就能看到默认值,而每个源文件只看到自己 include 的声明,行为一致。

函数重载允许同名函数携带不同的参数列表,调用时编译器根据实参的类型、个数挑选最合适的版本。注意两点:重载只看参数,不按返回值区分——只改返回类型、参数完全相同的两个函数无法重载,因为调用语句无法据此选择;参数个数相

同但类型可转换时,编译器按优先级挑选:精确匹配优于整型提升(如 int 升 long),提升优于标准转换(如 double 转 int),转换优于用户定义转换,转换代价相同的调用直接判为二义性错误。

C 的写法	C++ 的写法	说明
print_int(5); print_str("hi");	print(5); print("hi");	C 靠不同名字表达不同输入,C++ 同名重载
int clamp(int v, int lo, int hi)	int clamp(int v, int lo, int hi)	C 只能给三个参数,C++ 可给默认值省去调用
编译期无法检查	参数不匹配直接编译失败	C++ 强类型签名,错误更早暴露

清单 4-3 默认参数与重载

```
#include <iostream>
#include <string>

// 默认参数:靠右,写在声明处
void logMessage(const std::string& msg, int level = 0);

// 重载:按参数类型区分
void print(int v) { std::cout << "整数:" << v << "\n"; }
void print(double v) { std::cout << "浮点:" << v << "\n"; }
void print(const std::string& s) { std::cout << "文本:" << s << "\n"; }

void logMessage(const std::string& msg, int level) { // 定义处不写默认值
    std::cout << "[L" << level << "] " << msg << "\n";
}

int main() {
    logMessage("启动");           // 省略 level,取默认值 0
    logMessage("警告", 2);        // 显式传 level
    print(5);                     // 精确匹配 print(int)
    print(3.14);                  // 精确匹配 print(double)
    print(std::string("hi"));      // 精确匹配 print(string)
    return 0;
}
```

 **易错点 2:默认参数与重载叠加产生二义性**

void f(int a, int b = 1); 与 void f(int a); 同时存在时,调用 f(5) 既可以解释为"第一个函数用默认参数",也可以解释为"精确调用第二个",编译器无法抉择,直接报错。设计接口时,默认参数与重载不要表达同一组调用,二选一即可。

§4.4 成员函数与 const 成员函数

C 没有"对象调用函数"的语法,只能写 struct 加全局函数,把对象指针显式传进去。C++ 把函数放进类里成为成员函数,对象用点号调用,编译器隐式地把对象的地址作为 this 指针传给函数。this 是 const 指针:指针本身不能改指向,但指向的对象可以修改。

const 成员函数是在参数列表之后加 const 修饰的函数,例如 std::string getName() const。它的语义是:this 变为指向 const 对象的指针,函数体内不能修改任何非 mutable 成员。规则反过来更常用:const 对象只能调用 const 成员函数;非 const 对象可以调用任意成员函数。于是"这个函数不改对象"成为接口契约的一部分,编译器强制检查——这在 C 里没有对应物,是 C++ 用类型系统防呆的典型例子。

偶尔需要在 const 成员函数里修改某个成员,比如统计调用次数、缓存计算结果,可以把该成员声明为 mutable。mutable 意味着"即使对象是 const,这个字段也可以改",它只应出现在内部实现细节上,不该出现在公开接口数据中。

C 的写法	C++ 的写法	说明
void set_name(Student* s, const char* n)	s.setName(n)	C 显式传对象指针,C++ this 隐式传递
无	void get() const;	const 成员函数,编译器检查只读性
const Student* s; s->set()	const Student s; s.set()	const 对象禁止调用非 const 成员,C++ 编译期拒绝

清单 4-4 类、this 与 const 成员函数

```
#include <iostream>
#include <string>

class Student {
public:
    Student(std::string n, double s) : name(n), score(s) {}

    const std::string& getName() const { return name; } // const 成员函数
    double getScore() const { return score; }           // 只读,任何对象可调

    void rename(const std::string& n) { name = n; }      // 非 const,修改对象
    void recordCall() const { ++calls; }                // mutable 计数

    int callCount() const { return calls; }

private:
    std::string name;
    double score;
    mutable int calls = 0; // const 成员函数里可以修改
};

int main() {
    Student s("张三", 85.5);
    s.rename("张三丰");
    const Student& c = s; // const 引用视角
    std::cout << c.getName() << " " << c.getScore() << "\n";
    // c.rename("李四"); // 错误:const 对象不能调非 const 成员函数
    c.recordCall();
    std::cout << "调用次数:" << s.callCount() << "\n";
    return 0;
}
```

§4.5 函数指针、函数对象与 std::function

函数指针与 C 完全一致:它保存函数的地址,通过它调用函数。C 里回调机制完全依赖函数指针,还要额外传一个 void* 上下文才能携带状态。C++ 在函数指针之上提供了两种更强大的可调用对象。

函数对象是重载了 operator() 的类,对象用起来像函数,但可以携带成员变量作为状态。C 的静态函数天然无状态,要用全局变量模拟,而函数对象的每个实例都有独立状态,天然线程安全。lambda 是函数对象的语法糖(见 §4.6),手写函数对象的场景已经很少,但理解它才能理解 lambda 的本质。

std::function 是通用的可调用包装器,来自 <functional> 头文件:函数指针、函数对象、lambda 都可以赋值给它,调用时统一用 f(args) 语法。它内部做类型擦除,把不同的可调用类型收进同一个类型,适合做回调参数、事件表、策略注入。代价是一次间接调用,比裸函数指针略慢,回调非热点路径时通常可忽略。

C 的写法	C++ 的写法	说明
void apply(int* d, int n, int (*f)(int))	void apply(vector<int>&, function<int(int)> f)	C 只认函数指针,C++ 接受任意可调用对象
void* ctx; f(ctx, x)	[&ctx](int x){ ... }	C 用 void* 传上下文,C++ 用 lambda 捕获
回调状态靠全局变量	函数对象成员变量即状态	C++ 状态与调用者绑定,可并发安全

清单 4-5 三种可调用对象并存

```
#include <iostream>
#include <functional>

int triple(int x) { return x * 3; } // 普通函数

struct Multiplier { // 函数对象:携带状态
    int factor;
    int operator()(int x) const { return x * factor; }
};

void run(const std::function<int(int)>& f, int v) { // 统一回调接口
    std::cout << "结果:" << f(v) << "\n";
}

int main() {
    run(triple, 10); // 函数指针
    run(Multiplier{7}, 10); // 函数对象
    run([](int x) { return x * x; }, 10); // lambda
    // 三者在同一个 std::function 参数上平等互换
    return 0;
}
```

§4.6 lambda 表达式

lambda 的完整语法是 [捕获列表](参数列表) -> 返回类型 { 函数体 },C++14 起返回类型可以省略让编译器推导,C++17 起支持泛型 lambda:[](auto x) 参数类型由实参推导,相当于模板函数。捕获列表决定 lambda 能访问外部哪些变量:[] 不捕获; [=] 按值捕获所有用到的自动变量; [&] 按引用捕获; [x, &y] 指定捕获,按值捕获 x、按引用捕获 y; [this] 捕获当前对象的指针。

理解 lambda 的关键是知道它的本质:编译器为每个 lambda 生成一个匿名类,捕获列表就是类的成员变量,函数体就是 operator()。也就是说 lambda 不过是我们手写函数对象的一份语法糖——按值捕获是拷贝成员,按引用捕获是引用成员,生命周期规则完全一致。因此“按引用捕获局部变量,却把 lambda 存到比局部变量活得更久的地方”就会产生悬空引用,这是 lambda 最常见的坑。

工程上,lambda 最常出现在标准库算法里:排序比较器、查找谓词、转换函数都是一行 lambda 的事。写函数对象和函数指针的机会因此大幅减少,接口参数尽量用 lambda 或 std::function 接收。

清单 4-6 lambda 的捕获与泛型用法

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v{5, 3, 8, 1, 9, 2};

    // [&] 按引用捕获:排序比较器
    std::sort(v.begin(), v.end(), [](int a, int b) { return a < b; });

    int threshold = 4;
    // [=] 按值捕获:统计大于阈值的个数
    int cnt = (int)std::count_if(v.begin(), v.end(),
                                [=](int x) { return x > threshold; });

    // 泛型 lambda:任何可比较类型都能用
    auto maxOf = [](auto a, auto b) { return a > b ? a : b; };

    // 指定捕获:混合按值与按引用
    int base = 100;
    auto shift = [base, &cnt](int x) { return x + base + cnt; };

    std::cout << "大于" << threshold << "的个数:" << cnt << "\n";
    std::cout << "max:" << maxOf(3, 7) << ", shift:" << shift(1) << "\n";
    return 0;
}
```

⚠ 易错点 3:按引用捕获后逃逸作用域

`auto f = [&]{ return x; };` 把 `f` 存到全局或容器里,等 `x` 所在函数返回后再调用 `f`,引用的对象已经销毁。排查口诀:按值捕获是安全的拷贝,按引用捕获的 `lambda` 只能在使用它的同一作用域内存活。

★ 关键点:传参选择速记

只读小对象传值;只读大对象传 `const` 引用;要修改传引用;允许为空传指针;返回值按值返回,依赖 RVO 与移动语义。这条经验能避开九成参数设计错误。

函数相关速查表

概念	语法示例	要点
函数声明	<code>int add(int, int);</code>	只给签名,放头文件
函数定义	<code>int add(int a, int b) { ... }</code>	给出函数体
传值	<code>void f(int x)</code>	拷贝,修改不影响实参
传引用	<code>void f(int& x)</code>	别名,非空,修改即实参
传 const 引用	<code>void f(const int& x)</code>	只读零拷贝,可绑临时对象
传指针	<code>void f(int* p)</code>	可为空,函数内判空
默认参数	<code>void f(int a, int b = 1)</code>	靠右原则,声明处给出
函数重载	<code>void f(int); void f(double);</code>	按参数区分,不按返回值
extern "C"	<code>extern "C" int cf(void);</code>	关闭名字修饰,兼容 C
函数指针	<code>int (*p)(int) = triple;</code>	与 C 相同,存函数地址
函数对象	<code>struct F { void operator()() {} };</code>	重载调用运算符,携带状态
std::function	<code>std::function<int(int)> f = g;</code>	类型擦除,统一可调用对象
const 成员函数	<code>void get() const;</code>	this 只读,mutable 例外
lambda	<code>auto f = [=](int x) { ... };</code>	闭包语法糖,捕获列表即成员

自测题

- Q 1. 传值、传引用、传指针三者的本质区别是什么?什么时候必须用指针?**
- Q 2. 为什么默认参数必须靠右?声明与定义分离时默认值写在哪?**
- Q 3. 为什么重载不能只靠返回值区分?重载决议的优先级顺序是什么?**
- Q 4. const 成员函数里能修改成员变量吗?mutable 解决什么问题?**
- Q 5. lambda 的 [=] 与 [&] 捕获有何区别?按引用捕获的 lambda 逃逸出作用域会怎样?**

✅ 参考答案

- 传值复制一份实参,修改形参不影响实参;传引用与传指针都把地址传进去,区别是引用非空且语法自然,指针可为空。需要表达"可能没有值"(如空指针表示空),就必须用指针;其余修改实参的场景优先引用。
- 调用方省略实参时,编译器必须知道省略到哪一位,靠右原则保证省略的总是末尾一段连续参数;默认值写在声明处(头文件),定义处再写属于重复默认实参,编译报错。
- 调用语句 `f(x)` 只提供实参,无法根据"将来的返回值类型"做选择,所以仅返回值不同的重载必然二义。决议优先级:精确匹配优于整型提升,提升优于标准转换,转换优于用户定义转换。
- `const` 成员函数中 `this` 指向 `const` 对象,不能修改普通成员;`mutable` 成员是例外,允许在 `const` 函数里修改,适合缓存与计数等内部细节。
- `[=]` 把用到的变量拷贝进闭包,安全但可能拷贝大对象;`[&]` 存引用,零拷贝但依赖被引用对象存活。按引用捕获的 `lambda` 若被存到作用域之外再调用,引用悬空,是未定义行为。

实验:学生成绩报表生成器

本实验用一个完整的报表程序串联本章全部知识点:重载的 print、带默认参数的打印函数、引用参数读入数据、const 成员函数、函数对象、lambda、std::function 回调。代码分三部分,逐块阅读,每个覆盖点已在注释中标明。程序从标准输入读入分数(负数结束),输出人数、平均分、高于平均的人数与及格率。

实验 4•第一部分:声明、重载与引用参数(覆盖:重载、默认参数、引用参数、const 成员函数)

```
#include <iostream>
#include <string>
#include <vector>
#include <functional>
#include <algorithm>
#include <numeric>

struct Student {                                // 覆盖:const 成员函数
    std::string name;
    double score;
    const std::string& getName() const { return name; }
    double getScore() const { return score; }
};

// 覆盖:函数重载,同名按参数类型区分
void print(double v) { std::cout << "分数:" << v << "\n"; }
void print(const std::string& s) { std::cout << "姓名:" << s << "\n"; }

// 覆盖:默认参数,靠右,声明处给出
void print(const Student& st, int rank = 0) {
    std::cout << "第" << rank << "名:" << st.getName()
               << " " << st.getScore() << "\n";
}

// 覆盖:引用参数,零拷贝写出结果
bool readScores(std::vector<double>& v) {
    double x;
    while (std::cin >> x && x >= 0.0) v.push_back(x);    // 负数结束输入
    return !v.empty();
}
```

实验 4•第二部分:函数对象与 lambda(覆盖:函数对象、lambda、std::function)

```
// 覆盖:函数对象,operator() 携带状态(及格线)
struct ScoreFilter {
    double low;
    bool operator()(double s) const { return s >= low; }
};

// 覆盖:std::function 做回调参数,函数指针/lambda/函数对象皆可传入
void forEach(const std::vector<Student>& sts,
             const std::function<void(const Student&)>& cb) {
    for (const auto& s : sts) cb(s);
}

// 覆盖:默认参数 + lambda 按引用捕获,延迟到调用时执行
double passRate(const std::vector<double>& scores, double line = 60.0) {
    auto pass = [&](double s) { return s >= line; };
    int n = (int)std::count_if(scores.begin(), scores.end(), pass);
    return n * 100.0 / scores.size();
}
```

实验 4•第三部分:main 调度(覆盖:引用、lambda、函数对象、std::function、重载综合)

```
int main() {
    std::vector<double> scores;
    if (!readScores(scores)) return 0;           // 引用参数传出数据

    double avg = std::accumulate(scores.begin(), scores.end(), 0.0)
                / scores.size();

    // 覆盖:lambda 排序比较器,降序
    std::vector<double> sorted = scores;
    std::sort(sorted.begin(), sorted.end(),
              [](double a, double b) { return a > b; });

    // 覆盖:函数对象参与统计
    ScoreFilter aboveAvg{avg};
    int cnt = (int)std::count_if(sorted.begin(), sorted.end(), aboveAvg);

    std::cout << "共" << scores.size() << "人,平均" << avg
               << ",高于平均" << cnt << "人,及格率"
               << passRate(scores) << "%\n";

    // 覆盖:std::function 回调 + 带默认参数的重载
    std::vector<Student> cls{{"张三", 85.0}, {"李四", 59.5}, {"王五", 73.0}};
    forEach(cls, [](const Student& s) { print(s, 3); });
    print(cls[0].getScore());                    // 重载的 print(double)
    return 0;
}
```

实验覆盖点核对:重载(print 三个版本)、默认参数(passRate 与 print 的 rank)、引用参数(readScores 与 const 引用形参)、const 成员函数(getName/getScore)、函数对象(ScoreFilter)、lambda(排序与统计)、std::function(forEach 回调)、函数指针与名字修饰的概念在 §4.1 与 §4.5 已讲解,可自行把 forEach 的回调换成普通函数名验证互操作性。全部编译通过后,试着给 ScoreFilter 增加上界字段,或把 print 换成输出到文件的版本。

工程建议 1:接口参数先写 const 引用

新写一个函数时,参数一律从 const 引用起步,编译器报"需要修改"再放开成引用,只有确实允许为空才用指针。这样既避免大对象拷贝,又把只读契约写进类型里,团队协作时接口意图一目了然。

工程建议 2:回调与算法优先 lambda

标准库算法的比较器、谓词,线程与事件回调,一律用 lambda 表达;需要复用或状态复杂再升级为函数对象;只有需要动态替换实现(插件、测试替身)时才用 `std::function`。三个层次各司其职,代码可读性最好。

本章小结

C++ 函数在 C 之上新增了引用参数、默认参数与重载,并衍生出成员函数、函数对象、lambda、`std::function` 一整套可调用体系。传参三原则(小对象传值、大对象传 const 引用、可空传指针)、默认参数靠右、重载不按返回值区分、lambda 按值捕获安全按引用捕获易悬空——这五条是本章必须内化的结论。下一章进入 C 程序员最陌生的领域:内存管理。

本章目标

从 malloc/free 出发,理解 C++ 的 new/delete 与对象生命周期;掌握三大件与五法则的深拷贝纪律;吃透 RAI 的资源管理思想;熟练运用 unique_ptr、shared_ptr、weak_ptr 三种智能指针,识别并修复循环引用。本章是 C 程序员向 C++ 转型最重要的一章。

前置知识

熟练掌握 C 的 malloc/free、指针与结构体;了解第4章的引用、成员函数与构造函数概念。本章代码基于 C++17,使用智能指针必须 include <memory>。

\$5.1 new/delete 与对象生命周期	\$5.2 三大件与五法则	\$5.3 RAII 资源管理
\$5.4 unique_ptr 独占所有权	\$5.5 shared_ptr 与 weak_ptr	\$5.6 常见内存错误与工具

\$5.1 从 malloc/free 到 new/delete

C 程序员用 malloc/free 管理堆内存:malloc 只分配一片字节,返回 void*,需要强转成目标类型;free 只回收字节,不做任何清理。C++ 对象有构造函数和析构函数,分配内存的同时必须构造对象、释放内存前必须先析构对象,所以 new/delete 把两步合并成一次操作:new 分配内存后调用构造函数,delete 先调用析构函数再释放内存。这是一切内存管理的起点。

配套规则:new 分配的对象用 delete 释放,new[] 分配的数组必须用 delete[] 释放,混用是未定义行为。new 失败时抛出 std::bad_alloc 异常,而不是像 malloc 那样返回 NULL——这是 C++ 默认的失败模型,如果你需要老式的不抛异常版本,可以用 new(std::nothrow),它会返回空指针。new[] 在数组开头额外记录元素个数,delete[] 靠它逐个调用析构函数,这也是二者必须配对的原因。

C 的写法	C++ 的写法	说明
int* p = malloc(sizeof(int) * 10);	int* p = new int[10];	C 手算字节数并强转,C++ 类型安全、自动计数
free(p);	delete[] p;	C 只回收字节,C++ 先析构每个元素再释放
if (!p) { 处理失败 }	new 抛 std::bad_alloc	C 靠返回值判断,C++ 走异常路径
结构体需要手动 memset 初始化	new 自动调用构造函数	C++ 对象保证构造完成,不可用半初始化状态

清单 5-1 new/delete 与 new[]/delete[]

```

#include <iostream>
#include <string>

class Timer {
public:
    explicit Timer(const std::string& n) : name(n) {
        std::cout << "构造:" << name << "\n";
    }
    ~Timer() { std::cout << "析构:" << name << "\n"; } // 释放前必被调用
private:
    std::string name;
};

int main() {
    Timer* t = new Timer("单对象"); // 分配 + 构造
    delete t; // 析构 + 释放,配对

    Timer* arr = new Timer[3]{ // new[]:每个元素构造
        Timer("甲"), Timer("乙"), Timer("丙")};
    delete[] arr; // 逐个析构后再释放

    int* raw = new (std::nothrow) int(42); // 失败返回空指针,不抛异常
    if (raw) {
        std::cout << *raw << "\n";
        delete raw;
    }
    return 0;
}

```

⚠ 易错点 1:new 与 delete 不配对

new 用 delete,new[] 用 delete[],漏写或混写都是错误:漏写泄漏内存,混写是未定义行为。现代 C++ 的答案不是"记得配对",而是根本不手写 new/delete——用容器与智能指针(\$5.4),让析构自动完成配对。

§5.2 三大件与五法则

一个类如果手动管理资源(比如持有一个 new 出来的指针成员),就必须同时提供拷贝构造、拷贝赋值和析构这三个函数,合称三大件:拷贝构造负责用另一个对象初始化自己,拷贝赋值负责把自己的内容替换成另一个对象的内容,析构负责释放自己占用的资源。编译器默认生成的版本对内置类型逐位拷贝——对指针成员来说,默认拷贝只是复制指针值,两个对象将指向同一块内存,析构时先后释放两次,这就是双重释放,是 C++ 最经典的内存崩溃之一。

C++11 又加入了移动构造与移动赋值,三大件扩成五法则:移动语义允许"把资源搬走而不是复制"。移动构造的要点是接管源对象的指针,然后把源指针置空,这样源对象析构时无事可做,不会释放已被接管的内存。移动操作必须声明 noexcept,否则 vector 扩容时无法放心使用移动。经验法则:写了析构函数就基本可以断定需要五件套,尤其是持有裸指针、文件句柄等资源的类。

C 的写法	C++ 的写法	说明
struct 逐位拷贝,共享指针成员	手写深拷贝,各自持有副本	C 默认浅拷贝,双重释放隐患;C++ 用五法则修正
memcpy 复制结构体	拷贝构造 / 拷贝赋值	C 不调用任何函数,C++ 构造与赋值语义完整
无法表达"搬走资源"	移动构造,源对象置空	C++ 转移所有权,大对象零拷贝传递

清单 5-2 五法则:手动管理指针的类

```
#include <iostream>
#include <string>
#include <utility>

class NameHolder {
public:
    explicit NameHolder(const std::string& n)
        : p(new std::string(n)) {} // new:分配 + 构造

    NameHolder(const NameHolder& o) // 三大件:拷贝构造
        : p(new std::string(*o.p)) {} // 深拷贝,绝不共享指针

    NameHolder& operator=(const NameHolder& o) { // 三大件:拷贝赋值
        if (this != &o) { // 自赋值防护
            *p = *o.p; // 复用已有内存
        }
        return *this;
    }

    NameHolder(NameHolder&& o) noexcept // 五法则:移动构造
        : p(o.p) { o.p = nullptr; } // 接管指针并置空源

    NameHolder& operator=(NameHolder&& o) noexcept { // 移动赋值
        if (this != &o) {
            delete p; // 先释放自己的
            p = o.p;
            o.p = nullptr;
        }
        return *this;
    }

    ~NameHolder() { delete p; } // 三大件:析构

    const std::string& get() const { return *p; }
private:
    std::string* p;
};

int main() {
    NameHolder a("原始");
    NameHolder b = a; // 拷贝构造:两块独立内存
    NameHolder c(std::move(a)); // 移动构造:a 被置空,资源归 c
    b = c; // 拷贝赋值
    std::cout << b.get() << " " << c.get() << "\n";
    return 0; // 三个对象各自释放,无双重释放
}
```

⚠ 易错点 2:忘记三大件导致的浅拷贝

类里出现裸指针成员,却依赖编译器默认的逐位拷贝,两个对象就会共享同一块内存:改一个影响另一个,析构时双双释放同一块内存,直接崩溃。诊断口诀:出现了 delete 却不见拷贝构造,先怀疑浅拷贝。

§5.3 RAII:把资源绑定到对象生命周期

RAII(Resource Acquisition Is Initialization,资源获取即初始化)是 C++ 最核心的工程思想:在构造函数里获取资源,在析构函数里释放资源,资源与对象的生命周期严格一致。对象创建,资源就绪;对象销毁,资源自动回收——无论正常返回还是抛出异常,析构函数都保证执行,资源绝不泄漏。

对比 C 的手工管理:文件要 fopen 后记得 fclose,而且每一条出错路径都得写一遍 fclose,漏一条就泄漏。RAII 把"记得释放"变成"必然释放":C++ 中栈对象离开作用域必然调用析构函数,这是语言保证,不需要程序员记忆。标准库就是 RAII 的样板:std::string、std::vector 自己分配内存自己释放,你从来不需要对它们调用 free;文件流 ifstream 关闭时自动关闭文件。写自定义类时,凡是"打开就要关闭、获取就要释放"的资源——文件、锁、套接字、数据库连接——都应包装成 RAII 类。

C 的写法	C++ 的写法	说明
FILE* f = fopen(...); ... fclose(f);	ofstream f(path); 结束自动关闭	C 每路径手工释放,C++ 析构保证释放
出错路径容易漏 fclose	抛异常也执行析构	RAII 异常安全,无遗漏路径
锁:lock/unlock 成对手写	std::lock_guard 自动解锁	C++ 锁对象即解锁,死锁少一半

清单 5-3 RAII 包装 C 资源

```

#include <cstdio>
#include <string>

class FileGuard {                                // RAII:构造获取,析构释放
public:
    explicit FileGuard(const std::string& path, const char* mode)
        : f(fopen(path.c_str(), mode)) {
        if (!f) throw std::runtime_error("打开失败: " + path);
    }
    ~FileGuard() {
        if (f) {
            fclose(f);                        // 无论正常还是异常返回,必被执行
            f = nullptr;
        }
    }
    FileGuard(const FileGuard&) = delete;        // 不可复制:句柄唯一
    FileGuard& operator=(const FileGuard&) = delete;
    void write(const std::string& s) { fputs(s.c_str(), f); }
private:
    FILE* f;
};

void logAll(const std::string& path) {
    FileGuard log(path, "w");                // 构造:打开文件
    for (int i = 0; i < 3; ++i) {
        if (i == 2) throw std::runtime_error("中途出错");
        log.write("第 " + std::to_string(i) + " 行\n");
    }
}                                           // 析构:即使抛异常,文件也关闭

int main() {
    try {
        logAll("run.log");
    } catch (const std::exception& e) {
        std::printf("捕获:%s\n", e.what());
    }
    return 0;
}

```

§5.4 unique_ptr:独占所有权

unique_ptr 是独占所有权的智能指针:同一时刻只有一个指针拥有这块内存,不允许拷贝,只能移动——移动即所有权转移。它在析构时自动 delete 管理的对象,把"手工配对"变成"自动释放"。创建用 std::make_unique(第一个参数是构造参数,自动推导类型),C++14 起可用。unique_ptr 可以持有数组,声明为 unique_ptr<int[]>,析构时自动用 delete[]。

unique_ptr 的价值不只是防泄漏,更是把所有权写进类型:函数参数用 const unique_ptr& 表示"只借用",返回值用 unique_ptr 表示"所有权移交",容器里放 unique_ptr 表示"容器独占这些元素"。裸指针可以随时被拷贝、被遗忘,所有权说不清楚;unique_ptr 让"谁拥有、谁释放"在编译期就确定下来。

清单 5-5 容器中的 unique_ptr

```

#include <memory>
#include <vector>

std::vector<std::unique_ptr<int>> bucket;    // 容器独占元素

void fill(int n) {
    for (int i = 0; i < n; ++i) {
        bucket.push_back(std::make_unique<int>(i));
    }
}    // 返回后元素仍存活,所有权在容器

int main() {
    fill(3);
    for (const auto& p : bucket) std::cout << *p << " ";
    bucket.clear();    // 清空时逐个 delete,无需手工释放
    return 0;
}

```

⚠ 易错点 3:裸指针的悬挂与重复释放

裸指针在 delete 之后仍然保存着地址,继续使用就是悬空指针,未定义行为;同一块内存被 delete 两次就是重复释放,直接崩溃。智能指针从机制上消灭这两类错误:unique_ptr 不能拷贝所以只有一个释放者,shared_ptr 只有计数归零才释放。

§5.5 shared_ptr 与 weak_ptr:共享所有权

shared_ptr 是共享所有权:多个 shared_ptr 可以指向同一对象,每个都保存一个引用计数,每拷贝一次计数加一,每析构一个计数减一,计数归零才真正释放对象。计数操作是原子的,多个线程同时拷贝与销毁是安全的。创建用 std::make_shared,它把控制块和对象一次性分配,比先 new 再包 shared_ptr 少一次分配。代价是引用计数本身有开销,以及可能产生循环引用。

循环引用是 shared_ptr 最著名的陷阱:两个对象互相持有对方的 shared_ptr,彼此的计数都大于零,谁也不会释放,内存悄然泄漏。解决办法是 weak_ptr:它是"旁观者",指向对象但不增加计数,不能直接访问,必须调用 lock() 提升为 shared_ptr,对象已销毁时 lock() 返回空指针。使用套路:单向共享用 shared_ptr,回指、缓存、观察等"不需要延长生命"的引用一律用 weak_ptr。

C 的写法	C++ 的写法	说明
裸指针传遍全程序,不知谁释放	unique_ptr:独占;shared_ptr:共享	所有权写进类型,释放点明确
手工引用计数结构体	make_shared 自动计数	计数线程安全,归零自动释放
无对应概念	weak_ptr 观察不持有	打破循环引用,避免内存泄漏

清单 5-4 三种智能指针与循环引用修复

```

#include <iostream>
#include <memory>

struct Worker;
struct Team {                                // 团队:shared_ptr 持有成员
    std::shared_ptr<Worker> member;
    ~Team() { std::cout << "团队析构\n"; }
};

struct Worker {                              // 员工:weak_ptr 回指团队,不增计数
    std::weak_ptr<Team> team;
    ~Worker() { std::cout << "员工析构\n"; }
};

int main() {
    std::cout << "--- 正常共享 ---\n";
    {
        auto t = std::make_shared<Team>();
        t->member = std::make_shared<Worker>();
        t->member->team = t;                // weak_ptr 赋值,不增加计数
        auto sp = t->member->team.lock();    // 提升为 shared_ptr
        if (sp) std::cout << "回指成功\n";
    }                                       // t 与 member 双双正常析构

    std::cout << "--- 独占所有权 ---\n";
    auto up = std::make_unique<int>(99);    // unique_ptr 独占
    std::unique_ptr<int> up2 = std::move(up); // 移动转移所有权
    // std::unique_ptr<int> up3 = up2;        // 错误:不允许拷贝
    std::cout << *up2 << "\n";              // 作用域结束自动 delete
    return 0;
}

```

⚠ 易错点 4:shared_ptr 循环引用悄然泄漏

Team 持 Worker,Worker 又持 Team,两边计数互相兜底,永远不为零,两个对象一起泄漏。工具类(如 AddressSanitizer 的报告)能看出"泄漏的是互相引用的两块内存",但根治靠设计:区分"拥有"与"观察",观察一律用 weak_ptr。

§5.6 内存布局与常见错误排查

C++ 程序的内存分四块:栈区存放局部变量,自动分配自动回收,速度快但容量小;堆区由 new/malloc 分配,程序员或智能指针负责回收;静态区存放全局变量与静态变量,整个程序生命周期都在;常量区存放字符串字面量等只读数据。栈上对象离开作用域必然析构,堆上对象只有释放才析构——这决定了"对象活了多久"的全部问题。

排查内存错误用三件武器:AddressSanitizer(ASan)在编译时加 -fsanitize=address,运行时报出越界、悬空、泄漏的精确行号;valgrind 适合没有 ASan 的环境,慢但全面;泄漏检测(LeakSanitizer)专门报告未释放的内存。现代工程默认开启 ASan,把它当成编译器自带的体检仪。性能方面,堆分配比栈慢两个数量级,热点路径避免频繁 new 小对象,优先复用对象或用容器一次性分配。

内存管理速查表

概念	语法示例	要点
new/delete	int* p = new int(1); delete p;	分配+构造 / 析构+释放,必须配对
new[]/delete[]	int* a = new int[10]; delete[] a;	数组用数组版本,混用是未定义行为
不抛异常分配	new (std::nothrow) int(1)	失败返回空指针,适合兼容老代码
三大件	拷贝构造/拷贝赋值/析构	手动管理资源必须三件齐全
五法则	三大件 + 移动构造/移动赋值	移动置空源指针,声明 noexcept
深拷贝	new 一份副本给新对象	杜绝浅拷贝共享指针
RAII	构造获取、析构释放	资源绑定对象生命周期,异常安全
unique_ptr	make_unique<T>(args)	独占所有权,只可移动不可拷贝
shared_ptr	make_shared<T>(args)	引用计数,归零释放,计数线程安全
weak_ptr	w.lock() 判空后使用	不增计数,破循环引用
内存泄漏	new 后无人 delete	ASan / valgrind 定位泄漏点
悬空指针	delete 后继续使用	未定义行为,智能指针根治
双重释放	同一内存 delete 两次	浅拷贝与裸指针的经典事故
make_unique 数组	make_unique<int[]>(10)	自动配对 delete[],C++14 起

自测题

- Q 1. new 与 malloc 的本质区别是什么?new[] 与 delete[] 为什么要配对?**
- Q 2. 为什么带指针成员类不能依赖默认拷贝?深拷贝与浅拷贝有何区别?**
- Q 3. 移动构造为什么要把源指针置空?移动函数为什么要声明 noexcept?**
- Q 4. shared_ptr 循环引用为什么泄漏?weak_ptr 是如何解决的?**
- Q 5. RAII 为什么能做到异常安全?请举一个"不用 RAII 就会泄漏"的场景。**

✓ 参考答案

- malloc 只分配字节、返回 void*、失败返回 NULL;new 分配后调用构造函数、类型安全、失败抛 std::bad_alloc。new[] 在数组头记录元素个数,delete[] 靠它逐个析构;与 delete 混用会错误解析内存布局,未定义行为。
- 默认拷贝逐位复制指针,两个对象指向同一内存,一方修改影响另一方,析构时双重释放。深拷贝为每个新对象分配独立副本,各自管理互不干扰,是手动管理内存的必修课。
- 移动是"搬走"而非"复制":接管源对象的指针后必须把源指针置空,源析构时才不会二次释放。声明 noexcept 让容器(如 vector 扩容)敢用移动而非退回拷贝,否则性能与正确性都打折。
- A 持 B、B 持 A,两者计数都因对方存在而大于零,永远不归零,对象成对泄漏。weak_ptr 不增加计数,只做观察;需要访问时 lock() 提升为 shared_ptr,对象已销毁则得到空指针。
- 栈对象离开作用域必然调用析构函数,这是语言保证,不依赖程序员记忆;无论函数正常返回还是中途抛异常,析构都执行。典型场景:打开文件后中间代码抛异常,若不用 RAII 包装,文件将永远不关闭。

实验:员工与部门的内存安全示例

本实验用一个"员工链表 + 部门共享 + 日志 RAII"的完整程序,串联本章全部知识点:new/delete、三大件、五法则的移动、RAII、unique_ptr、shared_ptr、weak_ptr 与循环引用修复、内存泄漏自检。程序构造三名员工组成链表,员工通过 weak_ptr 回指部门,最后用静态计数验证析构完全、无泄漏。代码分三部分,覆盖点已在注释中标明。

实验 5•第一部分:五法则类(覆盖:new/delete、三大件、五法则移动)

```
#include <iostream>
#include <string>
#include <memory>
#include <cstdio>

class Employee {
public:
    Employee(const std::string& n, double s)
        : name(new std::string(n)), salary(s) { ++alive; }

    Employee(const Employee& o) // 三大件:拷贝构造
        : name(new std::string(*o.name)), salary(o.salary) { ++alive; }

    Employee& operator=(const Employee& o) { // 三大件:拷贝赋值
        if (this != &o) {
            *name = *o.name; // 深拷贝,不共享
            salary = o.salary;
        }
        return *this;
    }

    Employee(Employee&& o) noexcept // 五法则:移动构造
        : name(o.name), salary(o.salary) { o.name = nullptr; }

    Employee& operator=(Employee&& o) noexcept { // 五法则:移动赋值
        if (this != &o) {
            delete name;
            name = o.name;
            salary = o.salary;
            o.name = nullptr;
        }
        return *this;
    }

    ~Employee() { delete name; --alive; } // 三大件:析构

    const std::string& getName() const { return *name; }
    double getSalary() const { return salary; }
    void raise(double by) { salary += by; }
    static int aliveCount() { return alive; }

private:
    std::string* name; // 手动管理的堆内存
    double salary;
    inline static int alive = 0; // 存活计数:结束时为 0 即无泄漏
};
```

实验 5•第二部分:RAII 日志与智能指针(覆盖:RAII、unique_ptr、shared_ptr、weak_ptr)

```
class LogGuard {                                // 覆盖:RAII,析构自动关文件
public:
    explicit LogGuard(const char* path)
        : f(fopen(path, "w")) {
        if (!f) throw std::runtime_error("日志打开失败");
    }
    ~LogGuard() { if (f) fclose(f); }
    void log(const std::string& s) { fputs(s.c_str(), f); }
    LogGuard(const LogGuard&) = delete;
    LogGuard& operator=(const LogGuard&) = delete;
private:
    FILE* f;
};

struct Department {                             // 覆盖:shared_ptr 共享持有
    std::string name;
    int quota;
    Department(const std::string& n, int q) : name(n), quota(q) {}
};

struct EmpNode {                                // 覆盖:unique_ptr 链表节点
    Employee emp;
    std::unique_ptr<EmpNode> next;
    std::weak_ptr<Department> dept;    // 覆盖:weak_ptr 回指,不增计数
    EmpNode(Employee e, const std::shared_ptr<Department>& d)
        : emp(std::move(e)), dept(d) {}
};

std::unique_ptr<EmpNode> buildList(const std::shared_ptr<Department>& d) {
    auto head = std::make_unique<EmpNode>(Employee("张三", 8000), d);
    head->next = std::make_unique<EmpNode>(Employee("李四", 9000), d);
    head->next->next = std::make_unique<EmpNode>(Employee("王五", 7500), d);
    return head;                               // 所有权随返回值转移
}
```

实验 5•第三部分:main 验证(覆盖:weak_ptr.lock、make_shared、五法则综合、new/delete 配对)

```

int main() {
    LogGuard log("dept.log");           // RAII:结束自动关闭

    auto dept = std::make_shared<Department>("研发部", 10);
    auto list = buildList(dept);        // unique_ptr 链表

    std::shared_ptr<Department> got = list->dept.lock(); // 提升为 shared_ptr
    if (got) {
        log.log("部门: " + got->name + ", 名额 " + std::to_string(got->quota));
    }

    Employee copy = list->emp;           // 拷贝构造:深拷贝
    Employee moved = std::move(copy); // 移动构造:接管后 copy 为空
    moved.raise(500);
    log.log("调动: " + moved.getName() + " -> " + std::to_string(moved.getSalary()));

    double total = 0.0;
    for (const EmpNode* p = list.get(); p; p = p->next.get()) {
        total += p->emp.getSalary();
    }
    log.log("工资总额: " + std::to_string(total));

    int* raw = new int(42);              // 覆盖:new/delete 手工配对
    std::cout << "原始值 " << *raw << ", 存活员工 " << Employee::aliveCount() << "\n";
    delete raw;

    return 0;                            // 链表、部门、日志自动析构
    // 程序结束前打印存活员工数:应回到 0,证明无泄漏
}

```

实验覆盖点核对:new/delete(Employee 构造与析构、main 中的 raw)、三大件(拷贝构造与拷贝赋值深拷贝)、五法则(移动构造与移动赋值置空源指针)、RAII(LogGuard 自动关文件,即使抛异常)、unique_ptr(链表独占节点,自动 delete[] 与 delete)、shared_ptr(make_shared 的部门)、weak_ptr(员工回指部门不增计数,lock 提升判空)。运行程序观察析构顺序与最后的存活计数,把 LogGuard 换成裸 fopen 再故意抛异常,对比文件是否关闭,体会 RAII 的价值。

工程建议 1:默认使用智能指针与容器

新代码中裸 new 只出现在 make_unique/make_shared 里,裸 delete 根本不该出现;字符串、数组一律交给 std::string 与 std::vector。它们都是 RAII 对象,自己管理内存,你的代码里将没有一条手工释放语句,内存错误先少一半。

工程建议 2:开 ASan 跑测试

本地编译加 -fsanitize=address -fno-omit-frame-pointer,持续集成里也跑一遍;越界、悬空、泄漏都会带精确行号报出来。它是 C/C++ 工程性价比最高的内存体检工具,配合 valgrind 双保险,内存 bug 无处遁形。

本章小结

内存管理的全貌:new/delete 把分配与构造、析构与释放绑定在一起;三大件与五法则保证手动管理资源的类拷贝安全;RAII 让资源释放成为语言保证;unique_ptr、shared_ptr、weak_ptr 把所有权语义写进类型,循环引用用 weak_ptr 破解。下一章进入面向对象:类、继承与多态。

🎯 本章目标

掌握 C++ 面向对象的核心机制:封装与访问控制、继承与组合、虚函数多态与虚表、多继承与虚继承、运算符重载。理解每个机制在底层是什么(C 程序员最能接受的视角),并能在工程中正确选择继承、组合与接口。

📦 前置知识

熟悉结构体与函数指针(第1、4章);了解构造函数与析构函数、this 指针(第4、5章)。本章代码基于 C++17,多态部分需要智能指针配合(第5章)。

§6.1 封装与访问控制	§6.2 继承与组合	§6.3 虚函数与虚表
§6.4 多继承与虚继承	§6.5 运算符重载	§6.6 设计对比与取舍

§6.1 封装与访问控制

类的封装有两层含义:数据与操作绑定在一起(成员函数直接操作自己的成员),外部只能通过公开接口访问内部状态。C++ 用三个关键字控制访问:public 任何代码可访问;private 只有本类的成员函数可访问;protected 本类与派生类可访问。struct 与 class 的唯一区别是默认访问级别:struct 默认 public,class 默认 private——这是 C 程序员最容易踩的第一个坑。

封装的工程价值在修改成本:内部实现怎么改都不影响调用方,只要公开接口不变。C 的结构体把所有字段暴露给所有人,改一个字段名,全部使用点全部要改;C++ 把字段放进 private,外部只能通过接口访问,内部从裸数组换成 vector 都无人察觉。const 成员函数(第4章)把"只读操作"也写进了接口,封装与不变式一起构成类的契约。

C 的写法	C++ 的写法	说明
struct Student { char name[32]; double score; };	class Student { private: ... public: ... };	C 字段全裸,C++ 默认私有强制走接口
全局函数操作结构体	成员函数直接访问私有字段	C++ 操作与数据同处一个作用域
改字段名牵连全部调用点	改实现不影响调用方	封装隔离变化,降低维护成本

清单 6-1 封装与访问控制

```

#include <iostream>
#include <vector>

class BankAccount {
public:
    explicit BankAccount(double init) : balance(init) {}

    void deposit(double v) {
        if (v > 0) balance += v;          // 不变式:余额永远非负
    }

    bool withdraw(double v) {
        if (v < 0 || v > balance) return false;    // 拒绝透支
        balance -= v;
        return true;
    }

    double getBalance() const { return balance; }    // 只读接口

private:
    double balance;                                // 私有:外部无法直接修改
};

int main() {
    BankAccount acc(100);
    acc.deposit(50);
    if (acc.withdraw(200)) std::cout << "取款成功\n";
    else std::cout << "余额不足:" << acc.getBalance() << "\n";
    // acc.balance = -999;                        // 错误:private 不可访问
    return 0;
}

```

§6.2 继承与组合

继承建立"is-a"关系:派生类是一个基类,自动拥有基类的公开接口,并可以扩展或改写。组合建立"has-a"关系:一个类持有另一个类的对象作为成员。经验法则:能组合就不继承——组合更灵活,运行期可替换,没有继承带来的耦合;只有语义上确定"派生类确实是一种基类",并且需要多态(见下节)时才用继承。判断方法叫 Liskov 替换:任何使用基类的地方,换成派生类行为都不应出错。

构造与析构的顺序是继承的基础纪律:构造时先基类后成员最后自己的构造体,析构时严格相反。这意味着派生类的构造函数在基类构造完成之前不能访问基类成员——初始化列表里的基类初始化先于一切。C 没有继承语法,老代码用"结构体里放结构体"模拟,但类型之间没有"is-a"关系,也无法多态,只是数据布局上的巧合。

C 的写法	C++ 的写法	说明
struct Manager { struct Employee base; ... };	class Manager : public Employee { ... };	C 是组合模拟,C++ 有真正的 is-a 关系
mgr.base.name 访问基类字段	mgr.name 直接访问	C++ 基类成员对派生类无缝可见
无法把 Manager* 当 Employee* 用	Manager* 隐式转换为 Employee*	C++ 派生类指针可当基类指针用

清单 6-2 继承的构造析构顺序

```
#include <iostream>

class Base {
public:
    Base() { std::cout << "1 基类构造\n"; }
    virtual ~Base() { std::cout << "3 基类析构\n"; }
};

class Member {
public:
    Member() { std::cout << "2 成员构造\n"; }
    ~Member() { std::cout << "4 成员析构\n"; }
};

class Derived : public Base {
public:
    Derived() : Base(), m() { std::cout << "5 派生类构造\n"; }
    ~Derived() override { std::cout << "6 派生类析构\n"; }
private:
    Member m;          // 组合成员
};

int main() {
    Derived d;          // 输出顺序:1 基类 → 2 成员 → 5 派生类
    return 0;           // 析构顺序:6 → 4 → 3,与构造完全相反
}
```

§6.3 虚函数与虚表:多态的实现

多态的核心需求:通过基类指针调用函数,实际执行的是派生类的版本。C++ 用 `virtual` 关键字声明虚函数,调用时按对象的实际类型动态绑定。机制是虚表:每个含虚函数的类编译期生成一张虚函数表(vtable),对象里藏一个指向虚表的指针(vptr);通过基类指针调用虚函数时,编译器生成"取 vptr → 查表 → 间接调用"的代码,实际调用哪个函数由对象自己决定。这与 C 里 FILE 结构体的函数指针表是同一个套路,只不过 C++ 是编译器自动生成。

三条配套纪律。第一,基类析构函数必须是 `virtual`:通过基类指针 `delete` 派生对象时,只有析构是虚函数才会先调用派生类析构再调用基类析构,否则派生类部分漏析构,资源泄漏。第二,C++11 起用 `override` 标记重写,用 `final` 禁止继续派生,编译器会检查 `override` 是否真的重写了基类虚函数——拼错签名立刻报错。第三,纯虚函数让类变成抽象类:抽象类不能实例化,只能作为接口与骨架,派生类必须实现全部纯虚函数才能创建对象。

C 的写法	C++ 的写法	说明
<code>struct FileOps { int (*read)(...); ... };</code>	<code>class Stream { virtual int read(...) = 0; };</code>	C 手写函数指针表,C++ 虚表自动生成
<code>obj->ops->read(obj, buf, n)</code>	<code>obj->read(buf, n)</code>	C 显式传对象,C++ 隐式 <code>this</code>
换实现要手工改函数指针表	继承 + <code>override</code> 即可	C++ 扩展新类型不改既有代码

清单 6-3 虚函数、纯虚函数与虚析构

```

#include <iostream>
#include <memory>
#include <vector>

class Shape {                                // 抽象类:含纯虚函数
public:
    virtual ~Shape() = default;              // 虚析构:多态删除的安全保障
    virtual double area() const = 0;         // 纯虚函数:必须由派生类实现
};

class Circle final : public Shape { // final:不许再被继承
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
private:
    double r;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
private:
    double w, h;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rect>(3.0, 4.0));
    for (const auto& s : shapes) { // 同一接口,各自实现
        std::cout << "面积:" << s->area() << "\n";
    }
    return 0;
}

```

⚠ 易错点 1:析构函数不是虚函数

class Base 有虚函数却写普通析构,通过 Base* delete 派生对象时,只有 Base 的析构被调用,派生类的成员与资源不释放,悄悄泄漏。规则一句话:类里有虚函数,析构函数就必须是 virtual,这是编译器无法替你检查的纪律。

⚠ 易错点 2:构造函数里调用虚函数不会多态

构造派生类时,基类构造阶段虚表还指向基类,调用虚函数执行的是基类版本;析构时相反。想在构造期间完成"虚初始化"是错的,应该用工厂函数或两阶段构造,不要在构造/析构函数里依赖虚调用。

§6.4 多继承与虚继承

一个类可以有多个基类,这叫多继承。多继承最常见的用途是"接口式继承":一个实现类同时继承一个功能基类和几个纯接口(如 Drawable、Serializable),获得多方面的能力。风险在菱形继承:类 A 有数据,B 和 C 都继承 A,D 同时继承 B 和 C,那么 D 里会有两份 A 的子对象,成员访问产生歧义,数据冗余,语义混乱。

解决办法是虚继承:把 A 声明为虚基类(virtual public A),B 与 C 共享同一份 A 子对象,菱形变成共享的三角。虚继承有代价:虚基类子对象的地址需要运行时偏移计算,对象内存布局复杂,构造时虚基类只能由最派生类直接初始化。因此虚继承只在"确实需要共享基类状态"时使用;日常编码用接口式多继承 + 组合,能避开大多数菱形场景。

清单 6-4 菱形继承与虚继承修复

```
#include <iostream>
#include <string>

class Person {                      // 虚基类:共享唯一子对象
public:
    explicit Person(std::string n) : name(n) {}
    std::string name;
};

class Student : virtual public Person { // 虚继承
public:
    Student(std::string n, double g)
        : Person(n), grade(g) {}
    double grade;
};

class Teacher : virtual public Person { // 虚继承
public:
    Teacher(std::string n, int y)
        : Person(n), years(y) {}
    int years;
};

// 既是学生又是教师:D 中只有一份 Person
class TeachingAssistant : public Student, public Teacher {
public:
    TeachingAssistant(std::string n, double g, int y)
        : Person(n), Student(n, g), Teacher(n, y) {}
    void who() const {
        std::cout << name << ": 成绩 " << grade
                    << ", 教龄 " << years << " 年\n";
    }
};

int main() {
    TeachingAssistant ta("张三", 92.0, 3);
    ta.who();                      // name 无歧义:虚继承保证唯一
    return 0;
}
```

§6.5 运算符重载

运算符重载让自定义类型使用与内置类型一致的表达式语法: $a + b$ 、 $v[i]$ 、 $p \rightarrow x$ 。C++ 允许重载绝大多数运算符,但有几条铁律:不能发明新运算符(如 $**$),不能改变优先级与结合性,不能改变操作数个数;重载后不保证"短路的 $\&\&$ 、 $\|\|$ "与求值顺序的语义——所以工程上严禁重载逻辑与、逻辑或和逗号运算符。比较运算符最常见,重载 $==$ 时通常配套重载 $!=$,否则两个运算符语义不一致。

实现形态有两种:成员函数重载,左操作数必须是本类对象(如 $a + b$ 中 a 是本类);非成员函数重载,适合左操作数是内置类型或需要隐式转换的场景。输出运算符与输入运算符(左操作数是流)只能写成非成员函数,且要声明为类的 friend 才能访问私有成员。friend 的意义是"指定外部函数为友",它只是破例放行,不是反向的 public——应保持最小化。

C 的写法	C++ 的写法	说明
Point add_point(p, q);	Point r = p + q;	C 手写函数名,C++ 表达式自然可读
int eq_point(p, q);	if (p == q) ...	C 靠命名约定,C++ 运算符语义内建
printf("%d,%d", p.x, p.y)	std::cout << p	C++ 重载输出流运算符,格式随类型

清单 6-5 运算符重载:复数类

```
#include <iostream>

class Complex {
public:
    Complex(double r = 0, double i = 0) : re(r), im(i) {}

    Complex operator+(const Complex& o) const {    // 成员重载 +
        return Complex(re + o.re, im + o.im);
    }
    Complex operator-() const { return Complex(-re, -im); } // 一元负号
    bool operator==(const Complex& o) const {
        return re == o.re && im == o.im;
    }

    friend std::ostream& operator<<(std::ostream& os, const Complex& c);

private:
    double re, im;
};

std::ostream& operator<<(std::ostream& os, const Complex& c) {
    return os << c.re << (c.im >= 0 ? "+" : "") << c.im << "i";
}

int main() {
    Complex a(1, 2), b(3, 4);
    Complex c = a + b;           // 调用 operator+
    std::cout << "a+b=" << c << "\n";           // 输出 4+6i
    std::cout << "-a=" << -a << "\n";           // 输出 -1-2i
    std::cout << (c == Complex(4, 6) ? "相等" : "不等") << "\n";
    return 0;
}
```

 **易错点 3:重载运算符破坏直觉语义**

重载 && 和 || 会丢失短路求值与求值顺序,重载逗号运算符会打乱表达式语义,这些运算符被禁得越远越好;重载 == 却不重载 !=、重载 < 却不重载 > 也让调用方困惑。原则:重载运算符必须保持内置运算符的直觉语义,拿不准就不重载,改用命名函数。

§6.6 设计对比与取舍

把 C 与 C++ 的面向对象放在一起看:数据 + 函数指针表就是 vtable 的雏形,结构体嵌套就是继承的雏形,显式传对象指针就是 this 的雏形——C++ 的全部机制都能在 C 里找到手工版本,只是编译器接管了样板代码与安全检查。差别在于:C 的模拟是约定,靠程序员自觉;C++ 是语言规则,靠编译器强制。同样的代码,C 的版本漏改一处就出 bug,C++ 的版本编译期就拒绝。

工程上的取舍建议:接口(纯虚函数)定义能力边界,功能基类承载通用实现,组合表达依赖关系,虚继承只用于真正的共享菱形。多态容器用 unique_ptr 或 shared_ptr 持有基类指针(第5章),析构顺序与虚析构配合,内存与对象模型两不误。面向对象不是目的,可维护性才是;能用值语义与算法解决的需求,不必硬造继承树。

面向对象速查表

概念	语法示例	要点
封装	private 字段 + public 接口	实现与接口分离,变化隔离
继承	class D : public B	is-a 关系,自动获得基类接口
组合	class D { B b; }	has-a 关系,优先于继承
虚函数	virtual double area();	动态绑定,运行时按对象类型分派
虚析构	virtual ~Base() = default;	基类有虚函数就必须有虚析构
override	double area() override;	编译器核对确实重写了虚函数
final	class Circle final : public Shape	禁止继承或继续重写
纯虚函数	virtual void f() = 0;	抽象类不可实例化,派生类必须实现
抽象类	class Shape { virtual ... = 0; }	定义接口,不做实现
多继承	class D : public B, public C	接口式多继承,警惕菱形
虚继承	class B : virtual public A	菱形共享唯一子对象
运算符重载	Complex operator+(const Complex&)	保持直觉语义,不重载 && 与逗号
friend	friend std::ostream& operator<<(...)	指定友元,只给必要的函数放行
虚表 vtable	编译器自动生成	虚函数查表间接调用,略有开销

自测题

- Q 1. struct 与 class 的区别是什么?为什么封装能降低维护成本?
- Q 2. 继承与组合分别表达什么关系?为什么"能组合就不继承"?
- Q 3. 虚函数多态的底层机制是什么?为什么基类析构必须声明为 virtual?
- Q 4. 菱形继承有什么问题?虚继承如何解决?虚继承有什么代价?
- Q 5. 运算符重载有哪些铁律?为什么严禁重载逻辑与和逻辑或?

✅ 参考答案

1. 两者唯一的区别是默认访问级别:struct 默认 public,class 默认 private。封装把实现细节关进私有区,内部改动只要接口不变,调用方零修改;同时不变式(如余额非负)由成员函数统一维护,外部无法绕过。
2. 继承是 is-a(派生类是一种基类),组合是 has-a(对象内部持有另一个对象)。组合耦合更弱、可运行期替换,继承则把派生类与基类实现绑定,改动基类常波及所有派生类。语义上不是 is-a 就不该继承。
3. 每个含虚函数的类有一张虚表,对象携带虚表指针;虚调用是"取指针、查表、间接跳转"。基类析构若非虚,通过基类指针 delete 派生对象只析构基类部分,派生类资源泄漏;声明为 virtual 后删除按实际类型走完整的析构链。
4. 菱形中 D 继承 B 与 C,而 B、C 各有 A 的子对象,D 中就有两份 A 的成员,访问歧义且数据重复。虚继承让 B、C 共享同一份 A 子对象,歧义消失;代价是虚基类偏移需运行时计算,且虚基类构造只能由最派生类负责。
5. 不能发明新运算符、不能改优先级与结合性、不能改操作数个数;重载后的逻辑与、逻辑或不再短路,求值顺序也不保证,违背内置语义,工程上直接禁用;重载 == 要配套 !=,输出运算符要返回流引用。

实验:图形系统

本实验把本章机制用在一个完整的图形系统上:抽象基类定义接口,派生类实现面积计算,接口式多继承让图形可绘制,运算符重载让点支持加减与相等比较,多态容器统一管理。代码分三部分,覆盖点已在注释中标明。

实验 6•第一部分:抽象基类与多继承(覆盖:纯虚函数、虚析构、override、多继承)

```

#include <iostream>
#include <string>
#include <vector>
#include <memory>

class Shape {                                // 覆盖:抽象类 + 纯虚函数
public:
    virtual ~Shape() = default;              // 覆盖:虚析构,多态删除安全
    virtual double area() const = 0;
};

class Drawable {                             // 覆盖:接口式基类(纯接口)
public:
    virtual ~Drawable() = default;
    virtual void draw() const = 0;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
protected:
    double w, h;
};

// 覆盖:多继承——同时是形状与可绘制物
class Square : public Rect, public Drawable {
public:
    explicit Square(double side) : Rect(side, side) {}
    double area() const override { return w * w; }
    void draw() const override {
        std::cout << "绘制正方形,边长 " << w << "\n";
    }
};

class Circle : public Shape, public Drawable {
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    void draw() const override {
        std::cout << "绘制圆形,半径 " << r << "\n";
    }
private:
    double r;
};

```

实验 6·第二部分:运算符重载(覆盖:成员重载、一元运算符、friend 输出)

```

class Point {
public:
    Point(double x = 0, double y = 0) : x(x), y(y) {}

    Point operator+(const Point& o) const {          // 成员重载 +
        return Point(x + o.x, y + o.y);
    }
    Point operator-(const Point& o) const {          // 成员重载 -
        return Point(x - o.x, y - o.y);
    }
    Point operator*(double k) const {                // 数乘
        return Point(x * k, y * k);
    }
    bool operator==(const Point& o) const {          // 相等比较
        return x == o.x && y == o.y;
    }
    double norm() const { return x * x + y * y; }

    friend std::ostream& operator<<(std::ostream& os, const Point& p);

private:
    double x, y;
};

std::ostream& operator<<(std::ostream& os, const Point& p) {
    return os << "(" << p.x << ", " << p.y << ")";
}

```

实验 6•第三部分:多态容器与组合(覆盖:虚表动态绑定、unique_ptr 多态、组合)

```

int main() {
    // 覆盖:多态容器—基类指针持有不同派生对象
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Rect>(3.0, 4.0));
    shapes.push_back(std::make_unique<Square>(2.0));
    shapes.push_back(std::make_unique<Circle>(1.0));

    double total = 0.0;
    for (const auto& s : shapes) {
        total += s->area();           // 覆盖:虚函数动态绑定
        std::cout << "面积 " << s->area() << "\n";
    }
    std::cout << "总面积 " << total << "\n";

    // 覆盖:多继承对象的双接口使用
    Square sq(3.0);
    Shape& asShape = sq;              // 向上转型:is-a
    Drawable& asDraw = sq;
    std::cout << "正方形面积 " << asShape.area() << "\n";
    asDraw.draw();

    // 覆盖:运算符重载综合
    Point a(1, 2), b(3, 4);
    Point c = a + b - Point(1, 1);
    std::cout << c << (c == Point(3, 5) ? " 验证通过" : " 验证失败") << "\n";
    return 0;
}

```

实验覆盖点核对:封装(BankAccount 思路在 §6.1 已练)、继承与组合(Shape/Rect/Square 链与 shapes 容器)、虚函数与虚表(area 的动态绑定)、虚析构(基类声明,智能指针自动调用)、override 与 final、纯虚函数与抽象类(Shape/Drawable)、多继承(Square 同时继承 Rect 与 Drawable)、运算符重载(Point 的加减乘、相等、输出)、friend(输出运算符)。自行扩展:给 Point 增加 < 运算符后放入 std::map,或给 Circle 增加放大函数,体会"改一个类不动其他代码"的多态红利。

工程建议 1:接口与实现分离

定义功能的抽象基类只放纯虚函数,不放数据成员;实现细节全在派生类。这样新增一种图形只需写一个新类,既有代码零修改——开闭原则的直接落地,也是多态价值的最大化。

工程建议 2:多态对象交给智能指针

多态对象一律用 unique_ptr 或 shared_ptr 持有,容器只放指针:对象大小不同,值语义无法装进同一个 vector。配合虚析构与 make_unique,释放、扩容、异常路径全部自动,内存模型与对象模型互不打架。

本章小结

面向对象四件套:封装把实现关进私有区,继承与组合表达类型关系,虚函数用虚表实现动态绑定,运算符重载让自定义类型拥有自然语法。多继承与虚继承解决接口扩展与菱形共享,但日常优先组合与接口。下一章讲文件 I/O:从 C 的 FILE 到 C++ 的流与文件系统。

🔗 本章目标

掌握 C++ 的流式 I/O:文本读写、逐行解析、字符串流、二进制读写,以及 C++17 的 `std::filesystem` 文件系统库。理解 `iostream` 相比 `stdio` 的类型安全与 RAII 优势,能够写出跨平台、健壮的文件处理代码。

📦 前置知识

熟练使用 C 的 `fopen/fprintf/fscanf` 与 `fread/fwrite`(第5章 RAII 已铺垫);了解 `string` 与 `vector`(第1、3章)。本章代码基于 C++17,文件系统库需编译器支持 `<filesystem>`。

§7.1 从 FILE 到流	§7.2 文本文件读写	§7.3 字符串流与格式化
§7.4 二进制读写	§7.5 <code>std::filesystem</code>	§7.6 错误处理与性能

§7.1 从 C 的 FILE 到 C++ 的流

C 的文件 I/O 围绕 `FILE*` 展开:`fopen` 打开, `fprintf/fscanf` 格式化读写, `fread/fwrite` 二进制读写, `fclose` 关闭。C++ 换成了流(stream)模型:`std::ofstream` 写文件、`std::ifstream` 读文件、`std::fstream` 读写兼可,配合插入运算符与提取运算符使用。流的第一个优势是类型安全:`fprintf` 的格式串与实参类型不匹配在 C 里是未定义行为,经常悄无声息地读错内存;而 `<<` 与 `>>` 是重载运算符(第6章),`int` 调 `int` 版本,`double` 调 `double` 版本,类型错了编译期就报错。

流的第二个优势是 RAII(第5章):文件流对象离开作用域自动关闭文件,不需要也不能手工 `fclose`;C 的 `fopen` 之后每条出错路径都要写一遍 `fclose`,漏一条就泄漏句柄。第三个优势是状态机:流的 `good/fail/eof/bad` 四种状态可以显式检查,把"文件读完了"与"读出错误"区分开,而 C 的 `feof/ferror` 组合很容易写错。

流的另一个实用细节是缓冲:`cout` 与文件流默认都有缓冲,写入的内容先进缓冲区,凑满或主动刷新才真正落盘。需要立即落盘(如日志、崩溃前的心跳)时调用 `out.flush()`,或者用 `endl` 换行并强制刷新——但 `endl` 有性能代价,普通换行请写 `\n`。C 的 `printf` 与 C++ 的 `cout` 混用时要注意同步:默认两者共享底层缓冲,顺序有保证,代价是性能;高频输出时可以用 `ios::sync_with_stdio(false)` 关闭同步换取速度,前提是保证不再混用 `printf`。

C 的写法	C++ 的写法	说明
<code>FILE* f = fopen(path, "r");</code>	<code>std::ifstream in(path);</code>	C++ 构造即打开,析构自动关闭
<code>fprintf(f, "%d", x);</code> 类型不符是 UB	<code>out << x;</code> 类型由重载保证	C++ 类型安全,错误编译期暴露
<code>feof/ferror</code> 组合判断	<code>in.good() / in.fail() / in.eof()</code>	C++ 流状态明确可查
<code>fclose</code> 每路径手工写	析构自动关闭	RAII 保证无句柄泄漏

清单 7-1 流的状态检查

```

#include <iostream>
#include <fstream>

int main() {
    std::ifstream in("no_such_file.txt"); // 打开不存在的文件
    if (!in) {                             // 或 in.fail():打开失败
        std::cerr << "打开失败\n";
        return 1;
    }
    std::string word;
    while (in >> word) {                    // 读到文件尾之前持续循环
        std::cout << word << " ";
    }
    if (in.eof()) std::cout << "(正常读完)\n";
    if (in.bad()) std::cout << "(读取出错)\n";
    return 0;
}

```

§7.2 文本文件读写

写文本文件用 `ofstream`:默认以截断模式打开,内容直接覆盖;需要追加时传 `std::ios::app`。打开模式可以组合,例如 `std::ios::app | std::ios::out`。打开之后必须检查流是否进入失败状态——文件路径不存在、权限不足都会导致静默失败,不检查的后果是数据凭空消失。读文本文件最常用的是 `getline`:每次取一整行,包括空格,适合逐行处理配置、CSV、日志等格式;需要按单词或数字读取时用提取运算符,它会跳过空白。

与 C 对照:`fscanf` 遇到空白就停,读"张三 85"这类行必须手写复杂的格式串;`getline` 加 `stringstream`(下节)可以把"读一行、拆字段"拆成两步,每一步都直白。另外注意 Windows 与 Linux 换行符差异:文本模式下 C++ 流自动把 `"\n"` 转换成平台换行,二进制模式不做转换,所以二进制文件必须显式加 `std::ios::binary`,否则读写内容错位。

C 的写法	C++ 的写法	说明
<code>char buf[256]; fgets(buf, 256, f);</code>	<code>std::string line; getline(in, line);</code>	C 固定缓冲易截断,C++ 动态字符串自动扩容
<code>sscanf(buf, "%s %lf", name, &v);</code>	<code>ss.str() + stringstream 拆分</code>	C 格式串脆弱,C++ 分步解析直观
<code>fopen(path, "a")</code> 追加	<code>ofstream out(path, std::ios::app)</code>	同样的追加语义,RAII 关闭

清单 7-2 写文件与逐行读取

```

#include <iostream>
#include <fstream>
#include <string>

int main() {
    // 写:截断模式;若要追加改为 std::ios::app
    {
        std::ofstream out("notes.txt");
        if (!out) return 1;
        out << "第一行:你好\n";
        out << "第二行:" << 42 << "\n";    // 数字自动转文本
    }    // 离开作用域自动关闭

    // 读:逐行
    std::ifstream in("notes.txt");
    if (!in) return 1;
    std::string line;
    int n = 0;
    while (std::getline(in, line)) {        // 读到文件尾返回 false
        std::cout << "第 " << ++n << " 行:" << line << "\n";
    }
    return 0;
}

```

⚠ 易错点 1:getline 与提取运算符混用

先 `in >> x` 再 `getline(in, line)`, `getline` 读到的是 `>>` 留下的换行,得到空行。解决:在两者之间调用 `in.ignore()` 丢弃残留字符,或者统一用 `getline` 取整行再解析,别混着用。

§7.3 字符串流与格式化

`std::stringstream` 把字符串当作流来读写:插入运算符向字符串追加内容,提取运算符从字符串取字段, `getline` 按分隔符拆列。它最常见的两个用途是拼接与解析:拼接时不需要像 `sprintf` 那样手工数缓冲大小,解析 CSV 时先把一行装进 `stringstream`,再用 `getline(ss, field, ',')` 逐个取字段。内存由 `string` 管理,不存在缓冲溢出问题。

格式化输出还有更现代的选择:`std::to_string` 把数值转字符串,适合简单拼接;C++20 的 `std::format` 提供 `printf` 风格的占位符格式化(`{}` 占位),可读性与安全性兼得;老工程里 `snprintf` 仍然常用,但记住它返回写入长度,缓冲要预留 `'\0'` 的空间。`stringstream` 的精度控制用 `out.precision(n)` 或 `<< std::setprecision(n)`(需 `<iomanip>`)。

清单 7-3 stringstream 拼接与解析

```

#include <iostream>
#include <sstream>
#include <string>
#include <iomanip>

int main() {
    // 拼接:数字与文本自由混合
    std::ostringstream report;
    report << "成绩单 " << 2026 << " 年,及格线 "
        << std::setprecision(2) << 59.5 << "\n";
    std::cout << report.str();

    // 解析:"张三,85.5" 按逗号拆字段
    std::string line = "张三,85.5,优";
    std::istringstream ss(line);
    std::string name, score, level;
    std::getline(ss, name, ',');
    std::getline(ss, score, ',');
    std::getline(ss, level, ',');
    std::cout << "姓名 " << name << ", 分数 " << score
        << ", 等级 " << level << "\n";

    return 0;
}

```

§7.4 二进制读写

二进制文件按内存字节原样存储,读写用成员函数 `write` 与 `read`:`write(const char* p, streamsize n)` 把 `n` 字节写入,`read(char* p, streamsize n)` 读 `n` 字节。典型的序列化对象是定长结构体:全部字段是数值或定长数组时,布局是确定的,直接整块读写;但有两个致命陷阱。第一,结构体有对齐填充,不同编译器、不同平台的填充不同,文件换环境就读不对——写入前用 `pragma pack` 或干脆只写字段不写结构体。第二,字节序:大端机器写的整数,小端机器读出来是反的,跨平台传输必须约定字节序并手动转换。

相比之下,`std::string`、`vector` 这类带堆指针的类型绝对不能直接 `write`:写进去的是指针值,换个进程就是悬空地址。文本格式(CSV、JSON)天然跨平台,二进制格式(图片、压缩包、协议帧)追求紧凑与速度。工程决策很简单:需要互操作选文本,追求性能且自产自销选二进制,但必须自己实现"写字段、读字段"的序列化函数,而不是裸写结构体。

两种格式的体积差异也值得记住:文本格式里数字按字符存储,一个 4 字节整数写成文本最多 11 个字符,同量级数据下二进制通常更小,而且没有解析开销;但文本格式可以直接用文本编辑器查看、用 `diff` 对比、用 `grep` 排查,可调试性远超二进制。工程上的务实做法:配置与交换格式用文本(CSV、JSON、INI),缓存与中间产物用二进制,并在二进制文件头部写版本号与魔数,将来改格式时能识别旧文件。

C 的写法	C++ 的写法	说明
<code>fwrite(p, size, n, f)</code>	<code>out.write((const char*)p, n)</code>	语义一致,字节数写法不同
<code>fread(p, size, n, f)</code>	<code>in.read((char*)p, n); in.gcount()</code>	C++ 用 <code>gcount</code> 查实际读入字节
结构体直接 <code>fwrite</code>	手写序列化函数逐字段写	C++ 更强调对齐与指针陷阱的规避

清单 7-4 二进制读写:定长记录

```

#include <iostream>
#include <fstream>
#include <cstdint>

struct Header {          // 定长布局:两个整数,无指针
    std::uint32_t magic;
    std::uint32_t count;
};

int main() {
    const char* path = "data.bin";

    // 写
    {
        std::ofstream out(path, std::ios::binary);    // 二进制必须声明
        if (!out) return 1;
        Header h{0x12345678, 3};
        out.write(reinterpret_cast<const char*>(&h), sizeof(h));
        std::uint16_t vals[3] = {10, 20, 30};
        out.write(reinterpret_cast<const char*>(vals), sizeof(vals));
    }

    // 读
    std::ifstream in(path, std::ios::binary);
    if (!in) return 1;
    Header h;
    in.read(reinterpret_cast<char*>(&h), sizeof(h));
    std::uint16_t vals[3] = {0, 0, 0};
    in.read(reinterpret_cast<char*>(vals), sizeof(vals));
    std::streamsize got = in.gcount();                // 实际读到的字节数

    std::cout << "magic " << std::hex << h.magic << std::dec
        << ", count " << h.count << ", 读到 " << got << " 字节\n";
    for (auto v : vals) std::cout << v << " ";
    std::cout << "\n";
    return 0;
}

```

⚠ 易错点 2:裸写结构体进文件

直接 `out.write((char*)&s, sizeof(s))` 的三大雷区:对齐填充因编译器而异、字节序因平台而异、string 与指针成员写进去的是地址。结果就是“本机自产自销偶尔能用,跨机器必炸”。正确的二进制序列化只写基本类型字段,且逐字段、约定字节序。

§7.5 std::filesystem:文件系统操作

C 操作目录是跨平台噩梦:stat 取信息、dirent 遍历目录、mkdir 在不同平台签名还不一样,还得自己拼接路径字符串。C++17 的 `std::filesystem` 提供了统一、跨平台、类型安全的文件系统接口:fs::path 类型管理路径(自动处理分隔符,Windows 反斜杠与 Linux 斜杠都不怕),exists、is_directory、file_size 查信息,create_directories 递归建目录,directory_iterator 遍历目录,remove、rename 删除与改名。所有函数都有异常版本与 error_code 版本:异常版本默认抛 `fs::filesystem_error`,适合正常流程;error_code 版本不抛异常,适合高频路径与“不存在不算错”的场景。

与 C 对比:opendir/readdir 需要自己判断类型、拼路径、处理隐藏文件,而 directory_iterator 直接给出完整条目对象,entry.path() 就是绝对路径,entry.is_directory() 直接判断。递归遍历用 recursive_directory_iterator,一行声明顶 C 的一页代码。路径拼接用 "/" 运算符:dir / "scores.csv",自动处理分隔符——C 里手拼路径字符串在 Windows 上经常因为漏写反斜杠出 bug。

C 的写法	C++ 的写法	说明
stat(path, &st); st.st_size	fs::file_size(path)	C 填结构体再取字段,C++ 一步到位
DIR* d = opendir(...); readdir(d)	fs::directory_iterator(dir)	C 循环读条目,C++ 范围 for 遍历
mkdir 各平台签名不同	fs::create_directories(p)	C 跨平台苦旅,C++ 统一且可递归
strcat 拼路径	fs::path dir / "a" / "b.txt"	path 类型自动处理分隔符

清单 7-5 filesystem 目录操作

```
#include <iostream>
#include <filesystem>

namespace fs = std::filesystem;    // 惯例:命名空间别名

int main() {
    fs::path dir = "backup";        // path 类型
    fs::create_directories(dir / "sub");    // 递归创建

    std::ofstream(dir / "sub" / "log.txt") << "hello\n";    // 临时文件流

    std::error_code ec;              // 不抛异常版本
    if (!fs::exists(dir, ec)) {
        std::cout << "目录不存在(错误码 " << ec.message() << ")\n";
        return 1;
    }

    for (const auto& entry : fs::recursive_directory_iterator(dir)) {
        std::cout << (entry.is_directory() ? "[目录] " : "[文件] ")
            << entry.path().string() << " "
            << fs::file_size(entry.path()) << " 字节\n";
    }

    fs::remove_all(dir);              // 递归删除,清理现场
    return 0;
}
```

⚠ 易错点 3:目录删除与文件存在性检查的语义

fs::remove 只能删空目录,删非空目录抛异常;需要整树删除用 fs::remove_all。另外"检查 exists 后再操作"存在竞态窗口(检查与操作之间文件可能消失),严格场景直接操作并捕获 filesystem_error,或使用 error_code 版本判断结果。

§7.6 错误处理与性能

流的错误处理要分层:打开失败检查流状态;每次写入后若担心磁盘问题,检查 `out.good()`;读取时区分 `eof`(正常读完)与 `fail`(格式错误)——提取运算符读到类型不匹配的输入会把流置为失败态,此时数据还在缓冲里,可以用 `in.clear()` 复位后继续处理。与 C 的 `errno` 体系相比,流状态是对象自带的,不会在多个调用之间互相污染。

性能方面,高频小写入尽量合并:每写一行都触发系统调用很慢,一次构造整块字符串再写,或依赖流内部缓冲;读文件按行处理比按字符快一个量级,大文件用内存映射(如 `mmap`)或流式读取,别把整个文件读进内存。I/O 是程序里最慢的部分之一,能少读一次就少读一次,能用一次读大块就不用多次读小块。

最后补一个高频陷阱的预防:程序崩溃或断电时,缓冲区的数据会丢失,文件停留在上次刷新的状态。对关键数据要养成“写入后立即刷新或关闭”的习惯,并考虑先写临时文件再原子改名,避免读到半截文件。这套“写临时文件、校验、原子替换”的手法在配置文件更新、状态持久化等场景是标准答案,配合 `filesystem` 的 `rename` 即可实现。

文件 I/O 速查表

概念	语法示例	要点
写文本	<code>std::ofstream out(path);</code>	默认截断,RAII 自动关闭
读文本	<code>std::ifstream in(path);</code>	打开失败要检查流状态
追加模式	<code>ofstream out(p, std::ios::app)</code>	内容续写在文件末尾
逐行读取	<code>while (getline(in, line))</code>	行内含空格,自动扩容
提取数据	<code>in >> x;</code> 跳过空白	类型安全,失败置 <code>fail</code>
字符串流	<code>std::stringstream ss(line)</code>	拼接与拆分字段两用
二进制读写	<code>out.write(p, n); in.read(p, n);</code>	必须加 <code>std::ios::binary</code>
读入字节数	<code>in.gcount()</code>	<code>read</code> 实际读到的数量
路径类型	<code>std::filesystem::path</code>	跨平台分隔符处理
判断存在	<code>fs::exists(p, ec)</code>	<code>error_code</code> 版本不抛异常
创建目录	<code>fs::create_directories(p)</code>	可递归创建多级目录
遍历目录	<code>fs::directory_iterator(d)</code>	<code>recursive_</code> 前缀可递归
文件大小	<code>fs::file_size(p)</code>	目录上调用会抛异常
删除文件	<code>fs::remove(p); fs::remove_all(p)</code>	<code>remove</code> 不删非空目录
流状态	<code>good / fail / eof / bad</code>	读完了 <code>eof</code> ,格式错是 <code>fail</code>

自测题

- Q 1. `iostream` 相比 `stdio` 有哪些优势?为什么说它是类型安全的?
- Q 2. `getline` 与 `>>` 混用会有什么问题?如何规避?
- Q 3. 二进制文件为什么要显式加 `std::ios::binary`?文本模式做了什么?
- Q 4. 直接 `write` 结构体到文件有什么风险?正确的二进制序列化怎么做?
- Q 5. `filesystem` 的异常版本与 `error_code` 版本如何选择?

✅ 参考答案

1. 三方面:类型安全(<< 与 >> 按重载选择版本,格式串不匹配的未定义行为消失);RAII(流析构自动关闭,无句柄泄漏);状态机(good/fail/eof/bad 显式区分正常读完与格式错误)。
2. >> 读完数据后把换行留在流里,紧接着的 getline 读到的是空行。规避:统一用 getline 取整行再解析,或中间调用 in.ignore() 丢弃残留,不要两种读法交替。
3. 文本模式在读写时做换行符转换(Windows 把 "\n" 与 "\r\n" 互转),二进制数据里的 0x0A 会被改掉,内容错位。所以二进制文件必须加 std::ios::binary 关闭转换,保证字节原样。
4. 三大风险:对齐填充因编译器而异,字节序因平台而异,string/指针成员写入的是地址。正确做法:只序列化基本类型字段,逐字段写入并约定字节序,不整块写结构体。
5. 正常流程、预期文件一定存在时用异常版本,错误信息更全;高频路径或"文件可能不存在"是正常状态时用 error_code 版本,避免异常开销与流程污染。

实验:成绩文件管理系统

本实验把本章知识串成完整的成绩管理系统:写 CSV、追加记录、逐行读取解析、统计平均分,并用 filesystem 管理数据目录与遍历。代码分三部分,覆盖点已在注释中标明,运行后 data/scores.csv 会被创建并写满记录。

实验 7•第一部分:写入(覆盖:ofstream、追加模式、打开失败检查、RAII)

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <filesystem>

namespace fs = std::filesystem;    // 覆盖:filesystem 命名空间别名

struct Record {                    // 文本记录:姓名与分数
    std::string name;
    double score;
};

// 覆盖:ofstream 写文件,append 控制覆盖或追加
bool saveCSV(const std::string& path, const std::vector<Record>& rs,
             bool append) {
    std::ofstream out(path,
                      append ? std::ios::app : std::ios::trunc);
    if (!out) return false;        // 覆盖:打开失败必须检查
    for (const auto& r : rs) {
        out << r.name << "," << r.score << "\n";    // 文本格式,跨平台
    }
    return true;                    // 覆盖:离开函数自动 close
}
```

实验 7•第二部分:读取解析(覆盖:ifstream、getline 逐行、stringstream 拆分、stod)

```

bool loadCSV(const std::string& path, std::vector<Record>& rs) {
    std::ifstream in(path);
    if (!in) return false;
    std::string line;
    while (std::getline(in, line)) {          // 覆盖:逐行读取
        if (line.empty()) continue;          // 跳过空行
        std::istringstream ss(line);        // 覆盖:stringstream 解析
        std::string name, score;
        if (std::getline(ss, name, ',') && std::getline(ss, score)) {
            try {
                rs.push_back(Record{name, std::stod(score)});
            } catch (const std::exception&) {
                std::cerr << "格式错误行:" << line << "\n";
            }
        }
    }
    return true;                             // eof 表示正常读完
}

void printAvg(const std::vector<Record>& rs) {
    double sum = 0.0;
    for (const auto& r : rs) sum += r.score;
    std::cout << "共 " << rs.size() << " 条,平均 "
                << sum / rs.size() << "\n";
}

```

实验 7·第三部分:文件系统与 main(覆盖:create_directories、exists、file_size、directory_iterator)

```

int main() {
    fs::path dir = "data";
    fs::create_directories(dir);           // 覆盖:递归创建目录
    fs::path csv = dir / "scores.csv";     // 覆盖:路径拼接

    std::vector<Record> first{"张三", 85.5}, {"李四", 59.0}, {"王五", 73.5}};
    if (!saveCSV(csv.string(), first, false)) { // 覆盖:截断写入
        std::cerr << "写入失败\n";
        return 1;
    }
    std::vector<Record> extra{"赵六", 91.0}};
    if (!saveCSV(csv.string(), extra, true)) { // 覆盖:追加写入
        std::cerr << "追加失败\n";
        return 1;
    }

    std::vector<Record> loaded;
    if (!loadCSV(csv.string(), loaded)) {
        std::cerr << "读取失败\n";
        return 1;
    }
    for (const auto& r : loaded) {
        std::cout << r.name << ": " << r.score << "\n";
    }
    printAvg(loaded);

    // 覆盖:exists 检查 + 目录遍历 + 文件大小
    if (fs::exists(csv)) {
        std::cout << "文件大小 " << fs::file_size(csv) << " 字节\n";
    }
    for (const auto& entry : fs::directory_iterator(dir)) {
        std::cout << "目录条目:" << entry.path().filename().string()
            << "\n";
    }
    return 0;
}

```

实验覆盖点核对:流类体系(ofstream/ifstream 分工)、打开失败检查、追加与截断模式、getline 逐行读取、stringstream 拆分字段、stod 与异常处理、RAII 自动关闭(三个文件流全部无手工 close)、filesystem 的 path 拼接、create_directories、exists、file_size、directory_iterator。扩展练习:把 Record 增加一列等级并修改解析;或把 saveCSV 改成二进制版本,体会两种格式的取舍。

工程建议 1:文件操作统一封装

项目里所有文件读写走少量封装函数(如本章的 saveCSV/loadCSV),统一处理打开失败、格式错误与编码问题。调用方不碰裸流,换格式(CSV 改 JSON)只改封装内部,全局调用点零修改。

工程建议 2:路径一律用 filesystem

字符串拼路径、手写分隔符判断是跨平台 bug 的高发区。路径一律用 fs::path 与 "/" 拼接,遍历用 directory_iterator,删除用 remove_all——Windows 与 Linux 的行为差异由标准库吸收,代码不再需要 #ifdef _WIN32。

本章小结

流式 I/O 取代 stdio:类型安全的运算符、RAII 的自动关闭、明确的状态机;getline 与 stringstream 把文本解析变成两步直白的操作;二进制读写记住定长与字节序;filesystem 让目录操作跨平台且类型安全。下一章进入网络通信:套接字与协议。

🔗 本章目标

理解 TCP/UDP 与客户端-服务端模型;掌握套接字编程全流程:创建、绑定、监听、接受、连接、收发;用 RAII 封装套接字资源,处理粘包、字节序、超时与错误;能够手写 HTTP 请求并理解常见网络库的选型。

📦 前置知识

熟悉 C 的套接字接口或至少理解文件描述符;掌握第5章 RAII 与第7章错误处理思路。本章代码基于 POSIX 套接字(Linux/macOS),Windows 需要额外调用 WSStartup,差异见 §8.2。

§8.1 网络模型速览	§8.2 套接字与 RAII 封装	§8.3 TCP 服务端
§8.4 TCP 客户端	§8.5 HTTP 与网络库	§8.6 字节序、超时与安全

§8.1 网络模型速览

网络编程只需要抓住两层:TCP 与 UDP 之上的应用层协议,和 TCP/IP 之下的传输层。TCP 是面向连接的可靠字节流:三次握手建立连接,数据按序到达,丢失自动重传,适合需要完整性的场景(HTTP、文件传输、聊天);UDP 是无连接的数据报,不保证到达与顺序,延迟低,适合实时场景(音视频、游戏、DNS)。C 程序员已经熟悉 socket 系列函数,这些在 C++ 里原样保留——C++ 不发明新的系统调用,而是用 RAII、异常与标准库把同样的事情做得更安全。

客户端-服务端模型是网络编程的基本骨架:服务端创建套接字、绑定端口、监听,然后循环接受连接;客户端创建套接字、连接服务端地址,然后双方收发数据。全程的数据都是字节流,协议(HTTP、自定义消息)负责把这些字节组织成有意义的信息。理解这个模型,网络编程的七成概念就通了:剩下的三成是细节——粘包、字节序、超时、并发,本章逐一处理。

还有两个基础概念要提前建立:端口与套接字。一台机器上可以同时运行许多网络程序,端口号(0 到 65535)用来区分它们,服务端 bind 的是端口,客户端 connect 的也是端口;套接字则是"网络连接的文件描述符",读写它就像读写文件——C 程序员把文件描述符的直觉搬过来,网络代码立刻亲切一半。端口号小于 1024 的通常需要管理员权限,调试用 8000 以上的高位端口即可。

§8.2 套接字与 RAII 封装

C 的套接字代码有两个顽疾:文件描述符必须每路径 close,错误全靠 errno 与返回值判断。C++ 的做法是照搬第5章的内存管理经验:把 fd 包进 RAII 类,析构自动关闭;错误检查封装成辅助函数,统一抛出异常。这样套接字与内存一样,"创建即管理,结束即释放",任何提前 return 或异常路径都不会泄漏 fd——fd 是稀缺资源,泄漏多了进程就再也打不开文件。

平台差异先说清:Windows 上套接字不是普通文件描述符,使用前必须调用 WSStartup 初始化 Winsock,关闭用 closesocket 而不是 close,链接需要 ws2_32 库;Linux/macOS 直接可用。工程上通常用一层薄的平台封装把差异吸收掉(本章代码以 POSIX 为准),或者干脆用跨平台网络库,见 §8.5。

C 的写法	C++ 的写法	说明
int fd = socket(...); 手工 close	SocketGuard guard(fd);	C 每路径记得关闭,C++ 析构保证关闭
if (fd < 0) { perror(...); return; }	fail("socket") 抛异常	错误信息统一携带 strerror
结构体逐字段填 sockaddr_in	同样的字段,辅以 RAII 与检查	底层接口一致,外层体验不同

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>           // close
#include <arpa/inet.h>        // sockaddr_in / htons
#include <sys/socket.h>       // socket / bind / connect

class SocketGuard {          // RAII:构造持有,析构关闭
public:
    explicit SocketGuard(int fd) : fd(fd) {}
    ~SocketGuard() { if (fd >= 0) close(fd); }
    int get() const { return fd; }
    SocketGuard(const SocketGuard&) = delete;      // 不可复制
    SocketGuard& operator=(const SocketGuard&) = delete;
private:
    int fd;
};

void fail(const std::string& msg) {
    throw std::runtime_error(msg + ": " + strerror(errno));
}

int main() {
    try {
        int fd = socket(AF_INET, SOCK_STREAM, 0);
        if (fd < 0) fail("socket");
        SocketGuard guard(fd);                  // 之后任何异常,fd 都会被关闭
        std::cout << "套接字创建成功 fd=" << fd << "\n";
    } catch (const std::exception& e) {
        std::cerr << "错误: " << e.what() << "\n";
        return 1;
    }
    return 0;
}

```

§8.3 TCP 服务端

服务端四步曲:socket 创建套接字,bind 绑定本机地址与端口,listen 进入监听状态,accept 接受连接。accept 是阻塞调用:没有客户端连接时它一直等,来一个连接返回一个新的套接字——原来的监听套接字继续等下一个。sockaddr_in 是地址结构体,填法有讲究:sin_family 固定 AF_INET,sin_addr.s_addr 用 htonl(INADDR_ANY) 表示"监听本机所有网卡",sin_port 必须用 htons 把端口号转成网络字节序——这些转换函数的来历见 §8.6。

单线程服务端一次只能服务一个客户端,处理完才 accept 下一个。要并发服务多个客户端,经典路线是每连接一个线程(第9章),事件驱动路线是 select/poll/epoll。别急着上 epoll:先写对单线程与多线程版本,理解阻塞模型,再考虑高并发。工程上成熟的选择是直接使用网络库(§8.5),套接字的细枝末节交给库去处理。

```

#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

void echoServer(int port) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);    // 1 创建套接字
    if (fd < 0) throw std::runtime_error("socket 失败");
    SocketGuard guard(fd);

    sockaddr_in addr;                            // 本机地址
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);    // 监听所有网卡
    addr.sin_port = htons((uint16_t)port);      // 端口转网络字节序

    if (bind(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {    // 2 绑定
        throw std::runtime_error("bind 失败");
    }
    if (listen(fd, 8) < 0) {                            // 3 监听,排队上限 8
        throw std::runtime_error("listen 失败");
    }

    for (int round = 0; round < 3; ++round) {    // 服务 3 个客户端
        int cfd = accept(fd, nullptr, nullptr);    // 4 阻塞等连接
        if (cfd < 0) throw std::runtime_error("accept 失败");
        SocketGuard cguard(cfd);                  // 连接套接字同样 RAII
        std::cout << "已接入客户端 " << round + 1 << "\n";
        close(cfd);                                // 演示用:立即断开
    }
}

int main() {
    try {
        echoServer(8899);
    } catch (const std::exception& e) {
        std::cerr << "服务端错误: " << e.what() << "\n";
    }
    return 0;
}

```

§8.4 TCP 客户端与收发循环

客户端更简单:socket 创建套接字,connect 连接服务端,然后收发。地址结构体用 inet_pton 把 "127.0.0.1" 这类点分十进制字符串转成二进制地址,127.0.0.1 是回环地址,指本机自己,最适合本地调试。connect 是阻塞调用,服务端不在线会一直等,所以要在 connect 之前设置超时,或者用非阻塞模式加 select——工程上先学会设置超时这一招。

收发循环是本节的灵魂。TCP 是字节流,recv 一次返回的字节数"恰好等于你要的"是巧合,少收、多收都正常:send 也可能只发出去一部分。因此发送要写"发完为止"的循环,sendAll 函数把剩余字节反复 send;接收同样要"收满为止",这就是 recvMsg 循环的来历。粘包的概念也源于此:两次 send 的内容可能被一次 recv 收到,或者一次 send 的内容被两次

recv 拆开。解法是给每条消息加定长头部(如 4 字节长度前缀),接收方先收满头部,再按长度收满内容——这是应用层协议的通用设计,HTTP、WebSocket 都是这个思路。

C 的写法	C++ 的写法	说明
recv 一次,假设收满	recvMsg 循环收满	TCP 流语义,必须循环收发
消息边界靠约定与运气	4 字节长度前缀协议	定长头 + 内容,根治粘包
阻塞 connect 可能永远等待	SO_RCVTIMEO / SO_SNDTIMEO	超时兜底,避免挂死

清单 8-3 客户端:connect 与收发循环

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

// 发送全部字节:send 可能只发出部分,循环补齐
bool sendAll(int fd, const std::string& data) {
    size_t off = 0;
    while (off < data.size()) {
        ssize_t n = send(fd, data.data() + off, data.size() - off, 0);
        if (n <= 0) return false;
        off += (size_t)n;
    }
    return true;
}

// 接收完整消息:先收 4 字节长度,再收内容(与发送方协议一致)
std::string recvMsg(int fd) {
    uint32_t len = 0;
    size_t off = 0;
    while (off < 4) { // 收满长度头
        ssize_t n = recv(fd, (char*)&len + off, 4 - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    len = ntohl(len); // 还原主机字节序
    if (len > (1 << 20)) return ""; // 防御:拒绝超大消息
    std::string buf(len, '\0');
    off = 0;
    while (off < len) { // 收满内容
        ssize_t n = recv(fd, &buf[off], len - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    return buf;
}

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return 1;
    SocketGuard guard(fd);

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8899);
    if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) return 1;

    if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        std::cerr << "连接失败,服务端是否启动?\n";
        return 1;
    }
    if (!sendAll(fd, "你好,网络")) return 1;
}

```

```
std::string resp = recvMsg(fd);
std::cout << "回应: " << resp << "\n";
return 0;
}
```

⚠ 易错点 1:假设 recv 一次收完

`int n = recv(fd, buf, 100, 0);` 之后直接处理 `buf`,这是经典错误:TCP 是字节流,这次可能只到了 30 字节。一切接收都要"循环收满期望长度",一切发送都要"循环发完",协议设计则保证接收方知道期望长度。

§8.5 HTTP 与网络库

HTTP 是应用层最常用的协议,它本身也是"文本行 + 空行 + 正文"的结构:请求行(GET /index.html HTTP/1.1)、若干头部行、空行、可选的请求体。手写 HTTP 客户端是理解协议的最好练习:用 `connect` 连上 80 端口,把请求文本原样发过去,再用 `recv` 循环把响应收回来。注意服务端返回后不会主动断开,必须靠 `Content-Length` 或读取到连接关闭来判断响应结束。手写的版本能做实验、能学原理,但生产环境请用库:`curl` 命令行与 `libcurl`、`cpp-httpplib`(单头文件,适合小型服务)、`Boost.Asio`(底层与异步)、`POCO`。选型口诀:个人工具与教学用 `cpp-httpplib`,企业异步高并发用 `Boost.Asio` 或 `Asio` 系库,HTTP 客户端直接调 `libcurl`。

无论用哪个库,HTTP 的基础语义都要烂熟:状态码(200 成功、404 未找到、500 服务端错误)、请求方法(GET 取数据、POST 提交数据)、头部字段(`Content-Length` 正文长度、`Content-Type` 内容类型、`Connection` 连接策略)。调试网络程序的第一工具是抓包与命令行:`curl -v` 看完整交互,`netcat` 模拟任意 TCP 对话,它们在 C 时代就在,C++ 时代依然是排查协议问题的利器。

C 的写法	C++ 的写法	说明
手写 socket + 拼请求文本	<code>cpp-httpplib</code> / <code>libcurl</code> 一行调用	库封装细节,专注业务
解析响应靠手工字符串处理	库提供响应对象与状态码	C++ 生态成熟,错误处理完备
每个平台重写一遍	库跨平台	Windows/Linux/macOS 行为一致

清单 8-4 手写 HTTP GET 客户端

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

// 请求:连 80 端口,发 HTTP 请求文本,收响应
std::string httpGet(const std::string& host, const std::string& path) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return "";
    SocketGuard guard(fd);

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(80); // HTTP 默认端口
    if (inet_pton(AF_INET, host.c_str(), &addr.sin_addr) <= 0) return "";
    if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) return "";

    std::string req = "GET " + path + " HTTP/1.1\r\n"
        + "Host: " + host + "\r\n"
        + "Connection: close\r\n\r\n"; // 收完自动断开
    if (!sendAll(fd, req)) return "";

    std::string resp;
    char buf[4096];
    for (;;) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n <= 0) break; // 读到连接关闭
        resp.append(buf, (size_t)n);
    }
    return resp;
}

int main() {
    std::string r = httpGet("127.0.0.1", "/index.html");
    if (!r.empty()) std::cout << "响应头前 200 字符:\n" << r.substr(0, 200);
    return 0;
}

```

§8.6 字节序、超时与安全

字节序问题:C 程序员都知道 x86 是小端,网络协议统一用大端(网络字节序)。htonl/htons 把主机整数转成网络字节序,ntohl/ntohs 反向转换。规矩只有一条:凡是写入网络的多字节整数,一律先 hton;凡是从网络读到的多字节整数,一律先 ntoh——忘一次,跨机器通信立刻乱码。这正是 §8.4 长度前缀要 htonl、接收端要 ntohl 的原因。

超时与安全是网络程序的生存线:所有阻塞调用都设置超时(setsockopt 的 SO_RCVTIMEO/SO_SNDTIMEO,或者 select/poll 的等待时间),避免连接挂死;从网络读到的每一条数据都当作不可信输入:长度前缀要上限检查(防超大分配),字符串要检查终止符,数值要检查范围——网络对端可能是任何人的代码,缓冲区溢出与整数溢出的攻击入口都在这里。C 的缓冲区管理(固定数组、手工边界)在网络上几乎必出漏洞,C++ 的 string/vector 自动管理长度,配合防御性检查,能把一类经典漏洞挡在门外。

```
#include <iostream>
#include <arpa/inet.h>

int main() {
    uint16_t port = 8899;
    uint16_t net = htons(port);          // 主机转网络:大端
    uint16_t back = ntohs(net);          // 网络转主机:还原
    std::cout << port << " -> " << net
               << " -> " << back << "\n";    // 8899 -> ... -> 8899
    // 教训:发送方 htons,接收方 ntohs,成对使用
    return 0;
}
```

⚠ 易错点 2:字节序漏转换

sin_port 直接填 8899、长度前缀直接写整数、读回长度不 ntohs——每一条都是跨机器必炸的雷。检查口诀:见到"写入网络的多字节整数"就找 htons,见到"从网络读回"就找 ntohs,一个都不能少。

⚠ 易错点 3:阻塞调用不设超时

accept、connect、recv、send 默认都是阻塞调用,对端宕机、网络断开时可能无限等待,整个线程挂死。所有可能长时间阻塞的调用都要配超时:setsockopt 的 SO_RCVTIMEO/SO_SNDTIMEO 是最简方案,select/poll 给等待时间更灵活。上线前的压测要专门模拟对端消失,确认超时逻辑真的生效。

网络编程速查表

概念	语法示例	要点
创建套接字	socket(AF_INET, SOCK_STREAM, 0)	SOCK_STREAM 是 TCP,SOCK_DGRAM 是 UDP
绑定端口	bind(fd, (sockaddr*)&addr, len)	服务端必须,端口冲突返回错误
监听	listen(fd, 8)	第二个参数是排队上限
接受连接	accept(fd, nullptr, nullptr)	阻塞等待,返回新套接字
连接服务端	connect(fd, (sockaddr*)&addr, len)	客户端必须,失败检查返回值
发送	send / sendAll 循环	send 可能发部分,必须循环
接收	recv / recvMsg 循环	按协议长度收满,防粘包
长度前缀	htonl(len) + 内容	应用层协议的标准解法
地址转换	inet_pton(AF_INET, "1.2.3.4", &addr)	文本地址转二进制
字节序	htons / ntohs	网络字节序,进出都转换
回环地址	"127.0.0.1"	本机调试首选
超时	setsockopt(SO_RCVTIMEO)	防阻塞挂死,必设
HTTP 客户端	libcurl / cpp-httplib	生产环境用库,不手写
Windows 差异	WSAStartup + closesocket	需 ws2_32 库,平台封装吸收

自测题

- Q 1. TCP 与 UDP 的区别是什么?各自的典型场景?
- Q 2. 服务端的完整流程是什么?accept 返回的套接字与监听套接字有何不同?
- Q 3. 什么是粘包?长度前缀协议是如何解决它的?
- Q 4. 为什么 sin_port 和长度前缀都要做字节序转换?
- Q 5. 网络数据为什么必须当作不可信输入处理?举两个防御例子。

✓ 参考答案

1. TCP 面向连接、可靠、按序字节流,适合 HTTP、文件传输、聊天;UDP 无连接、不保证到达与顺序,延迟低,适合音视频、游戏、DNS。需要完整性选 TCP,追求低延迟可容忍丢包选 UDP。
2. socket 创建、bind 绑定、listen 监听、accept 循环接客。监听套接字只负责等待与接受,accept 返回的连接套接字才真正承载收发;监听套接字继续等下一个连接。
3. TCP 是字节流,没有消息边界,多次发送的内容可能合并到达,一次发送的内容也可能被拆分。长度前缀协议在每条消息前写 4 字节长度,接收方先收满长度再收满内容,边界精确还原。
4. 网络协议统一使用大端(网络字节序),而 x86 等主机是小端。不转换,整数在另一台机器上解读出的值就是错的;所有进出网络的多字节整数都必须显式转换。
5. 对端可能是恶意代码或损坏的客户端。防御例子:长度前缀加上限检查防止超大内存分配;字符串字段检查终止符与长度防止溢出;数值字段检查范围防止整数溢出,一切解析都加异常处理。

实验:回声服务器与客户端

本实验实现一个完整的"长度前缀回声"系统:服务端接受客户端连接,收到消息原样返回并带上前缀;客户端发送消息并打印回应。代码分三部分,覆盖点已在注释中标明。先在终端 A 运行服务端,再在终端 B 运行客户端。

实验 8•第一部分:工具层(覆盖:RAII 封装、错误转异常、sendAll 循环、长度前缀)

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <cstdint>

class SocketGuard {                                // 覆盖:RAII,任何路径都关闭 fd
public:
    explicit SocketGuard(int fd) : fd(fd) {}
    ~SocketGuard() { if (fd >= 0) close(fd); }
    int get() const { return fd; }
    SocketGuard(const SocketGuard&) = delete;
    SocketGuard& operator=(const SocketGuard&) = delete;
private:
    int fd;
};

void fail(const std::string& msg) { // 覆盖:错误统一转异常
    throw std::runtime_error(msg + ": " + strerror(errno));
}

// 覆盖:sendAll—send 可能发部分,循环到发完
bool sendAll(int fd, const std::string& data) {
    uint32_t len = htonl((uint32_t)data.size()); // 覆盖:字节序转换
    size_t off = 0;
    while (off < 4) {
        ssize_t n = send(fd, (const char*)&len + off, 4 - off, 0);
        if (n <= 0) return false;
        off += (size_t)n;
    }
    off = 0;
    while (off < data.size()) {
        ssize_t n = send(fd, data.data() + off, data.size() - off, 0);
        if (n <= 0) return false;
        off += (size_t)n;
    }
    return true;
}

```

实验 8•第二部分:服务端(覆盖:socket/bind/listen/accept、recv 循环、阻塞模型)

```

// 覆盖:recvMsg—先收 4 字节长度头,再按长度收内容
std::string recvMsg(int fd) {
    uint32_t len = 0;
    size_t off = 0;
    while (off < 4) {
        ssize_t n = recv(fd, (char*)&len + off, 4 - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    len = ntohl(len); // 覆盖:网络字节序还原
    if (len > (1 << 20)) return ""; // 覆盖:防御超大消息
    std::string buf(len, '\0');
    off = 0;
    while (off < len) {
        ssize_t n = recv(fd, &buf[off], len - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    return buf;
}

// 覆盖:服务端四步曲—bind 绑定、listen 监听、accept 循环
void runEchoServer(int port) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) fail("socket");
    SocketGuard guard(fd);

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);
    if (bind(fd, (sockaddr*)&addr, sizeof(addr)) < 0) fail("bind");
    if (listen(fd, 8) < 0) fail("listen");
    std::cout << "回声服务已在 " << port << " 端口就绪\n";

    for (;;) {
        int cfd = accept(fd, nullptr, nullptr); // 覆盖:阻塞等待连接
        if (cfd < 0) fail("accept");
        SocketGuard cguard(cfd);
        std::string msg = recvMsg(cfd);
        if (!msg.empty()) {
            sendAll(cfd, "回声: " + msg); // 原样返回
            std::cout << "收到并回应: " << msg << "\n";
        }
    }
}

```

实验 8•第三部分:客户端与 main(覆盖:connect、超时设置、双端组合)

```

int main(int argc, char** argv) {
    if (argc > 1 && std::string(argv[1]) == "-s") {
        try {
            runEchoServer(8899);                // 服务端模式
        } catch (const std::exception& e) {
            std::cerr << "服务端错误: " << e.what() << "\n";
        }
        return 0;
    }

    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return 1;
    SocketGuard guard(fd);

    timeval tv{2, 0};                          // 覆盖:超时兜底 2 秒
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8899);
    if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) return 1;
    if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        std::cerr << "连接失败:先以 -s 启动服务端\n";
        return 1;
    }

    std::string req = "你好,网络";
    if (!sendAll(fd, req)) return 1;            // 覆盖:发送循环
    std::string resp = recvMsg(fd);            // 覆盖:接收循环
    std::cout << "服务端回应: " << resp << "\n";
    return 0;
}

```

实验覆盖点核对:RAII 封装(SocketGuard 双端使用)、错误转异常(fail 与 try/catch)、sendAll 发送循环、recvMsg 接收循环、长度前缀协议(htonl/ntohl 成对)、服务端四步曲(socket/bind/listen/accept)、阻塞模型(accept 等待)、客户端 connect、超时设置(SO_RCVTIMEO)、回环地址(127.0.0.1)。扩展练习:把回声改成"统计每个单词出现次数"的协议;或启动两个客户端观察单线程服务端的串行行为,再引入线程(第9章)实现并发。

工程建议 1:协议先行,再写代码

任何网络程序先写协议文档:消息格式、长度前缀、字节序、错误码、超时值。两端按文档实现,联调时间从"猜对端行为"缩短为"核对文档"。协议里还要写清最大消息长度,这是所有防御性检查的依据。

工程建议 2:生产环境用网络库

手写套接字适合学习与教学,生产环境优先 cpp-httpplib、Boost.Asio 或 libcurl:线程模型、超时、TLS 加密、HTTP/2 都是成熟实现,自己造的轮子在安全与性能上很难追平。TLS(https)尤其不要手写,交给 OpenSSL 与库封装。

本章小结

网络编程的地基:TCP 可靠字节流与 UDP 无连接数据报;套接字四步曲与客户端两步走;RAII 封装与异常处理让资源安全;长度前缀协议解决粘包;字节序与超时是跨机器通信的纪律;网络数据一律不可信。下一章把服务端真正用起来:并发与异常处理。

🎯 本章目标

掌握 C++ 的异常机制与栈展开,理解异常安全与 noexcept;学会用 std::thread 创建线程、用 mutex 与 atomic 保护共享数据、用条件变量与 async 组织并发;识别数据竞争、死锁等并发陷阱,把第8章的单线程服务端升级为多线程版本。

📦 前置知识

熟悉 RAII(第5章)与 lambda(第4章);了解函数声明与变量作用域(第1、4章)。本章代码基于 C++17,编译多线程代码需要链接线程库(pthread 或 Windows 线程库)。

§9.1 异常机制与栈展开	§9.2 异常安全与 noexcept	§9.3 std::thread 线程
§9.4 互斥与原子	§9.5 条件变量与 async	§9.6 并发设计原则

§9.1 异常机制与栈展开

C 的错误处理是返回值加 errno:每个函数都要检查返回值,漏一次检查错误就静默扩散;setjmp/longjmp 能跳转,但跳过中间函数的清理代码,资源泄漏。C++ 的异常把"错误路径"从正常路径中分离:throw 抛出异常对象,异常沿着调用栈向上传播,直到匹配的 catch 捕获;传播过程中,栈上所有对象依次析构,这就是栈展开。展开的每一步都调用析构函数,RAII 类(第5章)因此保证"异常路径上资源同样被释放"——这是 C 的 longjmp 做不到的。

写 catch 的三条纪律:第一,按引用捕获 catch (const std::exception& e),按值捕获会把派生异常切片成基类,丢失信息;第二,捕获顺序从具体到宽泛,std::bad_alloc 要在 std::exception 之前;第三,不想处理的异常就让它继续传播,或者用 catch(...) 兜底,记录日志后再重新抛出。真正不知道怎么办时,让程序按默认规则终止(std::terminate),好过带着半损坏状态继续运行。

C 的写法	C++ 的写法	说明
if (ret < 0) { 处理错误 }	try { ... } catch (const X& e)	C 检查点散落,C++ 错误路径集中
errno 全局变量,易被覆盖	异常对象携带完整信息	C++ 异常类型可含上下文
longjmp 跳过清理代码	栈展开自动析构	C++ 与 RAII 配合,资源不泄漏

清单 9-1 异常基础与栈展开

```

#include <iostream>
#include <stdexcept>
#include <string>

class Guard {                                // RAII:观察析构时机
public:
    explicit Guard(const std::string& n) : name(n) {}
    ~Guard() { std::cout << "析构 " << name << "\n"; }
private:
    std::string name;
};

double divide(double a, double b) {
    Guard g("divide 作用域");
    if (b == 0.0) throw std::runtime_error("除数不能为零");
    return a / b;
}

int main() {
    try {
        Guard g("main 作用域");
        std::cout << divide(10, 0) << "\n";
    } catch (const std::runtime_error& e) {    // 按引用捕获
        std::cout << "捕获:" << e.what() << "\n";
    }
    return 0;    // 输出:析构 divide 作用域 → 捕获 → 析构 main 作用域
}

```

⚠ 易错点 1:按值捕获导致切片

`catch (std::exception e)` 会先把派生异常拷贝成基类再捕获,子类携带的额外信息全部丢失,虚函数调用也退化成基类版本。一律写 `catch (const std::exception& e)`,既避免切片又避免拷贝开销。

§9.2 异常安全与 noexcept

异常安全的级别有三档:基本保证(出错后对象保持有效状态)、强保证(操作要么完全成功要么像没发生过)、无异常保证(绝不抛出)。提供强保证的经典手段是"先构造副本,再一次性交换":把会抛异常的操作全部放在临时对象上,最后用不抛异常的 `swap` 提交——即使中途抛异常,原对象纹丝不动。RAII 是异常安全的地基:锁、内存、文件都在析构中释放,展开路径上不会泄漏。

`noexcept` 声明"这个函数不抛异常":函数内部若真的抛出,程序直接调用 `std::terminate` 终止。它既是承诺也是优化——编译器知道无需准备展开路径。移动构造与移动赋值必须声明 `noexcept`,否则 `vector` 扩容时保守地退回拷贝,性能退化;`swap`、析构函数、默认构造函数等"应当成功"的操作也习惯性标记 `noexcept`。判断口诀:不涉及资源获取、不调用可能抛出函数、失败也无妨的操作,都可以 `noexcept`。

清单 9-2 强异常保证与 noexcept

```

#include <iostream>
#include <vector>
#include <string>
#include <utility>

class Config {
public:
    void set(const std::string& k, const std::string& v) {
        std::vector<std::pair<std::string, std::string>> tmp = kv;
        for (auto& p : tmp) {
            if (p.first == k) { p.second = v; goto done; }
        }
        tmp.emplace_back(k, v);          // 可能抛异常:发生在临时对象上
    done:
        kv.swap(tmp);                    // 不抛异常的提交:强保证
    }
    std::string get(const std::string& k) const {
        for (const auto& p : kv) if (p.first == k) return p.second;
        return "";
    }
private:
    std::vector<std::pair<std::string, std::string>> kv;
};

class Token {                          // 移动必须 noexcept 的范例
public:
    explicit Token(std::string v) : v(std::move(v)) {}
    Token(Token&& o) noexcept : v(std::move(o.v)) {}
    Token& operator=(Token&& o) noexcept {
        v = std::move(o.v);
        return *this;
    }
    const std::string& value() const { return v; }
private:
    std::string v;
};

int main() {
    Config c;
    c.set("name", "张三");
    c.set("age", "18");
    std::cout << c.get("name") << " " << c.get("age") << "\n";
    std::vector<Token> v;
    v.push_back(Token("甲"));          // 扩容时用移动,无拷贝
    return 0;
}

```

§9.3 std::thread 线程

C 的并发工具是 pthread 一族:创建、传参、返回值全凭手工约定。C++11 的 std::thread 把线程包装成对象:构造函数直接接受可调用对象(函数、lambda、函数对象),参数跟着传;join() 等待线程结束,detach() 让线程独立运行。生命周期纪律:thread 对象析构时若线程还在运行且未 join/detach,程序直接终止——要么 join,要么 detach,没有第三种选择。

传参的两个细节:线程函数的引用参数必须用 `std::ref` 显式包装,否则按值拷贝一份,修改不生效;lambda 捕获与线程一起使用时,捕获的引用指向的变量必须活得比线程长——`detach` 之后尤其危险,线程还在跑,局部变量已经销毁,访问就是未定义行为。第8章的服务端要并发,最小改动就是每接一个连接,开一个线程去处理,处理完 `join` 或 `detached` 线程自己退出。

C 的写法	C++ 的写法	说明
<code>pthread_create(&t, ...)</code>	<code>std::thread t(f, args)</code>	C 填结构体传参,C++ 直接给参数
<code>pthread_join / pthread_detach</code>	<code>t.join() / t.detach()</code>	同语义,成员函数更直观
返回结果靠全局变量	<code>std::async + future</code> 取回	C++ 结果随对象返回

清单 9-3 线程的创建与生命周期

```
#include <iostream>
#include <thread>
#include <vector>

void work(int id, int& out) {
    out = id * id;           // 通过引用把结果写出去
}

int main() {
    std::vector<std::thread> pool;
    std::vector<int> results(4, 0);

    for (int i = 0; i < 4; ++i) {
        pool.emplace_back(work, i, std::ref(results[i])); // 引用要 std::ref
    }
    for (auto& t : pool) t.join();           // 逐个等待,不 join 就析构会终止程序

    for (int x : results) std::cout << x << " ";    // 0 1 4 9
    std::cout << "\n";

    // lambda 线程 + detach 的危险示范:注释掉看程序行为
    // int local = 42;
    // std::thread([&local] { std::cout << local << "\n"; }).detach();
    return 0;
}
```

⚠ 易错点 2:detach 后访问已销毁的局部变量

`std::thread([&x]{ use(x); }).detach();` 线程独立运行,函数返回后 `x` 已销毁,线程访问 `x` 是未定义行为——可能碰巧能跑,也可能随机崩溃。规则:detach 的线程只能访问它自己的拷贝与全局/堆上生命周期明确的数据,引用局部变量的线程必须 `join`。

§9.4 互斥与原子

多个线程同时读写同一份数据就是数据竞争,数据竞争是未定义行为——这不是“可能出错”,而是程序任何行为都可能。保护数据的手段有二:互斥锁与原子操作。`std::mutex` 提供 `lock/unlock`,但手工配对和 C 的 `fopen/fclose` 一样容易漏;正确用法是 RAII 包装:`std::lock_guard` 构造时加锁、析构时解锁,异常路径也安全;`std::unique_lock` 额外支持手动解锁与延迟加锁,是条件变量(下节)的搭档。工程准则:锁的粒度越小越好,持锁时间越短越好,绝不在持锁时做 I/O 或等待。

std::atomic 提供无锁的原子操作:对一个 atomic 整数的读写是线程安全的,底层是 CPU 的原子指令,比加锁轻量得多,适合计数器、标志位、共享的简单状态。注意"无锁"不等于"免同步":两个线程先读后写同一个 atomic,仍然需要顺序语义(compare_exchange 等),真正的复杂并发还是交给锁。死锁的典型成因是锁顺序不一致:线程 A 持有锁1等锁2,线程 B 持有锁2等锁1,互相等待永不释放。预防口诀:全项目统一加锁顺序,或者用 std::scoped_lock 一次锁多把。

C 的写法	C++ 的写法	说明
pthread_mutex_lock/unlock 成对	std::lock_guard 自动解锁	RAII 保证异常路径也解锁
__sync_fetch_and_add 或汇编	std::atomic<int> cnt	可移植的原子操作
锁顺序靠自觉	scoped_lock 一次锁多把	工具辅助,防死锁于设计

清单 9-4 互斥锁与原子计数器

```
#include <iostream>
#include <thread>
#include <mutex>
#include <atomic>
#include <vector>

std::mutex mtx;           // 保护 balance
double balance = 0.0;     // 被锁保护的共享数据
std::atomic<int> ops{0};  // 无锁计数器

void deposit(double amount, int times) {
    for (int i = 0; i < times; ++i) {
        {
            std::lock_guard<std::mutex> lock(mtx);    // RAII 锁
            balance += amount;                        // 持锁改数据
        }                                             // 解锁
        ++ops;                                       // 原子自增
    }
}

int main() {
    std::vector<std::thread> ts;
    ts.emplace_back(deposit, 1.0, 10000);
    ts.emplace_back(deposit, 2.0, 10000);
    for (auto& t : ts) t.join();
    std::cout << "余额 " << balance << ",操作数 " << ops
                << "\n";    // 30000 与 20000,顺序无关
    return 0;
}
```

§9.5 条件变量与 async

条件变量解决"等待某个条件成立":生产者准备好数据后 notify_one/notify_all 唤醒等待者,消费者在 wait 上挂起等待。使用套路固定:wait 必须配合 unique_lock,且用带谓词的版本 wait(lk, pred)——它循环检查谓词,自动处理"虚假唤醒"(条件变量可能被莫名唤醒,谓词循环是唯一可靠写法)。没有谓词的裸 wait 是并发 bug 的温床。

大多数并发需求其实用不着直接碰线程与条件变量:std::async 接受一个任务,返回 std::future,调用 future.get() 取结果并等待完成。async 决定在哪个线程执行交给实现,任务返回值直接送到 future,异常也会随 future 传递——get() 会把

任务里的异常重新抛出。这个“任务 + 未来结果”的模型比手写线程加共享变量安全得多,是“并发优先方案”。future 家族的底层是 promise: promise 承诺将来提供值, set_value 兑现, set_exception 报告错误, 适合跨线程传值。

清单 9-5 条件变量与 async

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <future>
#include <chrono>

std::mutex m;
std::condition_variable cv;
bool ready = false;           // 谓词:数据就绪

void producer() {
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    {
        std::lock_guard<std::mutex> lk(m);
        ready = true;         // 修改谓词必须持锁
    }
    cv.notify_one();          // 唤醒一个等待者
}

double slowAdd(double a, double b) {
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
    return a + b;
}

int main() {
    // 条件变量:等待就绪
    std::thread t(producer);
    {
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk, [] { return ready; }); // 带谓词,处理虚假唤醒
        std::cout << "数据已就绪\n";
    }
    t.join();

    // async:任务 + 未来结果
    std::future<double> f1 = std::async(std::launch::async, slowAdd, 1, 2);
    std::future<double> f2 = std::async(std::launch::async, slowAdd, 3, 4);
    std::cout << "和:" << f1.get() + f2.get() << "\n"; // 3 与 7 并行计算
    return 0;
}
```

⚠ 易错点 3:持锁时等待,死锁的温床

线程 A 持锁等条件变量,线程 B 需要同一把锁才能让条件成立——两边互相等,永远等不到。规则:等待条件、做 I/O、调用其他需要锁的代码,都不能发生在持锁期间;实在要等,先把锁释放(这正是 unique_lock 存在的意义)。

§9.6 并发设计原则

并发代码的正确性靠设计,不靠调试:数据竞争是未定义行为,能复现的竞争是运气,复现不了的竞争才是常态。四条原则:第一,最小共享——能不共享就不共享,每个线程只碰自己的数据,结果最后合并;第二,共享必保护——共享数据必须与锁或原子成对出现,注释里写明"由 mt_x 保护";第三,锁内不做等待——持锁时绝不调用可能阻塞的 I/O 或再等条件变量,否则死锁概率指数上升;第四,用更高级的工具——`async` 与 `future` 能解决八成的"算完取结果",线程加互斥留给真正的长期运行任务。

性能上还要破除两个误解:线程不是越多越好——创建销毁有开销,线程数接近 CPU 核心数时收益封顶,再多就是切换成本;加锁也有开销,热路径上的高频小操作优先用 `atomic`,实在要用锁,保持临界区极短。第8章的单线程服务端升级并发的标准路线:`accept` 循环不变,每接一个连接,把处理扔给线程池(见实验),连接多时再用事件驱动模型。

并发与异常速查表

概念	语法示例	要点
抛出异常	<code>throw std::runtime_error("msg");</code>	沿调用栈传播,栈展开逐层析构
捕获异常	<code>catch (const std::exception& e)</code>	必须按引用,防切片
不抛异常	<code>void f() noexcept</code>	内部抛出直接 <code>terminate</code>
强异常保证	副本修改 + <code>swap</code> 提交	失败时对象保持原状
创建线程	<code>std::thread t(f, arg1, arg2)</code>	参数直接传,引用需 <code>std::ref</code>
等待线程	<code>t.join()</code>	析构前必须 <code>join</code> 或 <code>detach</code>
独立线程	<code>t.detach()</code>	只能访问自持数据
互斥锁	<code>std::lock_guard<std::mutex> lk(m)</code>	RAII,异常路径也解锁
手动锁	<code>std::unique_lock<std::mutex></code>	条件变量的搭档
原子操作	<code>std::atomic<int> cnt</code>	无锁,适合计数器标志位
条件变量	<code>cv.wait(lk, pred) / notify_one</code>	必须带谓词,防虚假唤醒
异步任务	<code>std::async(f, args)</code>	返回 <code>future</code> , <code>get()</code> 取结果
未来结果	<code>std::future<T></code>	结果与异常随 <code>future</code> 传递
数据竞争	多线程同读写无保护	未定义行为,靠设计避免

自测题

- Q 1. 栈展开是什么?RAII 为什么与异常是天生一对?
- Q 2. 按值捕获异常为什么是错的?catch 的顺序有什么讲究?
- Q 3. `thread` 析构时线程还在运行会怎样?join 与 detach 各自适合什么场景?
- Q 4. 数据竞争为什么是未定义行为?mutex 与 atomic 怎么选?
- Q 5. 条件变量的 wait 为什么要带谓词?async 相比手写线程有什么优势?

✅ 参考答案

1. 异常抛出后,调用栈上每层函数的作用域对象依次析构,再逐层向外找匹配的 catch,这个过程叫栈展开。RAII 对象在展开中必然析构,资源自动释放,所以异常路径与正常路径同样安全,C 的 longjmp 没有这个机制。
2. 按值捕获先把派生异常切片成基类,类型信息与虚函数行为丢失;按引用捕获零拷贝且保留完整类型。捕获顺序从具体到宽泛:先捕获派生异常,最后才是 std::exception 与 catch(...),否则派生异常永远轮不到。
3. 线程对象析构时若线程仍可 join,程序直接 std::terminate。join 适合"等它干完"的场景(任务分发、汇总结果);detach 适合"后台自生自灭"的场景,但线程必须只访问自己的拷贝或生命周期明确的数据。
4. 数据竞争意味着程序行为在语言层面失去定义,任何结果都算"正确",没法调试。选择:m 变量:原子操作适合计数器、标志位等简单状态,开销小;复杂数据结构的修改必须用互斥锁保护。
5. 条件变量可能被虚假唤醒,裸 wait 返回不代表条件成立;带谓词的 wait 会循环检查,条件不满足就继续等。async 把"启动任务、取回结果、传递异常"全部封装,调用方不用管理线程生命周期与共享数据,出错面小得多。

实验:多线程账本系统

本实验把本章知识组装成并发记账系统:多个线程同时存款,互斥锁保护余额,原子计数统计操作笔数,条件变量通知全部完成,async 并行计算汇总,异常在任务中抛出后由 future 传递。代码分三部分,覆盖点已在注释中标明。

实验 9•第一部分:共享账本(覆盖:mutex、lock_guard、atomic、数据竞争防护)

```

#include <iostream>
#include <thread>
#include <mutex>
#include <atomic>
#include <vector>
#include <condition_variable>
#include <future>
#include <chrono>

class Ledger {                                // 覆盖:共享数据与锁成对出现
public:
    void deposit(double amount) {
        std::lock_guard<std::mutex> lk(mtx);    // 覆盖:RAII 锁
        balance += amount;                    // 临界区极短
    }
    double total() const {
        std::lock_guard<std::mutex> lk(mtx);
        return balance;
    }
private:
    mutable std::mutex mtx;                    // 保护 balance
    double balance = 0.0;
};

Ledger ledger;
std::atomic<int> opCount{0};                  // 覆盖:原子计数器,无锁
std::mutex doneMtx;
std::condition_variable doneCv;
bool allDone = false;                        // 谓词:全部线程完成

void worker(int id, int times) {              // 覆盖:线程函数 + 循环存款
    try {
        for (int i = 0; i < times; ++i) {
            ledger.deposit(1.0);
            ++opCount;
            if (i == times / 2 && id == 2) {
                throw std::runtime_error("模拟中途故障");    // 覆盖:异常
            }
        }
    } catch (const std::exception& e) {
        std::cerr << "线程 " << id << " 故障: " << e.what() << "\n";
    }
    {
        std::lock_guard<std::mutex> lk(doneMtx);
        if (--remaining == 0) allDone = true;    // 覆盖:谓词更新持锁
    }
    doneCv.notify_all();                        // 覆盖:通知等待者
}

```

实验 9•第二部分:async 并行任务(覆盖:std::async、future、get 取结果、异常随 future 传递)

```
// 覆盖:async 任务—每个区块的账目各自算,结果并行取回
double auditBlock(int id, int blocks) {
    double sum = 0.0;
    for (int i = 0; i < 10; ++i) {
        sum += ledger.total() * 0.001;    // 只读汇总,不持锁写
    }
    if (id == blocks - 1) {
        throw std::runtime_error("审计故障");    // 覆盖:任务内抛异常
    }
    return sum;
}

std::future<double> auditAll(int blocks) {
    std::vector<std::future<double>> fs;
    for (int i = 0; i < blocks; ++i) {
        fs.push_back(std::async(std::launch::async, auditBlock, i, blocks));
    }
    double total = 0.0;
    for (auto& f : fs) {
        try {
            total += f.get();    // 覆盖:get 等待并取结果
        } catch (const std::exception& e) {
            std::cout << "审计任务异常: " << e.what() << "\n";
        }
    }
    return std::async(std::launch::async, [total] { return total; });
}
```

实验 9•第三部分:main 调度(覆盖:join、条件变量等待、谓词、整体组合)

```
int main() {
    std::vector<std::thread> workers;    // 覆盖:线程容器
    for (int i = 0; i < 3; ++i) {
        workers.emplace_back(worker, i, 2000);    // 3 个线程各存 2000 次
    }
    {
        std::unique_lock<std::mutex> lk(doneMtx);
        doneCv.wait(lk, [] { return allDone; });    // 覆盖:带谓词的等待
    }
    for (auto& t : workers) t.join();    // 覆盖:join 等待线程结束

    std::cout << "存款完成,余额 " << ledger.total()
              << ",操作 " << opCount << " 笔\n";

    auto fut = auditAll(3);    // 覆盖:async 并行审计
    std::cout << "审计汇总 " << fut.get() << "\n";
    return 0;
}
```

实验覆盖点核对:mutex 与 lock_guard(Ledger::deposit 临界区)、atomic 计数器(opCount)、条件变量与谓词(等待全部线程完成)、thread 容器与 join、async/future/异常传递(auditAll 收集各任务结果,任务异常在 get 时重新抛出并被捕获)、lambda 任务(返回汇总的 async)。运行观察:余额恒为 6000(3 线程各 2000 次,故障线程只影响自身循环),操作数恒为 6000,审计异常被捕获后程序正常结束。扩展练习:把 remaining 改成原子变量对比行为;或给 worker 增加随机延迟,观察条件变量唤醒的时机。

工程建议 1:共享数据用类封装

把"数据 + 保护它的锁"装进同一个类(Ledger 模式),锁不出类,数据不出类,外部只能通过加锁的接口访问。这样"谁保护什么"一目了然,代码评审时不用满文件找裸共享变量。

工程建议 2:优先 `async`,线程交给框架

新写并发逻辑先问:能不能用 `std::async` 表达?任务返回结果、互相独立——`async` 一行搞定,还免费获得异常传递。只有长期运行的循环、事件循环、线程池等才值得亲手管理 `std::thread`;线程池直接采用成熟库(如 TBB),别从零造。

本章小结

异常机制把错误路径从正常路径分离,栈展开与 RAII 让资源释放成为语言保证;`noexcept` 与强异常保证是接口质量的一部分。并发三件套:`thread` 管理执行,`mutex/atomic` 保护数据,条件变量与 `async` 组织协作;数据竞争与死锁靠设计规避。下一章进入 C++ 最强大的部分:泛型与模板元编程。

🔗 本章目标

把模板理解成代码生成器:函数模板与类模板如何实例化,特化与偏特化如何定制行为,可变参数与折叠表达式如何展开;掌握 SFINAE、enable_if 与 C++20 概念(concepts);用 constexpr 在编译期完成计算。理解模板与 C 宏、Java 泛型的本质区别。

📦 前置知识

熟练使用 std::vector、std::string 等模板容器(第1章起一直在用);了解函数重载(第4章)。本章代码基于 C++17,concepts 部分按 C++20 语法展示。模板错误信息长是常态,先看第一个错误行。

\$10.1 模板是代码生成器	\$10.2 函数模板与推导	\$10.3 类模板与特化
\$10.4 可变参数模板	\$10.5 SFINAE 与 concepts	\$10.6 constexpr 元编程

\$10.1 模板是代码生成器

C 的泛型工具只有宏与 void*:宏是纯文本替换,没有类型检查,还有双重求值这类暗坑,void* 丢掉类型信息,一切类型错误推迟到运行时。C++ 的模板走完全不同的路线:模板是代码生成器——写一份 template,编译器为每个用到的类型生成一份独立代码。std::vector<int> 和 std::vector<double> 是两份不同的类,各自的成员函数都是独立的机器码,类型检查在生成时完成,运行期零抽象开销。

理解"生成"二字,模板的许多现象就都说得通了:为什么模板通常要整体放在头文件里——因为每个编译单元都需要看到模板定义才能生成代码,这与普通函数"声明放头文件、定义放源文件"的纪律不同;为什么模板实例化越多编译越慢——生成代码也是要花时间的,工程上要节制模板的使用面。与之对照,Java 的泛型是擦除的:编译后只有一份代码,类型参数被替换成 Object,运行期靠强制转换与检查,基本类型还得装箱;C++ 是"每个类型生成一份代码,类型信息保留到运行期之前",因此 std::vector<int> 与原生数组一样快,这是模板在 C++ 里全面取代宏与 void* 的根本原因。

C 的写法	C++ 的写法	说明
#define MAX(a,b) ((a)>(b)?(a):(b))	template<typename T> T maxOf(T a, T b)	宏是文本替换,C++ 类型安全
宏双重求值:MAX(i++, j++)	模板参数求值一次	宏的经典暗坑,模板没有
void* 传数据,丢失类型	每种类型生成独立代码	错误留在编译期,运行期零开销

清单 10-1 函数模板:第一个模板

```

#include <iostream>
#include <string>

// 模板:一份定义,为每个 T 生成一份代码
template <typename T>
T maxOf(T a, T b) {
    return a > b ? a : b;
}

int main() {
    int m1 = maxOf(3, 7);           // 推导 T = int
    double m2 = maxOf(2.5, 1.5);    // 推导 T = double
    std::string m3 = maxOf(std::string("甲"), std::string("乙"));
    std::cout << m1 << " " << m2 << " " << m3 << "\n";
    // 上面三行,编译器实际生成了三个不同版本的 maxOf
    return 0;
}

```

§10.2 模板参数与推导

模板参数不只是类型:它可以是类型(`typename T`)、非类型值(`int N`)与模板模板参数。非类型参数让模板拥有“编译期常量”能力:`std::array<int, 5>` 的 5 就是非类型参数,数组长度在编译期就固定并写进类型,越界检查不再需要运行时开销。调用时,函数模板从实参自动推导类型参数;推导不出(比如两个参数类型不同)就显式指定:`maxOf<double>(3, 2)`。推导遵循一套精确规则,最常见的原则是:按值传参时数组与函数会退化成指针,引用传参时保留完整类型——`std::forward` 的完美转发正是靠这条规则实现的。

推导失败或显式指定错误都会在实例化时报错,错误信息长而费解,这是模板的学费。工程应对:先从第一个错误行开始读(往往是“找不到匹配的调用”或“模板实参不匹配”),再看它引用的模板定义;复杂模板库(如标准库算法)的错误,先怀疑自己的类型不满足该算法要求的操作——`vector` 传给了需要排序的接口,通常就是缺少某个运算符。C++20 的 `concepts`(§10.5)用约束把这类错误提前到函数签名处,信息质量大幅改善。

清单 10-2 非类型参数与显式指定

```

#include <iostream>
#include <array>

// 非类型参数:N 是编译期常量
template <typename T, int N>
T sumOf(const std::array<T, N>& a) {
    T s{};
    for (int i = 0; i < N; ++i) s += a[i];
    return s;
}

int main() {
    std::array<int, 3> x{1, 2, 3};      // N = 3 写进类型
    std::array<double, 2> y{1.5, 2.5};
    std::cout << sumOf(x) << " " << sumOf(y) << "\n";    // 6 4
    std::cout << sumOf<int, 3>(x) << "\n";    // 显式指定参数
    return 0;
}

```

§10.3 类模板、特化与偏特化

类模板与函数模板同理:template<typename T> class Box { ... },成员函数在使用到的方法内按需实例化。写一个最简单的 Box 立刻就能体会:数据成员 T value,get 与 set 的类型随 T 变化,而成员函数只有被调用时才生成代码——没调用 set 的 Box 不会生成 set 的机器码。这让模板成为"按需定制"的机制,而不是一揽子复制。

特化是模板家族最强大的定制工具,也是 Java 泛型完全没有的能力:全特化 template<> class Box<bool> 为特定类型提供专属实现;偏特化 template<typename T> class Box<T*> 为一族类型(所有指针)提供实现。标准库大量使用特化:std::vector<bool> 用位压缩存储、std::hash 为每个类型定制哈希——编译器选择"最匹配"的特化版本,这和重载决议是同一思想,只是发生在类型层面。写特化时注意:特化不能出现在首次使用之后(一般紧跟主模板声明),这是 C++ 对"代码生成顺序"的硬性要求。

C 的写法	C++ 的写法	说明
为每种类型复制粘贴一份结构体	一份类模板 + 特化	特化按需定制,不复制整体
无对应能力	template<> class Box<bool>	全特化:单一类型专属实现
无对应能力	template<typename T> class Box<T*>	偏特化:一族类型的专属实现

清单 10-3 类模板与特化

```

#include <iostream>
#include <string>

template <typename T>
class Box {
public:
    explicit Box(T v) : value(v) {}
    T get() const { return value; }
    void set(T v) { value = v; }
    std::string describe() const { return "普通盒"; }
private:
    T value;
};

// 全特化:bool 版用 1 字节,输出专属描述
template <>
class Box<bool> {
public:
    explicit Box(bool v) : value(v) {}
    bool get() const { return value; }
    void set(bool v) { value = v; }
    std::string describe() const { return value ? "真" : "假"; }
private:
    bool value;
};

// 偏特化:指针版,get 返回解引用后的值
template <typename T>
class Box<T*> {
public:
    explicit Box(T* p) : ptr(p) {}
    T get() const { return *ptr; }
    void set(T v) { *ptr = v; }
    std::string describe() const { return "指针盒"; }
private:
    T* ptr;
};

int main() {
    Box<int> a(42);
    Box<bool> b(true);           // 命中全特化
    int raw = 7;
    Box<int*> c(&raw);           // 命中偏特化
    std::cout << a.describe() << " " << b.describe() << " "
              << c.describe() << "\n";
    return 0;
}

```

§10.4 可变参数模板与折叠表达式

可变参数模板让函数接受任意数量的任意类型参数:template<typename... Args>,参数包 Args... 在展开点逐个展开。打印任意多个值是经典入门:递归展开(取第一个参数,剩余继续递归)是 C++11 的写法;C++17 的折叠表达式把常见模

式压缩成一行:(args + ...) 求全部参数的和,折叠运算符可以是任何二元运算符。printf 式的格式化与 std::tuple、std::variant 的底层都是参数包。

折叠表达式有三种形态需要分清:一元右折叠 (args + ...) 等价 a + (b + c);一元左折叠 (... + args) 等价 (a + b) + c ——加法无所谓,减法等非结合运算结果不同;二元折叠 (0 + ... + args) 提供初始值。参数包还可以与 sizeof... 配合查询个数,与转发引用配合实现完美转发。学会把"任意多参数"的接口交给参数包,你的函数签名就从固定参数表解放出来。

清单 10-4 参数包与折叠表达式

```
#include <iostream>
#include <string>

// 递归展开:每次取第一个参数,剩余继续
template <typename T>
void printAll(const T& v) {                // 递归终点:只剩一个
    std::cout << v << "\n";
}

template <typename T, typename... Rest>
void printAll(const T& v, const Rest&... rest) {
    std::cout << v << " ";
    printAll(rest...);                    // 展开剩余参数
}

// C++17 折叠表达式:一行求所有参数之和
template <typename... Args>
auto sumAll(const Args&... args) {
    return (args + ...);                // 一元右折叠
}

int main() {
    printAll(1, 2.5, "三", std::string("四"));
    std::cout << "个数:" << sizeof...(int) << "\n";    // 语法演示
    std::cout << sumAll(1, 2, 3, 4) << " " << sumAll(1.5, 2.5) << "\n";
    return 0;
}
```

§10.5 SFINAE、enable_if 与 concepts

SFINAE(Substitution Failure Is Not An Error,替换失败不是错误)是模板世界的"条件编译":实例化时,某个模板替换出的声明不合法(如类型没有某种运算符),编译器不把它当错误,而是把这个候选从重载集合中移除,继续找别的匹配。基于它,C++11 的 std::enable_if 实现"条件启用":模板参数满足条件才存在,不满足就像没声明过一样。经典用法是给"只接受数值类型"的函数加约束:std::enable_if<std::is_arithmetic_v<T>>::type 作为返回类型或默认模板参数。

C++20 的 concepts 把 SFINAE 的繁琐压缩成自然语言:概念(concept)是一组约束的命名,如 std::integral、std::floating_point、std::ranges::range;requires 子句或 requires 表达式声明约束:template<typename T> requires std::integral<T> void f(T);。概念约束错误发生在函数签名处,报错信息直接说"不满足 integral",而不是一屏模板展开。工程建议:新代码优先 concepts,老代码维持 enable_if;两者背后是同一套替换机制。

C 的写法	C++ 的写法	说明
宏 #ifdef 条件编译	enable_if / concepts 约束	宏是文本级条件,C++ 是类型级约束
传错类型编译不过但报错玄妙	requires std::integral<T> 直说	concepts 让错误信息可读
无对应能力	if constexpr 编译期分支	按类型特性裁剪代码路径

清单 10-5 enable_if 与 concepts

```
#include <iostream>
#include <type_traits>
#include <string>
#include <concepts>      // C++20

// enable_if 写法(C++11):只接受算术类型
template <typename T>
typename std::enable_if<std::is_arithmetic_v<T>, T>::type
twice(T v) {
    return v * 2;
}

// concepts 写法(C++20):约束写进签名
template <std::integral T>      // 只接受整型
T triple(T v) {
    return v * 3;
}

template <typename T>
requires std::is_arithmetic_v<T>    // requires 子句
T plusOne(T v) {
    return v + 1;
}

int main() {
    std::cout << twice(21) << " " << twice(2.5) << "\n";    // 42 5
    std::cout << triple(14) << "\n";                        // 42
    std::cout << plusOne(41) << "\n";                        // 42
    // std::cout << twice(std::string("x"));    // 错误:不满足约束
    return 0;
}
```

⚠ 易错点 1:模板定义必须在使用前可见

模板的“代码生成”性质决定了它不能像普通函数那样声明与定义分离:定义在 .cpp 里的函数模板,另一个 .cpp 里调用会直接链接失败(找不到实例)。规则:函数模板与类模板的完整定义放头文件;确要隐藏实现,用显式实例化声明,但只对预定的类型生效。

§10.6 constexpr 与编译期计算

constexpr 函数承诺“能编译期求值就在编译期求值”:只要参数是编译期常量,调用结果就成为常量;参数不是,它就退化为普通函数。constexpr 变量的值在编译期就确定,可以用于模板非类型参数、数组大小等场景。C++14 放宽了

constexpr 函数的限制(允许循环与局部变量),C++17 的 if constexpr 更进一步:按编译期条件裁剪代码分支,不满足的分支根本不会被实例化——这是写"同构算法适配多种类型"的主力工具。

与 C 的宏编程对比,constexpr 的优势是显而易见的:C 用宏做编译期计算,先有 #if 预处理,后有 sizeof 技巧,但宏没有类型、没有作用域、不参与重载决议;constexpr 函数是真正的函数,有类型检查、有调试信息、能被测试。现代 C++ 的编译期计算栈是 constexpr 函数 + if constexpr + 模板,替代了 C 时代的大部分宏魔术,可读性与可维护性都不可同日而语。

清单 10-6 constexpr 函数与 if constexpr

```
#include <iostream>
#include <type_traits>

// constexpr:参数是编译期常量时,结果也是常量
constexpr long long factorial(int n) {
    long long r = 1;
    for (int i = 2; i <= n; ++i) r *= i;    // C++14 起允许循环
    return r;
}

// if constexpr:按类型裁剪分支,不满足的分支不实例化
template <typename T>
auto describe(T v) {
    if constexpr (std::is_integral_v<T>) {
        return std::string("整数 ") + std::to_string(v);
    } else if constexpr (std::is_floating_point_v<T>) {
        return std::string("浮点 ") + std::to_string(v);
    } else {
        return std::string("其他 ") + std::string(v);
    }
}

int main() {
    constexpr long long f = factorial(10);    // 编译期算好
    std::cout << f << "\n";                  // 3628800
    static_assert(f == 3628800, "编译期断言");    // 编译期验证
    std::cout << describe(42) << "\n";
    std::cout << describe(3.14) << "\n";
    std::cout << describe(std::string("文本")) << "\n";
    return 0;
}
```

⚠ 易错点 2:constexpr 不是免费的

constexpr 在编译期递归或循环很深的计算(如 factorial(100000))会让编译器卡死;constexpr 函数体里不能做 I/O、动态内存分配等运行期操作。原则:编译期计算用于小规模常量推导(表、偏移、尺寸),重计算留在运行期。

⚠ 易错点 3:vector<bool> 是特化的陷阱

std::vector<bool> 为节省内存按位存储,元素是代理对象而非真正的 bool 引用:auto b = v[0] 得到代理,改 b 改不到原数组,&v[0] 也取不到 bool 地址。需要真正的布尔数组用 std::vector<char> 或 std::bitset;这个例子提醒我们:特化可以改变语义,读标准库文档比猜行为可靠。

泛型与元编程速查表

概念	语法示例	要点
函数模板	<code>template<typename T> T f(T v)</code>	按实参推导,一份定义多份代码
类模板	<code>template<typename T> class Box</code>	成员函数按需实例化
非类型参数	<code>template<typename T, int N></code>	编译期常量,如 array 长度
全特化	<code>template<> class Box<bool></code>	单一类型专属实现
偏特化	<code>template<typename T> class Box<T*></code>	一族类型的专属实现
参数包	<code>template<typename... Args></code>	任意数量任意类型参数
折叠表达式	<code>(args + ...)</code>	参数包整体运算,注意结合方向
SFINAE	替换失败不是错误	约束候选集,启用条件模板
<code>enable_if</code>	<code>std::enable_if<cond, T>::type</code>	条件启用,老式约束
concepts	<code>template<std::integral T></code>	C++20,约束进签名,报错清晰
constexpr 函数	<code>constexpr int f(int)</code>	编译期可求值,运行期也可用
if constexpr	<code>if constexpr (cond)</code>	编译期分支,不满足的不实例化
<code>static_assert</code>	<code>static_assert(expr, "msg")</code>	编译期断言,失败即编译错误
<code>decltype</code>	<code>decltype(expr)</code>	取表达式类型,配合 auto 使用

自测题

- Q** 1. 模板与 C 宏、Java 泛型在实现上的本质区别是什么?
- Q** 2. 为什么函数模板的定义通常要放在头文件里?
- Q** 3. 全特化与偏特化有何区别?各自举一个标准库中的例子。
- Q** 4. 折叠表达式 `(args - ...)` 与 `(... - args)` 的结果一样吗?为什么?
- Q** 5. SFINAE 与 concepts 解决什么问题?concepts 的优势是什么?

✅ 参考答案

1. C 宏是纯文本替换,无类型检查,有双重求值问题;Java 泛型编译后擦除为一份代码,类型信息丢失,基本类型要装箱;C++ 模板为每个类型实例化生成独立代码,类型检查在编译期,运行期零抽象开销。
2. 模板实例化发生在编译期,每个编译单元都需要看到完整定义才能生成代码;声明与定义分离后,其他编译单元看不到定义,链接时找不到实例。要隐藏实现只能用显式实例化,但只对预定类型生效。
3. 全特化针对单一类型(`template<> class Box<bool>`),如 `std::vector<bool>` 的位压缩;偏特化针对一族类型(`template<typename T> class Box<T*>`),如 `std::hash<T*>` 指针哈希。编译器选择最匹配的特化。
4. 不一样。 $(args - \dots)$ 是右折叠,等价 $a - (b - c)$; $(\dots - args)$ 是左折叠,等价 $(a - b) - c$ 。减法不满足结合律,结果不同;加法、乘法等结合运算则相同。
5. 它们解决"约束模板能接受什么类型":不满足条件的候选从重载集合中移除或直接报错。concepts 把约束写进函数签名(`requires` 子句),报错信息直接指出违反的概念,而 `enable_if` 的报错是长串模板展开,concepts 可读性完胜。

实验:通用统计工具集

本实验用一个小型统计库串联本章全部知识点:函数模板做均值与极值,类模板做固定容量缓冲区,偏特化给指针版缓冲区, `enable_if` 与 `concepts` 约束数值类型,参数包与折叠表达式求总和, `constexpr` 提供编译期常量。代码分三部分,覆盖点已在注释中标明。

实验 10•第一部分:函数模板与数值约束(覆盖:函数模板、`enable_if`、`concepts`、折叠表达式)

```

#include <iostream>
#include <string>
#include <type_traits>
#include <concepts>      // C++20 概念
#include <vector>

// 覆盖:函数模板 + enable_if—只接受算术类型
template <typename T>
typename std::enable_if<std::is_arithmetic_v<T>, T>::type
mean(const std::vector<T>& v) {
    T s{};
    for (const T& x : v) s += x;
    return v.empty() ? T{} : s / (T)v.size();
}

// 覆盖:concepts 约束 + 参数包 + 折叠表达式
template <std::integral... Ts>          // 全部参数都必须是整型
auto sumAll(const Ts&... args) {
    return (args + ...);               // 覆盖:一元右折叠
}

// 覆盖:约束的模板—泛型极值,要求 T 可比较
template <typename T>
requires std::is_arithmetic_v<T>
T maxOf(const std::vector<T>& v) {
    T m = v.front();
    for (const T& x : v) if (x > m) m = x;
    return m;
}

```

实验 10•第二部分:类模板与特化(覆盖:类模板、全特化、偏特化)

```
// 覆盖:类模板—固定容量缓冲区
template <typename T, int N>
class Buffer {
public:
    void push(T v) {
        if (count < N) data[count++] = v;    // 满则丢弃
    }
    T at(int i) const { return data[i]; }
    int size() const { return count; }
    std::string describe() const { return "普通缓冲"; }
private:
    T data[N] = {};
    int count = 0;
};
```

```
// 覆盖:全特化—bool 缓冲,用字节记录
template <int N>
class Buffer<bool, N> {
public:
    void push(bool v) {
        if (count < N) bits[count++] = v;
    }
    bool at(int i) const { return bits[i]; }
    int size() const { return count; }
    std::string describe() const { return "位缓冲"; }
private:
    bool bits[N] = {};
    int count = 0;
};
```

```
// 覆盖:偏特化—指针缓冲,打印地址信息
template <typename T, int N>
class Buffer<T*, N> {
public:
    void push(T* v) {
        if (count < N) data[count++] = v;
    }
    T at(int i) const { return *data[i]; }
    int size() const { return count; }
    std::string describe() const { return "指针缓冲"; }
private:
    T* data[N] = {};
    int count = 0;
};
```

实验 10•第三部分:constexpr 与 main(覆盖:constexpr、if constexpr、static_assert)

```

// 覆盖:constexpr 编译期函数
constexpr int fib(int n) {
    return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

template <typename T>
std::string kind(const T& v) {
    // 覆盖:if constexpr 编译期分支
    if constexpr (std::is_integral_v<T>) {
        return "整型 " + std::to_string(v);
    } else {
        return "其他 " + std::to_string((double)v);
    }
}

int main() {
    // 覆盖:编译期常量与编译期断言
    constexpr int F10 = fib(10);
    static_assert(F10 == 55, "fib(10) 必须是 55");

    Buffer<int, 4> ib; // 类模板
    ib.push(1); ib.push(2); ib.push(3); ib.push(4); ib.push(5);
    std::cout << ib.describe() << ",大小 " << ib.size() << "\n";

    Buffer<bool, 8> bb; // 全特化
    bb.push(true); bb.push(false);
    std::cout << bb.describe() << " " << bb.at(1) << "\n";

    int raw = 9;
    Buffer<int*, 2> pb; // 偏特化
    pb.push(&raw);
    std::cout << pb.describe() << " " << pb.at(0) << "\n";

    std::vector<double> v{1.5, 2.5, 3.5};
    std::cout << "均值 " << mean(v) << ",最大 " << maxOf(v) << "\n";
    std::cout << "总和 " << sumAll(1, 2, 3, 4) << "\n";
    std::cout << kind(42) << ", " << kind(3.14) << "\n";
    return 0;
}

```

实验覆盖点核对:函数模板(mean/maxOf)、enable_if(mean 的算术约束)、concepts(sumAll 的 integral 约束)、参数包与折叠表达式(sumAll)、类模板(Buffer)、非类型参数(N)、全特化(Buffer<bool>)、偏特化(Buffer<T*>)、constexpr(fib)与 static_assert、if constexpr(kind)。扩展练习:给 Buffer 增加全特化 Buffer<char> 记录字符串;或给 mean 增加 concept 版本,对比两种约束写法的报错信息。



工程建议 1:接口用约束,实现用 if constexpr

对外模板接口用 concepts 或 enable_if 声明"接受什么",内部用 if constexpr 按类型裁剪行为。约束在外,分支在内,调用者看到的签名可读,维护者看到的分支清晰,这是模板工程的黄金分工。

工程建议 2:控制模板规模,警惕编译膨胀

模板是代码生成器,实例化一多编译时间与二进制体积都会膨胀。几十个实例是常态,成千上万个实例就要反思:是否该用虚函数(第6章)把运行时变化隔离,或用类型擦除(如 `std::function`)收敛模板面。能用值语义与虚函数解决的问题,不必全部模板化。

本章小结

模板是类型安全的代码生成器,与 C 宏和 Java 泛型有本质区别;特化与偏特化在类型层面定制行为,参数包与折叠表达式处理任意参数;SFINAE、`enable_if` 与 `concepts` 让约束成为签名的一部分;`constexpr` 把计算搬到编译期。下一章讲"没有反射的 C++":RTTI 与动态特性。

🔗 本章目标

诚实认识 C++ 的反射现状:标准至今没有运行时反射,动态特性由 RTTI(`typeid`、`dynamic_cast`)、类型萃取(编译期反射)与宏/代码生成三块拼成。掌握这三块工具的能力边界,知道生产环境用哪些库补齐反射,并理解多态、序列化、日志等场景的真实解法。

📦 前置知识

熟悉虚函数与多态(第6章)、模板与特化(第10章)、强制转换的初步概念。本章代码基于 C++17;RTTI 默认开启,若用 `-fno-rtti` 编译,`typeid` 与 `dynamic_cast` 不可用。

§11.1 没有反射的 C++	§11.2 <code>typeid</code> 与 <code>type_info</code>	§11.3 <code>dynamic_cast</code> 安全转型
§11.4 编译期类型查询	§11.5 宏与代码生成	§11.6 生产环境的选择

§11.1 先说实话:C++ 没有标准反射

Java 的 `getClass()`、C# 的 `typeof` 都能在运行时拿到类型的完整信息:字段列表、方法表、注解,然后做序列化、依赖注入、对象浏览器。C++ 呢?标准委员会讨论了二十多年,反射提案到 C++26 仍在推进,标准库至今没有 `getFields()` 这种东西。作为工程师,正确姿势不是叹息,而是看清替代拼图:`typeid` 能拿到类型身份,`type_traits` 能在编译期查询类型性质,模板特化能按类型生成定制行为,宏与代码生成工具能产出"字段表"。四块拼起来,反射的主要用途(序列化、日志、类型分派)都做得成,只是不像 Java 那样"免费"。

第二个要说清的事实:很多"需要反射"的需求其实不需要反射。想在运行时按对象类型执行不同逻辑?虚函数(第6章)就是类型分派的原生机制,效率远高于查反射表。想写"每种类型都适用的算法"?模板(第10章)在编译期就完成了分派。反射真正不可替代的场景是有限的:把对象转成字符串/JSON(序列化)、从配置/数据库按字段名填值、类型名注册表。抓住这三类场景,下文逐一给出现有工具的正解。

§11.2 `typeid` 与 `type_info`

`typeid` 是 RTTI(Run-Time Type Information,运行时类型信息)的入口:`typeid(表达式)` 或 `typeid(类型)` 返回 `std::type_info` 的引用,携带类型的身份信息。`typeid` 对多态对象给出动态类型(派生类),对非多态对象给出静态类型。`type_info` 能做的有限:`==` 判断两个类型是否相同,`hash_code()` 得到稳定哈希,`name()` 返回实现定义的名称——注意 `name()` 的格式因编译器而异(GCC 是修饰名),只适合日志,绝不能当协议字符串。

`typeid` 的实际用途比想象中窄:它适合"查类型身份"(如 `assert` 某个对象确实是预期类型)、调试与日志(打印动态类型名)。它不适合字段遍历、方法调用——`type_info` 根本不含这些信息。RTTI 的开销:每个多态类型增加一份类型信息表,`typeid` 是查表操作,代价很小;但 `-fno-rtti` 的嵌入式环境会去掉全部 RTTI,此时 `typeid` 无法编译,只能退回到模板方案。

清单 11-1 `typeid` 与 `type_info`

```

#include <iostream>
#include <string>
#include <typeinfo>

class Animal {
public:
    virtual ~Animal() = default;    // 多态:typeid 才反映动态类型
};

class Dog : public Animal {};
class Cat : public Animal {};

int main() {
    Animal* p = new Dog;
    // 多态对象:typeid 给出实际类型 Dog
    std::cout << "动态类型: " << typeid(*p).name() << "\n";
    if (typeid(*p) == typeid(Dog)) {
        std::cout << "确实是 Dog\n";
    }
    // 非多态对象:typeid 只认静态类型
    Animal plain;
    std::cout << "静态类型: " << typeid(plain).name() << "\n";

    std::cout << "hash: " << typeid(Dog).hash_code() << "\n";
    delete p;
    return 0;
}

```

C 的写法	C++ 的写法	说明
struct { int tag; union {...}; } 手写类型标记	typeid 由编译器维护	类型信息不可伪造,不会忘初始化
switch (obj->tag) 手工分派	虚函数 / dynamic_cast	C 每加类型改全部分派点,C++ 新增类零改动
tag 与真实数据可能对不上	typeid 永远与对象一致	RTTI 信息由编译器生成

§11.3 dynamic_cast:带检查的向下转型

C 的多态转型只有一个工具:(Dog*)p 裸强转——程序员保证 p 真的是 Dog,编译器不检查,错了就是未定义行为。C++ 的 dynamic_cast 把检查做进语言:dynamic_cast<Dog*>(p) 运行时核实 p 的动态类型与目标兼容,成功返回指针,失败返回 nullptr;引用版本失败抛 std::bad_cast。转型的目标必须是多态类型(含虚函数),否则编译错误——这是编译期的强制约束,杜绝了"C 里对任意结构体瞎转"的自由。

工程上 dynamic_cast 的正确位置:偶尔需要"基类指针 + 派生类专属方法"时,它比维护一整套虚函数接口要轻;但把它当常规流程(每个对象都转一遍)就是设计气味——能靠虚函数表达的多态,别靠转型表达。规则:基类接口能完成的事用虚函数;确实只有特定派生类才有的能力,用 dynamic_cast 判空后使用;判空不可省略,空指针访问是崩溃。

C 的写法	C++ 的写法	说明
(Dog*)p 裸强转,不检查	dynamic_cast<Dog*>(p) 运行时检查	C 错了是未定义行为,C++ 失败返回空
对任意结构体都能强转	目标必须多态,否则编译错误	C++ 在编译期就拦住乱转
类型判断靠手写标记字段	typeid 比较动态类型	C++ 类型信息由编译器维护

清单 11-2 dynamic_cast 安全转型

```

#include <iostream>
#include <vector>
#include <memory>

class Shape {
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

class Circle : public Shape {
public:
    explicit Circle(double r) : r(r) {}
    double area() const override { return 3.14159 * r * r; }
    double radius() const { return r; }          // 派生类专属方法
private:
    double r;
};

class Rect : public Shape {
public:
    Rect(double w, double h) : w(w), h(h) {}
    double area() const override { return w * h; }
private:
    double w, h;
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rect>(2.0, 3.0));

    for (const auto& s : shapes) {
        if (auto c = dynamic_cast<Circle*>(s.get())) { // 失败返回空
            std::cout << "圆形,半径 " << c->radius() << "\n";
        } else {
            std::cout << "非圆形,面积 " << s->area() << "\n";
        }
    }
    return 0;
}

```

⚠ 易错点 1:dynamic_cast 的结果不判空

dynamic_cast<T*> 转型失败返回空指针,直接解引用就是空指针崩溃。引用版本失败则抛 std::bad_cast。指针版必判空,或用 if (auto x = dynamic_cast<T*>(p)) 的组合写法,把检查与使用绑在一起。

⚠ 易错点 2:对空指针做 typeid

typeid(*p) 当 p 是空指针时,对多态类型会抛出 std::bad_typeid,而不是安静地返回未知类型。日志代码最容易踩:为了打印类型名对空指针解引用。先判空再 typeid,或让空指针走独立分支。

§11.4 编译期类型查询:type_traits

反射的另一个方向是编译期:std::type_traits 头文件提供一整组类型性质查询——is_same_v 判断类型相同、is_base_of_v 判断继承关系、is_arithmetic_v 判断算术类型、is_convertible_v 判断可转换性,加上 decltype 取表达式类型,配合 if constexpr(第10章),类型判断全部在编译期完成,零运行开销。这就是 C++ 的"编译期反射":信息不藏在运行时表里,而是由模板机制在编译时推导。

类型萃取与模板特化组合,能做出"类型字典"级别的定制:为每种类型声明一个 traits 特化,提供类型名称、默认值、序列化函数等元信息;通用模板给默认实现,特化按类型覆盖。标准库就是这样组织的:std::iterator_traits、std::numeric_limits 都是 traits 的样板。这套方案的价值在"按类型定制且零开销",局限也在编译期——字段列表、运行时按名字查找字段,仍然做不了,那是 §11.5 的领域。

清单 11-3 type_traits 编译期查询

```

#include <iostream>
#include <string>
#include <type_traits>

// traits 特化:每种类型提供"类型名"
template <typename T>
struct TypeName { static constexpr const char* value = "未知"; };

template <>
struct TypeName<int> { static constexpr const char* value = "整数"; };

template <>
struct TypeName<double> { static constexpr const char* value = "浮点"; };

template <typename T>
std::string typeInfo(T v) {
    if constexpr (std::is_arithmetic_v<T>) {
        return std::string(TypeName<T>::value) + " " + std::to_string(v);
    } else {
        return std::string(TypeName<T>::value) + " " + v;
    }
}

int main() {
    std::cout << typeInfo(42) << "\n";           // 整数 42
    std::cout << typeInfo(3.14) << "\n";         // 浮点 3.14
    std::cout << typeInfo(std::string("文本")) << "\n"; // 未知 文本
    std::cout << std::boolalpha
                << std::is_base_of_v<std::string, std::string> << "\n";
    return 0;
}

```

§11.5 宏与代码生成:运行时反射的替代品

真正需要"运行时拿到字段/枚举信息"时,标准工具是宏与代码生成。X-macro 是最优雅的模式:把数据列表写一遍,用宏把它展开进多份代码——一份生成枚举定义,一份生成枚举到字符串的映射,一份生成字符串到枚举的解析。改动列表只需改一处,三份代码同步更新,彻底消灭"枚举与字符串数组对不上"这类 C 时代的经典 bug。这就是"代码生成"思想的手工版:让数据定义成为唯一事实来源。

生产环境通常直接用现成的代码生成器:protobuf 从 .proto 文件生成 C++ 序列化类,flatbuffers 同理,生成的代码自带字段表、序列化、反射接口;header 级库方面,PFR(基于模板技巧在编译期数出结构体字段个数,无宏)、magic_enum(枚举与字符串互转)、reflect-cpp(完整序列化框架)都是经过生产检验的选择。选型建议:字段多、要跨语言 → protobuf 类工具;只想要枚举转字符串 → magic_enum;只想少写样板 → PFR 或 reflect-cpp。它们共同的哲学与 X-macro 一致:元信息由工具生成,而不是手写。

C 的写法	C++ 的写法	说明
枚举与字符串数组两份手工维护	X-macro:一份列表生成多份代码	单一事实来源,改一处全同步
结构体字段表手写,易对不上	protobuf / PFR 生成字段信息	生成代码由工具保证一致
类型名映射手写数组	type_traits + 特化自动查询	编译期查询,零运行开销

```

#include <iostream>
#include <string>

// 数据列表:唯一事实来源
#define COLOR_LIST(X) \
    X(Red) \
    X(Green) \
    X(Blue) \
    X(Count)

// 第一次展开:生成枚举
enum Color {
#define X(name) name,
    COLOR_LIST(X)
#undef X
};

// 第二次展开:生成枚举到字符串的映射
const char* colorName(Color c) {
    switch (c) {
#define X(name) case name: return #name;
        COLOR_LIST(X)
#undef X
    }
    return "未知";
}

// 第三次展开:生成字符串到枚举的解析
bool colorParse(const std::string& s, Color& out) {
#define X(name) if (s == #name) { out = name; return true; }
    COLOR_LIST(X)
#undef X
    return false;
}

int main() {
    Color c = Green;
    std::cout << colorName(c) << "\n";           // Green
    Color parsed;
    if (colorParse("Blue", parsed)) {
        std::cout << "解析成功 " << colorName(parsed) << "\n";
    }
    return 0;
}

```

⚠ 易错点 3:宏不尊重作用域,展开不可调试

X-macro 的 #define 会污染全局命名空间,#undef 必须与 #define 成对,宏内的拼接与字符串化写法晦涩,调试时断点打在展开后的代码上。规则:宏模式只用于"多份代码必须同步"的机械展开;能写成模板或 constexpr 的逻辑,优先语言特性,宏是最后的工具。

§11.6 生产环境的选择:需求对表

把本章的工具按需求对表:需要对象转 JSON/序列化——protobuf、reflect-cpp 或手写 toJson(模板+if constexpr 自动遍历);需要枚举与字符串互转——magic_enum,一个头文件;需要按字段名从配置填值——代码生成器生成的解析器;需要类型名/注册表——typeid 加静态注册;需要按运行时类型分派——虚函数,而不是反射。对照下来你会发现,Java 反射解决的问题,在 C++ 里几乎都有专门的正解,代价是"每个问题都要选对工具",而不是一个万能 API。

还有一个常被忽略的工具是静态注册:程序启动时用全局对象的构造函数把"类型名与工厂函数"登记进注册表(见清单 11-5),运行时按字符串查找。配合 typeid 的类型名与模板注册函数,几十行代码就能实现插件系统与按名创建对象,这是反射需求里性价比最高的自建方案,也是 codegen 之外最常见的工程实践。

最后是心态建议:不要为"没有反射"写哀歌,也别为"假装有反射"造一个蹩脚的通用框架。C++ 的哲学是把能力分散给专门的机制:多态给虚函数,编译期计算给模板,元信息给生成代码。三者的组合比单一反射更强——虚函数快、模板零开销、生成代码可审计,这正是 C++ 在系统级软件里不可替代的原因。

清单 11-5 类型注册表:typeid 哈希做键

```
#include <functional>
#include <map>
#include <memory>
#include <typeinfo>
#include <string>

struct Registry {
    // 类型名 -> 工厂函数
    std::map<std::string, std::function<void*>> makers;

    template <typename T>
    void reg() {
        // 模板注册:自动取类型名
        makers[typeid(T).name()] = [] { return new T(); };
    }

    void* create(const std::string& name) {
        auto it = makers.find(name);
        return it == makers.end() ? nullptr : it->second();
    }
};

struct A {}; struct B {};

int main() {
    Registry r;
    r.reg<A>();
    r.reg<B>();
    void* a = r.create(typeid(A).name()); // 按名字创建
    std::cout << (a ? "A 创建成功" : "A 创建失败") << "\n";
    delete static_cast<A*>(a);
    return 0;
}
```

反射与动态特性速查表

概念	语法示例	要点
RTTI	运行时类型信息	typeid 与 dynamic_cast 的基础
typeid	typeid(x) / typeid(T)	多态对象返回动态类型
type_info	typeid(x).name()	name 实现定义,只适合日志
类型比较	typeid(a) == typeid(B)	判断动态类型是否相同
hash_code	typeid(T).hash_code()	稳定哈希,可做注册表键
dynamic_cast	dynamic_cast<T*>(p)	多态向下转型,失败返回空
bad_cast	引用版转型失败抛出	catch 处理引用转型失败
类型萃取	std::is_same_v<T, U>	编译期查询,零运行开销
继承判断	std::is_base_of_v<B, D>	编译期判断继承关系
decltype	decltype(expr)	取表达式类型
X-macro	列表宏展开多份代码	枚举/字符串/解析同步生成
magic_enum	magic_enum::enum_name(v)	枚举字符串互转,单头文件
PFR	pfr::get<i>(obj)	编译期结构体字段访问
代码生成	protobuf / flatbuffers	从描述文件生成序列化代码

自测题

- Q 1. C++ 为什么没有 Java 那样的运行时反射?标准反射提案的现状如何?**
- Q 2. typeid 返回什么?type_info::name() 为什么不能当协议字符串?**
- Q 3. dynamic_cast 与 C 的裸强转有何区别?为什么目标类型必须多态?**
- Q 4. type_traits 能实现哪类"反射"?它有什么局限?**
- Q 5. X-macro 为什么能保证枚举与字符串数组一致?生产环境有哪些现成替代?**

✓ 参考答案

- 反射提案在标准委员会讨论了二十多年,C++26 仍在推进,标准库没有 getFields 之类的接口。设计上 C++ 优先零开销抽象与显式机制,反射的通用性以运行时开销与代码膨胀为代价,与语言哲学相悖。
- typeid 返回 std::type_info 引用,描述类型的身份;name() 的格式由编译器实现决定(GCC 是修饰名),不同编译器不同版本都可能不同,只适合日志展示,作为协议字符串会跨编译器失配。
- C 的裸强转不做任何检查,类型错了是未定义行为;dynamic_cast 运行时核实动态类型,失败指针版返回空、引用版抛 bad_cast。目标必须多态是因为检查要查虚表里的类型信息,非多态类没有虚表。
- type_traits 在编译期回答类型性质问题:是否相同、是否算术类型、是否继承、是否可转换,配合 if constexpr 做零开销分派。局限:没有运行时字段表,不能按名字查字段、不能枚举方法。
- X-macro 只有一份数据列表,枚举定义、名称映射、解析函数都从它展开,改列表即改全部,不存在两份手工维护的偏差。生产替代:magic_enum(枚举)、PFR(结构体字段)、protobuf/reflect-cpp(完整序列化)。

实验:日志系统的类型感知输出

本实验构建一个带类型感知的日志与序列化系统:typeid 打印动态类型,type_traits 与 if constexpr 按类型格式化数值与文本,traits 特化提供类型名,X-macro 定义日志级别枚举与字符串映射,dynamic_cast 安全访问派生类专属字段。代码分三部分,覆盖点已在注释中标明。

实验 11•第一部分:类型名与格式化(覆盖:typeid、type_traits、if constexpr、traits 特化)

```
#include <iostream>
#include <string>
#include <typeinfo>
#include <type_traits>
#include <sstream>

// 覆盖:traits 特化—每个类型提供中文类型名
template <typename T>
struct TypeName { static constexpr const char* value = "未知"; };
template <>
struct TypeName<int> { static constexpr const char* value = "整数"; };
template <>
struct TypeName<double> { static constexpr const char* value = "浮点"; };
template <>
struct TypeName<std::string> { static constexpr const char* value = "文本"; };

// 覆盖:type_traits + if constexpr 按类型格式化
template <typename T>
std::string formatValue(const T& v) {
    if constexpr (std::is_arithmetic_v<T>) {
        return std::to_string(v);
    } else if constexpr (std::is_same_v<T, std::string>) {
        return v;
    } else {
        return "?";
    }
}

// 覆盖:typeid 打印动态类型名,type_info 比较
std::string dynamicTag(const void* p, const std::type_info& dyn) {
    (void)p;
    return dyn.name();           // 实现定义的名称,仅日志用
}
```

实验 11•第二部分:日志级别与动态转型(覆盖:X-macro、dynamic_cast、多态)

```

// 覆盖:X-macro—日志级别:一份列表,生成枚举与字符串
#define LEVEL_LIST(X) \
    X(Debug) \
    X(Info) \
    X(Warn) \
    X(Error) \
    X(Count)

enum Level {
#define X(name) name,
    LEVEL_LIST(X)
#undef X
};

const char* levelName(Level l) {
    switch (l) {
#define X(name) case name: return #name;
        LEVEL_LIST(X)
#undef X
    }
    return "?";
}

// 覆盖:多态基类,为 dynamic_cast 提供目标
struct LogEntry {
    virtual ~LogEntry() = default;
    virtual std::string summary() const = 0;
};

struct ErrorEntry : LogEntry {          // 派生类专属字段
    int code;
    explicit ErrorEntry(int c) : code(c) {}
    std::string summary() const override {
        return "错误码 " + std::to_string(code);
    }
};

struct TextEntry : LogEntry {
    std::string text;
    explicit TextEntry(std::string t) : text(std::move(t)) {}
    std::string summary() const override { return text; }
};

```

实验 11•第三部分:main 调度(覆盖:typeid 分派、dynamic_cast 判空、综合输出)

```

int main() {
    // 覆盖:typeid 动态类型识别
    TextEntry t("连接建立");
    LogEntry* p = &t;
    std::cout << "动态类型: " << typeid(*p).name() << "\n";
    if (typeid(*p) == typeid(TextEntry)) {
        std::cout << "识别为文本条目\n";
    }

    // 覆盖:dynamic_cast 判空后访问派生类专属字段
    LogEntry* e = new ErrorEntry(500);
    if (auto err = dynamic_cast<ErrorEntry*>(e)) {
        std::cout << "错误码 " << err->code << "\n";
    } else {
        std::cout << "不是错误条目\n";
    }
    delete e;

    // 覆盖:traits 类型名 + 格式化 + X-macro 级别
    int n = 42;
    std::cout << levelName(Warn) << " " << TypeName<decltype(n)>::value
        << " " << formatValue(n) << "\n";
    std::cout << levelName(Info) << " " << TypeName<std::string>::value
        << " " << formatValue(std::string("运行中")) << "\n";
    std::cout << levelName(Error) << " " << TypeName<double>::value
        << " " << formatValue(0.5) << "\n";
    return 0;
}

```

实验覆盖点核对:typeid 动态类型识别与 type_info 比较、dynamic_cast 判空后访问派生类专属字段、type_traits 的 is_arithmetic_v/is_same_v 与 if constexpr 格式化、traits 特化提供类型名(TypeName)、X-macro 生成枚举与字符串映射(levelName)、多态基类与虚函数(summary)。扩展练习:给 Level 增加解析函数(字符串转枚举);或实现一个简单的 Entry 注册表,用 typeid(T).hash_code() 做键。

工程建议 1:日志与序列化优先生成代码

新项目要序列化与日志反射,第一时间评估 protobuf、flatbuffers、reflect-cpp,而不是手写 X-macro 与映射函数:生成代码经得起生产考验,字段变更全自动同步,还有跨语言红利。手写元信息只适合字段极少、库引入成本高的场合。

工程建议 2:分派优先虚函数,转型是最后手段

代码里出现"逐个 dynamic_cast 试类型"是设计信号:把分支逻辑上移为虚函数或访问者模式,新类型不用改分发代码。dynamic_cast 保留给"偶尔需要派生类专属能力"的少数场景,并且判空永远不省。

本章小结

C++ 没有 Java 式反射,但四块工具各司其职:typeid 识别类型身份,dynamic_cast 做安全检查和转型,type_traits 在编译期查询类型,宏与代码生成提供运行时元信息。序列化、日志、类型分派的真实解法都落在这四块上。下一章进入注解与编译期指令:标准属性。

 本章目标

掌握 C++ 的属性(attribute)机制:标准属性 `nodiscard`、`deprecated`、`fallthrough`、`maybe_unused`、`noreturn`、`likely/unlikely` 的用途与写法;理解 GCC/Clang 的 `__attribute__` 与 MSVC 的 `__declspec` 扩展;学会用兼容性宏与条件编译写出跨编译器、跨平台的可移植代码。

 前置知识

熟悉函数声明、`switch` 语句(第1、3章);了解预处理宏(第10章)。本章代码基于 C++17(属性语法 C++11 起),`likely/unlikely` 为 C++20,编译器扩展部分按 GCC/Clang 语法展示并给出 MSVC 对照。

§12.1 属性与注解	§12.2 标准属性全览	§12.3 属性的契约价值
§12.4 编译器扩展	§12.5 与 C 的对比	§12.6 工程实践

§12.1 属性是什么:给编译器的契约

C++ 的属性(attribute)是写在双中括号里的标记:`[[nodiscard]]`、`[[deprecated("原因")]]`、`[[fallthrough]]`,放在函数、变量、类型或语句之前,向编译器声明某种契约。属性不引入新的运行时行为——它要么是编译期检查的开关(`nodiscard` 丢弃返回值就报警),要么是给实现的信息(`deprecated` 的弃用警告),要么是让程序员意图显式化(`fallthrough` 声明穿透是有意的)。与 C#/Java 的注解不同,那里的注解是运行时可查的元数据,可以驱动框架行为;C++ 的属性基本是“编译期的话”,运行时不留下痕迹。

属性语法有三个要素:名字(标准属性名或命名空间限定名)、可选参数(括号里的字符串等)、位置(函数前、返回类型后、语句前等)。编译器遇到不认识的属性应当忽略并给出警告(标准要求实现可以忽略未知属性)——这给了扩展空间,也让拼错名字的后果变得隐蔽:属性不生效只是少一次检查,不报错。工程上因此要配合 `-Werror` 之类的严格编译选项,让属性错误显形。

属性在工具链里的角色也要说清:clang-tidy、GCC 的 `-Wattributes` 能检查属性写法的合法性与位置;静态分析器会读取 `nodiscard`、`deprecated` 并把发现写进报告。属性是“人写契约、编译器执行、工具链审计”三环节的中间层——契约写得越准,编译器与静态分析就越有用。把属性当作接口文档的一部分来写,而不是随手加的花哨装饰。

清单 12-0 属性的位置

```

#include <iostream>
#include <string>

[[nodiscard]] int f(int v);           // 位置 1:函数声明前
int g(int v) [[nodiscard]] { return v; } // 位置 2:函数定义后

class [[deprecated]] OldApi { };      // 类型前:整个类型弃用

[[nodiscard]] int mustUse() { return 7; }

int main() {
    int x = mustUse();                // 正确使用
    // mustUse();                     // 警告:丢弃返回值
    std::cout << x << "\n";
    return 0;
}

```

§12.2 标准属性全览

标准属性数量不多,个个有用。[[nodiscard]] 声明"返回值不得丢弃":调用方忽略返回值时编译器报警,适合错误码、句柄、资源类、以及一切"结果必须被处理"的函数;C++17 起可以带原因文字 [[nodiscard("必须处理返回值")]],报警时一并显示。[[deprecated]] 标记弃用,并附迁移建议;调用弃用接口产生警告,让旧接口的存量调用逐步现形,是库演进的温和工具。

[[fallthrough]] 用在 switch 的 case 末尾,声明"这里故意穿透到下一个 case",抑制"穿透警告"并给读者明确信号——没有它,编译器(与同事)都默认穿透是疏忽。[[maybe_unused]] 声明"这个实体可能用不到",抑制未使用警告,适合调试参数、条件编译产物与占位形参。[[noreturn]] 声明函数不会返回(如 terminate、自定义错误处理),让编译器知道函数之后的代码不可达,顺带优化。C++20 的 [[likely]] 与 [[unlikely]] 给分支预测提示:告诉编译器哪个分支是热路径,对性能敏感的分支判断有实际收益。

清单 12-1 标准属性全家福

```

#include <iostream>
#include <string>

// nodiscard: 丢弃返回值必报警,带原因文字
[[nodiscard("结果必须被处理")]]
int check(int v) {
    return v > 0 ? 0 : -1;
}

// deprecated: 旧接口弃用,提示迁移方向
[[deprecated("改用 check()")]]
int legacyCheck(int v) { return v > 0 ? 0 : -1; }

// noreturn: 函数永不返回
[[noreturn]] void fatal(const std::string& msg) {
    std::cerr << "致命错误: " << msg << "\n";
    std::exit(1);           // 之后代码不可达
}

// maybe_unused: 这个参数某些编译配置下用不到
void logAt([[maybe_unused]] int verbose, const std::string& msg) {
    if (verbose > 0) std::cout << msg << "\n";
}

int classify(int v) {
    switch (v) {
    case 0:
        return 0;
    case 1:
        [[fallthrough]];           // 故意穿透: 1 与 2 同结果
    case 2:
        return 1;
    default:
        return -1;
    }
}

int main() {
    if (check(5) == 0) std::cout << "通过\n";
    // check(5);                  // 警告: 丢弃了带 nodiscard 的结果
    // legacyCheck(5);            // 警告: 调用已弃用接口
    std::cout << classify(1) << " " << classify(2) << "\n";
    logAt(0, "静默模式");
    return 0;
}

```

C 的写法	C++ 的写法	说明
case 穿透靠运气与注释	case 1: [[fallthrough]];	C 穿透是默认行为,C++ 要求显式声明意图
错误码调用点检查靠自觉	[[nodiscard]] 编译器强制	契约从注释升级为编译期检查
弃用接口直接删除或留着	[[deprecated]] 温和过渡	存量调用先报警后清理,不破接口

§12.3 属性的契约价值:nodiscard 的工程意义

属性里工程价值最高的是 `[[nodiscard]]`, 它的本质是把"必须使用返回值"从口头约定变成编译期强制。C 时代, 错误码函数的手册上写着"调用者必须检查返回值", 但编译器不管, 漏检查的代码静默通过, 错误悄悄吞掉——这是 C 程序里最隐蔽的一类 bug。加上 `[[nodiscard]]` 之后, 所有调用点漏检查都会报警, 配合 `-Werror` 直接编译失败, 错误无处遁形。这正是 C 程序员最该尽快用上的 C++ 能力之一。

应用场景清单: 返回错误码的函数(必查)、返回资源句柄/智能指针的函数(必接管, 否则泄漏)、返回新分配的对象的工厂函数、以及"返回值是唯一出口"的查询函数。标准库带头示范: `std::make_unique`、`std::vector::empty`、`std::map::insert` 等在新标准中都被标记了 `nodiscard`。库作者给公共 API 的每个"结果型"函数都加 `nodiscard`, 接口自文档化, 调用方被编译器教育着写对代码。

清单 12-2 `nodiscard` 的工程应用

```
#include <iostream>
#include <memory>
#include <fstream>
#include <string>

// 错误码函数: 结果必须检查
enum class Err { Ok, NotFound, Full };
[[nodiscard]] Err tryInsert(int v) {
    return v > 0 ? Err::Ok : Err::Full;
}

// 工厂函数: 指针必须被接管, 否则泄漏
[[nodiscard]] std::unique_ptr<std::string>
makeGreeting(const std::string& name) {
    return std::make_unique<std::string>("你好, " + name);
}

// 打开文件: 失败必须被处理
[[nodiscard]] std::ifstream openLog(const std::string& path) {
    return std::ifstream(path);           // C++11 起 ifstream 可移动返回
}

int main() {
    Err e = tryInsert(42);                 // 正确处理
    if (e != Err::Ok) return 1;

    auto g = makeGreeting("张三");         // unique_ptr 接管
    std::cout << *g << "\n";

    std::ifstream log = openLog("run.log");
    if (!log) {
        std::cerr << "日志打不开, 但不致命\n";    // 处理失败路径
    }
    // tryInsert(-1);                       // 错误: 丢弃错误码, 编译报警
    // makeGreeting("李四");                 // 错误: 泄漏风险, 编译报警
    return 0;
}
```

⚠ 易错点 1:属性位置写错,静默失效

`[[nodiscard]]` 放在返回类型之前是有效的,但和函数名之后、参数列表之前等位置混用时,编译器可能当作未知属性忽略并只给警告;函数指针、lambda、类型别名上的属性位置规则更绕。规则:属性紧跟它修饰的实体,标准写法按 `cppreference` 示例抄,别自由发挥;开 `-Wattributes` 让位置错误显形。

⚠ 易错点 2:nodiscard 的边界

`[[nodiscard]]` 只对"丢弃返回值"报警:void 函数加上无意义;C++20 之前的版本,先把返回值存进变量再丢弃、或 `static_cast<void>(f())` 显式忽略,都不报警——后者其实是合法的"我确实不想处理"。它拦的是疏忽,不是恶意绕过,别指望它替代代码评审。

§12.4 编译器扩展: `__attribute__` 与 `__declspec`

标准属性之外的天地由编译器方言统治:GCC/Clang 用 `__attribute__((...))`,MSVC 用 `__declspec(...)`。常见扩展:`__attribute__((packed))` 取消结构体对齐填充,用于协议头等紧凑布局;`__attribute__((aligned(16)))` 指定对齐,适合 SIMD 数据;`__attribute__((unused))` 抑制未使用警告(标准版是 `[[maybe_unused]]`);`__attribute__((visibility("hidden")))` 控制动态库符号可见性;MSVC 的 `__declspec(dllexport/dllimport)` 是 Windows 动态库导出导入的标配。它们与标准属性的区别:能力更强、语法不同、不可移植。

不可移植意味着直接裸用,换编译器就编译失败。工程标准做法是封装成宏:CPP_API、CPP_PACKED 之类,内部用条件编译分别展开成各编译器的写法,业务代码只写宏。宏还要处理"未知编译器怎么办"——退化为空,保持可编译。这套兼容性封装是跨平台库的地基:Windows 下展开成 `__declspec`,Linux 下展开成 `__attribute__`,macOS 与 Linux 一致,三个平台共用一份头文件。

C 的写法	C++ 的写法	说明
<code>#pragma pack(1)</code> 全局生效	<code>[[gnu::packed]]</code> 或封装宏	<code>pragma</code> 影响范围含糊,属性精确到实体
<code>__attribute__</code> 方言裸用	兼容宏 + 条件编译	C++ 惯例:方言藏进宏,代码可移植
错误码检查靠自觉	<code>[[nodiscard]]</code> 编译器强制	C++ 把约定变成检查

清单 12-3 `packed` 协议头与兼容宏

```

#include <iostream>
#include <cstdint>

// 兼容宏:一个名字,三种平台各自的写法
#if defined(_MSC_VER)
#define CPP_PACKED_BEGIN __pragma(pack(push, 1))
#define CPP_PACKED_END __pragma(pack(pop))
#elif defined(__GNUC__) || defined(__clang__)
#define CPP_PACKED_BEGIN
#define CPP_PACKED_END
#define CPP_PACKED_ATTR __attribute__((packed))
#else
#define CPP_PACKED_BEGIN
#define CPP_PACKED_END
#define CPP_PACKED_ATTR
#endif

CPP_PACKED_BEGIN
// 网络协议头:紧凑布局,无对齐填充
struct PacketHeader {
    std::uint32_t magic;        // 魔数
    std::uint16_t type;        // 消息类型
    std::uint16_t length;      // 正文长度
    std::uint8_t checksum;     // 校验和
} CPP_PACKED_ATTR;
CPP_PACKED_END

int main() {
    // packed 后:4 + 2 + 2 + 1 = 9 字节,没有填充
    std::cout << "包头大小: " << sizeof(PacketHeader) << " 字节\n";
    PacketHeader h{0x12345678, 1, 100, 0xAA};
    std::cout << "magic=" << std::hex << h.magic << std::dec
              << ", type=" << h.type << "\n";
    return 0;
}

```

⚠ 易错点 3:packed 的代价

取消对齐填充换来紧凑布局,代价是未对齐访问:成员横跨字边界,读取可能变慢,在部分架构(如老式 ARM)上直接是硬件错误;同时 packed 结构体不能安全地取成员地址交给需要对齐的接口。协议头用 packed 合理,热点数据结构别用,能用字段序列化就别整块打包。

§12.5 与 C 的对比:从 pragma 到属性

C 程序员对"注解"并不陌生,只是它的形态是一堆方言:#pragma pack、#pragma GCC diagnostic、__attribute__((packed)),各编译器语法不一、行为不齐、没有标准。C23 标准开始引入 [[nodiscard]]、[[deprecated]]、[[fallthrough]] 等属性,语法与 C++ 对齐,但数量仍远少于 C++ 标准库的约定。C++ 的路线更彻底:C++11 起就把属性语法标准化,标准属性随语言演进持续增加,编译器扩展通过命名空间属性(如 [[gnu::always_inline]])与标准语法统一。

迁移到 C++ 时,最该做的三件事:把 return 型错误检查的函数全部加上 [[nodiscard]];把 #pragma pack 的用法改成 C++ 属性或封装宏;把 __attribute__ 裸用收进兼容宏。改动量不大,收益立竿见影:编译器的检查能力立刻多了一层。至于"注解能驱动框架行为"的期待,回到上一章的结论——C++ 的注解是给编译器的,框架行为请交给模板与代码生成。

C 的写法	C++ 的写法	说明
#pragma pack(1) 文件级生效	实体级属性,作用域精确	C++ 注解不污染上下文
__attribute__((packed)) 无标准	标准 [...] 语法统一	C++ 属性标准化,方言靠边
错误码函数无强制检查	[[nodiscard]] 编译期报警	C23 开始跟进,C++ 已成熟

清单 12-4 C 的 pragma 与 C++ 的属性对照

```
/* C 的写法:#pragma 与方言,影响范围不精确 */
#pragma pack(push, 1)
struct Packet {
    unsigned char ver;
    unsigned short len;
};
#pragma pack(pop)

// C++ 的写法:属性精确到实体,方言收进宏
#define PACKED __attribute__((packed))
struct Packet {
    std::uint8_t ver;
    std::uint16_t len;
} PACKED;

// C23 起,标准属性与 C++ 对齐:
// [[nodiscard]]、[[deprecated]]、[[fallthrough]]
```

§12.6 属性的工程实践

属性用得好不好,取决于项目纪律。四条实践:第一,严格编译选项打底——-Wall -Wextra -Werror(MSVC 对应 /W4 /WX),让属性报警升级为编译失败,否则"报警忽略"和"没写属性"没区别;第二,公共 API 的结果型函数全部 nodiscard,错误码、句柄、工厂函数一个不漏;第三,弃用走温和路线——先 deprecated 给过渡期,统计存量调用清理完再删除,别一夜之间删接口;第四,方言扩展全部收进封装宏,头文件里一个裸 __attribute__ 都不该有,换编译器只需改宏定义。

最后澄清一个常见误解:属性不是性能魔法。[[likely]]/[[unlikely]] 只是提示,现代 CPU 的分支预测器有自己的统计;[[noreturn]]、[[maybe_unused]] 主要改善诊断与代码可读性。属性的价值集中在两处:让编译器替你检查约定(contract),让代码意图显式化(documentation)。把这两点用好,属性就值回票价了。

代码评审时,属性应当作为接口评审的一环:新函数有没有结果被丢弃的风险?旧接口有没有明确弃用?packed 有没有出现在不该出现的地方?评审清单里加一条"属性检查",比事后补报警要便宜得多。另外记住属性是渐进能力:C++11 的六个属性用到熟,再拥抱 C++20 的 likely/unlikely 与 C++23 的 [[assume]],不必一次学全。

注解与属性速查表

属性	语法示例	要点
nodiscard	[[nodiscard]] int f();	丢弃返回值即报警,C++17 可带原因
deprecated	[[deprecated("用新接口")]]	调用即警告,带迁移建议
fallthrough	[[fallthrough]];	声明 switch 穿透是有意的
maybe_unused	[[maybe_unused]] int x;	抑制未使用警告
noreturn	[[noreturn]] void die();	声明函数不返回,代码不可达
likely	if ([[likely]] cond)	C++20,热路径提示
unlikely	if ([[unlikely]] cond)	C++20,冷路径提示
命名空间属性	[[gnu::always_inline]]	编译器扩展的标准语法
packed	__attribute__((packed))	取消填充,协议头常用
aligned	__attribute__((aligned(16)))	指定对齐,配合 SIMD
visibility	__attribute__((visibility("hidden")))	动态库符号可见性
dllexport	__declspec(dllexport)	MSVC 动态库导出
兼容宏	CPP_API / CPP_PACKED	方言收进宏,代码可移植
严格编译	-Wall -Wextra -Werror	属性报警升级为编译失败

自测题

- Q 1. C++ 属性与 Java/C# 注解的本质区别是什么?**
- Q 2. [[nodiscard]] 适合哪些场景?为什么错误码函数最应该加?**
- Q 3. [[fallthrough]] 解决什么问题?不写它编译器会怎样?**
- Q 4. __attribute__ 与 [[...]] 有何关系?跨平台代码为什么要封装宏?**
- Q 5. 属性报警被忽略的后果是什么?工程上如何让属性检查真正生效?**

✓ 参考答案

- Java/C# 注解是运行时可查询的元数据,能驱动框架行为;C++ 属性是编译期的契约与提示,运行时不留下痕迹。C++ 的框架行为由模板与代码生成承担,属性只负责编译期检查与意图显式化。
- 所有"结果必须被处理"的函数:错误码、资源句柄、工厂函数、查询函数。错误码最该加,因为 C 时代漏查错误码是静默的、不可见的,加 nodiscard 后漏查直接编译报警,配合 -Werror 变成编译失败。
- switch 的 case 穿透默认被当作"忘记 break",编译器警告;[[fallthrough]] 声明穿透是有意设计,抑制警告并让读者明确看到意图。不写它会收到警告,写了又没穿透会收到另一个警告。
- [[...]] 是标准语法,__attribute__ 是 GCC/Clang 方言(MSVC 是 __declspec)。方言不可移植,直接裸用换编译器就失败;封装成宏后业务代码统一,换平台只改宏定义。
- 报警被忽略等于属性没写:漏检查的错误码、被遗忘的弃用接口都继续静默存在。工程上开 -Wall -Wextra -Werror(/W4 /WX),属性报警全部升级为编译失败,让检查不可绕过。

实验:安全协议库

本实验把本章属性用在真实场景:协议库的错误码全部 `nodiscard`,旧接口走 `deprecated` 过渡,协议头解析器用 `fallthrough` 表达多字节前缀,打包函数用 `packed` 结构体,日志参数用 `maybe_unused`,致命错误用 `noreturn`,最后用兼容宏保证三平台可编译。代码分三部分,覆盖点已在注释中标明。

实验 12•第一部分:契约属性(覆盖:`nodiscard`、`deprecated`、`noreturn`)

```
#include <iostream>
#include <string>
#include <cstdint>
#include <cstdlib>
#include <vector>

enum class ProtoErr { Ok, BadMagic, BadLength };

// 覆盖:nodiscard 带原因——协议处理结果必须被检查
[[nodiscard("必须检查协议错误")]]
ProtoErr parseHeader(const std::vector<std::uint8_t>& data) {
    if (data.size() < 9) return ProtoErr::BadLength;
    if (data[0] != 0x78 || data[1] != 0x56) return ProtoErr::BadMagic;
    return ProtoErr::Ok;
}

// 覆盖:deprecated——旧接口过渡,提示迁移
[[deprecated("改用 parseHeader()")]]
bool parseHeaderOld(const std::vector<std::uint8_t>& data) {
    return parseHeader(data) == ProtoErr::Ok;
}

// 覆盖:noreturn——致命错误直接终止
[[noreturn]] void protocolFatal(const std::string& why) {
    std::cerr << "协议致命错误: " << why << "\n";
    std::exit(2);
}
```

实验 12•第二部分:`packed` 与兼容宏(覆盖:`packed`、兼容宏、条件编译)

```

// 覆盖:兼容宏—Windows 用 pragma,Linux/macOS 用 attribute
#if defined(_MSC_VER)
#define CPP_PACK_BEGIN __pragma(pack(push, 1))
#define CPP_PACK_END __pragma(pack(pop))
#define CPP_PACKED_ATTR
#elif defined(__GNUC__) || defined(__clang__)
#define CPP_PACK_BEGIN
#define CPP_PACK_END
#define CPP_PACKED_ATTR __attribute__((packed))
#else
#define CPP_PACK_BEGIN
#define CPP_PACK_END
#define CPP_PACKED_ATTR
#endif

// 覆盖:packed 协议头,9 字节紧凑布局
CPP_PACK_BEGIN
struct Header {
    std::uint32_t magic; // 0x12345678 的网络序
    std::uint16_t type;
    std::uint16_t length;
    std::uint8_t flags;
} CPP_PACKED_ATTR;
CPP_PACK_END

// 覆盖:maybe_unused—日志级别在某些构建里用不到
void debugDump(const std::vector<std::uint8_t>& data,
               [[maybe_unused]] int verbose) {
#ifdef _DEBUG
    if (verbose > 0) {
        for (auto b : data) std::printf("%02X ", b);
        std::printf("\n");
    }
#endif
}

```

实验 12•第三部分:解析器与 main(覆盖:fallthrough、属性综合)

```

// 覆盖:fallthrough—类型字段的连续分支有意穿透
int typeGroup(std::uint16_t type) {
    switch (type) {
        case 0x0001:           // 请求
            return 1;
        case 0x0002:           // 响应
        case 0x0003:           // 通知
            [[fallthrough]];    // 覆盖:三种通知类统一归类
        case 0x0004:
            return 2;
        default:
            return 0;
    }
}

int main() {
    std::vector<std::uint8_t> good{0x78, 0x56, 0x34, 0x12,
                                   0x01, 0x00, 0x64, 0x00, 0x00};

    // 覆盖:nodiscard—检查结果不可丢弃
    ProtoErr r = parseHeader(good);
    if (r != ProtoErr::Ok) protocolFatal("头部校验失败");

    std::vector<std::uint8_t> bad{0x00, 0x00, 0x00, 0x00,
                                   0x01, 0x00, 0x64, 0x00, 0x00};

    r = parseHeader(bad);           // BadMagic
    std::cout << "坏包结果: "
               << (r == ProtoErr::BadMagic ? "魔数错误" : "其他") << "\n";

    Header h{0x12345678, 0x0002, 100, 0x01};
    std::cout << "包头大小 " << sizeof(h) << " 字节"
               << ",组别 " << typeGroup(h.type) << "\n";

    debugDump(good, 1);             // 覆盖:maybe_unused 参数
    // parseHeaderOld(good);         // 警告:已弃用接口
    return 0;
}

```

实验覆盖点核对:nodiscard(parseHeader 的错误码强制检查)、deprecated(parseHeaderOld 过渡)、noreturn(protocolFatal)、fallthrough(typeGroup 的穿透分组)、packed 与兼容宏(Header 三平台可编译)、maybe_unused(debugDump 参数)、属性综合应用。扩展练习:把 parseHeader 改成解析 Header 结构体并校验 magic 字段;或给 typeGroup 增加 [[unlikely]] 的 default 分支,对比开关警告。



工程建议 1:公共接口一次写对属性

库的公共 API 发布前过一遍属性清单:结果型函数 nodiscard,废弃接口 deprecated,平台细节 packed/导出宏。属性发布后补加会带来一波调用方编译警告,宁可写接口时一次做对,也不在 v2 里补课。



工程建议 2:把方言全部收进头文件宏

项目里建立一个 portability.h,集中定义 CPP_API、CPP_PACKED 等兼容宏,业务代码只 include 不裸用方言。换编译器、加平台时的改动收敛到一个文件,这是跨平台工程的最低成本方案。

本章小结

属性是 C++ 给编译器的契约语言:标准属性覆盖检查与意图(nodiscard、deprecated、fallthrough、maybe_unused、noreturn、likely),编译器扩展提供平台能力(packed、aligned、dllexport),兼容宏让方言可移植。配合严格编译选项,属性把"约定"变成"检查"。下一章进入模块系统:从 #include 到 C++20 模块。

🔗 本章目标

理解 C++ 的编译模型:翻译单元、头文件文本包含与链接;掌握 include guard 的两种写法与 #pragma once 的取舍;学会头文件与源文件的分离纪律、前置声明与 Pimpl;看懂 C++20 模块(modules)如何替代头文件,并能在新项目中动手使用 import/export。

📦 前置知识

熟悉函数、类、模板的声明与定义(第4、6、10章);了解链接错误(第1章);本章代码 C++17 可编译,C++20 模块部分需要 C++20 编译器(GCC 11+、Clang 16+、MSVC 2019 16.11+)。

§13.1 编译模型	§13.2 头文件纪律	§13.3 分离编译
§13.4 ODR 与多定义	§13.5 包含成本	§13.6 C++20 模块

§13.1 编译模型:#include 是文本粘贴

C++ 从 C 继承了头文件与源文件的二分法。一个 .cpp 文件连同它包含的所有头文件,展开成一个翻译单元(translation unit)。`#include` 的语义朴素得惊人:把另一个文件的内容原样粘进来。也就是说,"头文件"在 C++ 标准里并不存在,只有预处理后的文本;你看到的 `#include <iostream>` 在编译真正开始前,已经被替换成了上千行标准库代码。理解这一点,才能理解后面所有关于头文件的规则。

编译的产物是目标文件(object file),里面记录每个函数和变量的定义,以及"我用到但没定义"的符号引用。链接器负责把多个目标文件与库拼在一起:找到每个符号唯一的定义,填上引用。于是出现两类型错误:声明了却没人定义(链接错误 undefined reference),和同一个实体定义了多份(链接错误 multiple definition)。模块系统要解决的,就是这套文本粘贴模型带来的重复、泄漏与低效。

设计上要记住一个原则:头文件里放声明与内联/模板定义,源文件里放普通定义。声明是"对外承诺",可以重复出现多次而不冲突;定义是"唯一事实",在程序里只能有一份。

C 的写法	C++ 的写法	说明
<code>#include</code> 文本粘贴,无其他选择	头文件之外,C++20 有模块可选	模块提供语义级导入,替代文本展开
头文件内容对包含方全部可见	模块里未 <code>export</code> 的符号不可见	C++20 起可见性由语言保证
每次包含都重复解析文本	模块预编译一次,import 零重复开销	大型项目编译时间收益显著

设计上要记住一个原则:头文件里放声明与内联/模板定义,源文件里放普通定义。声明是"对外承诺",可以重复出现多次而不冲突;定义是"唯一事实",在程序里只能有一份。谁违反了这个分工,谁就制造链接错误或编译膨胀。接下来的小节把这条原则展开成可操作的纪律。

§13.2 头文件纪律:guard 与自包含

头文件被多个源文件包含,甚至被同一个源文件包含两次,是家常便饭。没有防护,同一个类会被定义两遍,编译直接失败。`include guard` 就是在头文件首尾设置的"开关":第一次进入时定义宏并贴内容,以后再来,宏已存在,整段跳过。经典写法用宏名(工程名+路径+随机串),另一种写法 `#pragma once` 由编译器直接识别"这个文件只处理一次",更简洁且没有宏名冲突问题,代价是不被旧标准保证、某些诡异场景(同一文件通过不同路径包含、符号链接)可能失效。

清单 13-1 include guard 两种写法

```
// 写法 A:经典宏 guard(一切编译器可用)
#ifndef CPP_TUTOR_MATH_UTILS_H_
#define CPP_TUTOR_MATH_UTILS_H_

int add(int a, int b);
double avg(const double* xs, int n);

#endif // CPP_TUTOR_MATH_UTILS_H_

// 写法 B:#pragma once(GCC/Clang/MSVC 都支持)
#pragma once

int mul(int a, int b);
```

现代工程几乎全部用 #pragma once:不用操心宏名唯一、少两行样板、可读性好。唯一推荐经典宏的场景是要兼容远古编译器,或者项目规范强制。但注意:#pragma once 是按"文件路径"去重的,如果同一内容被复制成两份不同文件,它拦不住;宏 guard 至少能通过同一个宏名拦住。工程上不必为此焦虑,二选一,保持一致即可。

更重要的纪律是"头文件自包含":任何头文件,单独 #include 它就应该能编译通过。检查方法:把每个头文件放进一个空的 .cpp 里包含并编译,不通过就修。自包含的收益是使用方不必记住"要先包含它的依赖",排序自由,依赖不泄漏。与自包含配套的是每个头文件只包含自己真正需要的东西,能用前置声明就不用 #include(详见 13.5),能不放定义就不放定义。

§13.3 分离纪律:声明与定义各就各位

头文件与源文件的经典分工:接口放 .h(类的成员声明、自由函数声明、常量、内联函数、模板定义),实现放 .cpp(成员函数定义、自由函数定义、静态变量)。使用者只看头文件就能调用,实现改动只重编译库本身,不牵连所有调用方——这是分离编译最大的工程价值:编译时间与依赖面都被压缩。

清单 13-2 头文件与源文件分离

```

// ---- math_utils.h ----
#pragma once
#include <vector>

// 类:成员声明在头文件
class Stats {
public:
    void add(double v);           // 声明
    double mean() const;         // 声明
private:
    std::vector<double> values_;
};

// 自由函数声明
double variance(const std::vector<double>& xs);

// 内联函数定义可以放头文件
inline double square(double x) { return x * x; }

// ---- math_utils.cpp ----
#include "math_utils.h"
#include <cmath>
#include <numeric>

void Stats::add(double v) {      // 定义:类外写,带类名限定
    values_.push_back(v);
}

double Stats::mean() const {
    return values_.empty() ? 0.0
        : std::accumulate(values_.begin(), values_.end(), 0.0)
            / static_cast<double>(values_.size());
}

double variance(const std::vector<double>& xs) {
    if (xs.empty()) return 0.0;
    Stats s;
    for (double x : xs) s.add(x);
    double m = s.mean(), sum = 0.0;
    for (double x : xs) sum += (x - m) * (x - m);
    return sum / static_cast<double>(xs.size());
}

```

哪些东西必须放进头文件:类定义(成员声明+内联实现)、函数模板与类模板的完整定义、inline 函数、constexpr 函数、inline 变量、以及 enum class。它们的共同点是编译器在调用点必须看到完整定义才能生成代码,或语言允许它们有多份相同定义(内联与模板的"多份相同可接受"规则见 13.4)。普通函数、全局变量的定义则一律放 .cpp,只把声明留在头文件。

还有两个细分技巧。其一,头文件里尽量少放 #include,能用前置声明就用;这样头文件的依赖变薄,包含它的文件不会被连锁拖累。其二,类的数据成员不变的前提下,头文件应该"常年稳定":公共头文件一改,全项目重编译,所以把实现细节藏进 .cpp 是贵族级素养——最极致的形式是 Pimpl(pointer to implementation)。

清单 13-3 Pimpl:把实现藏进源文件

```
// ---- widget.h:只暴露稳定接口 ----
#pragma once
#include <memory>

class Widget {
public:
    Widget(); // 析构等特殊成员必须在 cpp 定义
    ~Widget();
    Widget(Widget&&) noexcept;
    Widget& operator=(Widget&&) noexcept;
    Widget(const Widget&) = delete; // 复制代价大,直接禁掉
    Widget& operator=(const Widget&) = delete;

    void setText(const char* text);
    const char* text() const;
private:
    struct Impl; // 前向声明,不泄露任何成员
    std::unique_ptr<Impl> p_; // 智能指针持有不完整类型
};

// ---- widget.cpp ----
#include "widget.h"
#include <string>

struct Widget::Impl { // 真正的成员都在这里
    std::string text = "空";
    int version = 1;
};

Widget::Widget() : p_(std::make_unique<Impl>()) {}
Widget::~~Widget() = default; // 这里才需要完整类型
Widget::Widget(Widget&&) noexcept = default;
Widget& Widget::operator=(Widget&&) noexcept = default;

void Widget::setText(const char* t) { p_->text = t ? t : ""; }
const char* Widget::text() const { return p_->text.c_str(); }
```

C 的写法	C++ 的写法	说明
.c 文件里 extern 声明共享变量	头文件 extern 声明 + .cpp 定义, 或 inline 变量	C++17 起头文件可安全放共享变量
泛型靠宏与 void*	模板定义必须放头文件(或模块)	模板是 C++ 头文件文化的最大来源
头文件里的宏污染无法根除	constexpr、inline、命名空间替代宏	头文件里的 #define 是最难查的 bug 源之一

Pimpl 的收益:widget.h 里没有任何标准库头文件依赖,调用方编译飞快;Impl 里加成员、加头文件,使用方零感知。代价是每次访问成员多一层指针解引用、对象分配堆内存、移动语义要自己维护。它适合"被广泛包含且长期稳定"的库类型,不适合性能热点上的小对象。C++20 模块出现后,Pimpl 的地位有所松动,但存量代码里它仍是主力武器。

§13.4 ODR:为什么"多定义"有时合法

链接器发现同一个符号有两份定义时报错,但内联函数、模板、类定义明明会被每个包含它们的翻译单元各生成一份——为什么它们不冲突?这靠的是 ODR(单一定义规则)的例外条款:如果这些实体在多处定义,只要每处定义"完全相同"(token 级一致),就视为同一实体,链接器取一份即可。这是头文件模型能成立的理论基石:头文件内容在所有包含点相同,于是类与模板的重复定义被豁免。

ODR 的两个实操教训。第一,内联函数与模板的"完全相同"要求很脆:不同翻译单元里宏定义不同,展开出不同的函数体,就违反了 ODR,行为未定义,而且经常编译期不报错、运行期诡异。排查办法:禁止在头文件里依赖条件编译改变接口含义。第二,普通函数与全局变量没有豁免:定义放 .cpp,头文件里只放声明,否则换来的就是 multiple definition 链接错误。

清单 13-4 违反 ODR 的典型现场

```
// ---- config.h(错误示范!)----
#pragma once
#include <string>

// 问题 1:非内联函数定义放头文件,两个 cpp 各展开一份
// std::string version() { return "1.0"; } // 链接错误

// 问题 2:全局变量定义放头文件
// int g_debugLevel = 2; // 链接错误

// 正确做法:声明放头文件
std::string version();
extern int g_debugLevel;

// ---- config.cpp ----
#include "config.h"
std::string version() { return "1.0"; }
int g_debugLevel = 2;

// ---- main.cpp ----
#include "config.h"
#include <iostream>
int main() {
    g_debugLevel = 3; // 用 extern 声明访问全局变量
    std::cout << version() << "\n";
    return 0;
}
```

C++17 给"头文件里想放全局变量"开了一个口子:inline 变量。inline 变量的定义可以出现在多个翻译单元且取同一份存储,常用于头文件里的配置常量、单例指针等场景。它的行为与函数模板的"多份相同"一致,是安全的。但别滥用:能用函数调用封装的状态,优先用函数(局部静态变量),可读性更好,初始化顺序问题也更少。

§13.5 包含成本与循环依赖

头文件模型最大的隐形成本是编译时间。每个 #include 都是文本粘贴:一个头文件的 #include 会拉进它的传递依赖,一行头文件改动引发全项目级联重编译。百万行 C++ 项目的编译时间以十分钟计,头文件贡献了绝大部分。缓解手段

按性价比排序:前置声明代替 #include;头文件瘦身(少包含、少放实现);Pimpl;统一编译头(precompiled header)把高频稳定头一次编译缓存;再就是迁移到 C++20 模块。

前置声明(forward declaration)是只声明"存在这么个类",不展开它的定义:class Logger;。代价是不能定义该类型的对象、不能调用成员、不能 delete 指针——只能声明指针/引用、用作函数参数与返回类型。对"互相引用"的类,前置声明是唯一的解药:A 有 B 的指针、B 有 A 的指针,两个头文件互相 #include 就是死循环,guard 虽然能阻止编译崩溃,却让其中一方在包含点看不到完整类型而报错。正确结构是:各自只前置声明对方,把需要完整类型的实现放进 .cpp,在 .cpp 里 #include 对方头文件。

清单 13-5 前置声明解开循环依赖

```
// ---- a.h ----
#pragma once
class B;                // 前置声明:只知道存在 B

class A {
public:
    void greet(const B& b) const;    // 引用参数,不完整类型就够
    B* partner() const { return partner_; }
private:
    B* partner_ = nullptr;    // 指针,不完整类型就够
};

// ---- b.h ----
#pragma once
class A;

class B {
public:
    void greet(const A& a) const;
};

// ---- a.cpp ----
#include "a.h"
#include "b.h"            // 实现里才需要完整类型
#include <iostream>

void A::greet(const B& b) const {
    std::cout << "A 对 B 说:你好\n";
    b.greet(*this);
}

// ---- b.cpp ----
#include "b.h"
#include "a.h"
#include <iostream>

void B::greet(const A& a) const {
    std::cout << "B 对 A 说:你好\n";
    (void)a;
}
```

包含顺序也是一门纪律:头文件之间相互包含的次序不稳定是毒药,工程标准做法是"每个头文件自包含 + 包含者先包含自己的头文件再包含标准库",把顺序敏感性连根拔掉。头文件里的宏污染要警惕:宏是全局的文本替换,一个头文件里

#define 的宏可能改写到处使用方代码里的同名标识符。这也是 C++ 讨厌宏、头文件尽量少用宏的又一个理由。

§13.6 C++20 模块:告别文本粘贴

模块(modules)是 C++20 对头文件模型的根本替代。模块文件(.cppm 等)内部声明 import 进来的依赖、export 出去的接口;使用方写 import math_utils; 即可,编译器把模块编译成带语义的二进制(带 GCC/Clang/MSVC 各自的实现格式),使用方直接读取语义,不再展开文本。好处是本质性的:接口与实现分离由语言保证(没 export 的符号对使用方不可见,宏更是无从泄漏);编译时间大幅下降(模块只编译一次,不随使用方数量翻倍);编译依赖顺序友好(无循环包含问题)。

清单 13-6 C++20 模块最小示例

```
// ---- math_utils.cppm(模块单元)----
export module math_utils;           // 导出模块名
import <vector>                      // 导入标准库头(模块化兼容)

export class Stats {                // export 的才对外可见
public:
    void add(double v) { values_.push_back(v); }
    double mean() const;
private:
    std::vector<double> values_;
};

export double variance(const std::vector<double>& xs);

// 未 export 的实现细节:对使用方不可见
double Stats::mean() const {
    double sum = 0;
    for (double v : values_) sum += v;
    return values_.empty() ? 0.0 : sum / values_.size();
}

double variance(const std::vector<double>& xs) {
    Stats s;
    for (double v : xs) s.add(v);
    return s.mean();
}

// ---- main.cpp ----
import math_utils;                  // 使用方:一行 import
import <iostream>

int main() {
    Stats s;
    s.add(1); s.add(2); s.add(6);
    std::cout << "平均: " << s.mean() << "\n";
    return 0;
}

// 编译(以 GCC 为例,先编模块再编主程序):
// g++ -std=c++20 -fmodules-ts math_utils.cppm main.cpp -o app
```

模块与头文件的关键差异要记住:一是可见性,模块里没 export 的一切(实现函数、辅助类型、宏)对使用方完全不可见,头文件模型里这些全在公共视界;二是包含成本,import 是读取预编译好的语义信息,与文本大小无关,重复 import 也不

重复开销;三是构建顺序,模块必须先于使用方编译,构建系统需要了解模块依赖图,而头文件只需要文件存在。四是兼容性,import <iostream> 这类"模块化头"是标准库的福利,第三方老库只能 import "xxx.h" 用头文件模式过渡。

迁移策略:新项目、新库优先用模块;存量大型项目按依赖自底向上把最稳定、最被广泛包含的底层库先迁(收益最大),边界处用 import "old.h" 过渡;编译命令与构建脚本要跟上(CMake 的 C++20 模块支持在 3.28+ 已可用)。模板在模块里不再需要"定义必须在调用处可见"——模块内模板定义会被编进模块二进制,使用方 import 即可实例化,这解决了头文件模型最大的痛点。工具链对模块的报错信息、IDE 支持尚在追赶,这是 2020 年代 C++ 工程的现实。

C 的写法	C++ 的写法	说明
#ifndef 宏 guard 是唯一选择	#pragma once(或 C++20 模块)	C 无标准替代,只能宏 guard;C++ 有编译器级与语言级方案
头文件里无法声明"别的东西不可见"	模块里未 export 的符号天然隐藏	C 的 #include 是全文可见,模块有可见性边界
包含顺序与宏污染全凭自觉	import 无顺序问题、无宏泄漏	模块从机制上消灭两类头文件顽疾

⚠ 易错点 1:guard 宏名重复,头文件被吞

两个毫不相干的头文件若写了同一个 guard 宏名(常见于复制粘贴),谁先被包含,谁就永久屏蔽另一个:第二个头文件的内容整个被跳过,缺声明,编译报"未定义"或类未声明的错,且报错位置远离源头。guard 宏名要带上项目与路径特征(如 CPP_TUTOR_MATH_UTILS_H),或干脆用 #pragma once。

⚠ 易错点 2:头文件放非内联定义

把函数定义、全局变量定义写进头文件,每个包含它的 .cpp 都会生成一份,链接时报 multiple definition。新手最常见:定义写在头文件里恰好单文件程序能跑,一旦多文件立刻炸。守则:普通函数与全局变量只声明进头文件;定义交给 .cpp。类内定义的成员函数、模板、inline 函数才允许(且必须)在头文件。

⚠ 易错点 3:模板定义放在 .cpp,链接失败

模板实例化发生在使用点:调用方编译时若看不到模板定义,只看到声明,会生成"未定义引用"链接错误。把模板定义塞进 .cpp 只编译了 .cpp 里用到的实例,使用方仍然找不到。模板要么整体放头文件,要么在 .cpp 末尾显式实例化(export template),要么用模块。头文件里写模板是 C++ 的常态,不是妥协。

快速参考:头文件与模块速查表

场景	做法
头文件防重复包含	#pragma once(兼容旧编译器用宏 guard)
头文件自包含检查	每个头文件单独包含进空 cpp 编译
类成员声明	头文件(定义在 .cpp 类外写)
类内实现的小函数	头文件类内定义,隐式内联
模板定义	头文件(或模块)
普通函数定义	.cpp,头文件只留声明
全局变量定义	.cpp + extern 声明;或头文件 inline 变量
互相引用的类	各自前置声明,实现放 .cpp
隐藏实现细节	Pimpl(unique_ptr + 不完整类型)
减少包含成本	前置声明、头文件瘦身、PCH、模块
C++20 模块导出接口	export module / export 声明
模块使用	import 模块名;编译时先编模块单元
ODR 豁免实体	inline 函数/变量、模板、类定义(内容须相同)

自测题

Q 1. #include 在预处理阶段做了什么?为什么说头文件"文本粘贴"模型决定了后面所有规则?

✓ 答案

把被包含文件的内容原样展开进当前翻译单元。编译从展开后的文本开始,因此头文件内容在所有包含点展开后必须一致(否则违反 ODR),头文件内容全部对包含方可见(因此要自己管 guard、管可见性),每次包含都要重新解析文本(因此有编译成本)。

Q 2. 一个头文件既被 a.cpp 又被 b.cpp 包含,里面有普通函数定义,链接会发生什么?为什么模板定义却没事?

✓ 答案

普通函数定义没有 ODR 豁免,两份不同翻译单元的定义产生 multiple definition 链接错误。模板定义允许"多份相同定义"(内容 token 级一致即视为同一实体),由链接器合并,所以模板定义放头文件是安全的。

Q 3. 前置声明 class B; 之后,哪些操作合法,哪些非法?

✓ 答案

合法:声明 B 的指针/引用、作为函数声明参数与返回类型。非法:定义 B 的对象(B 的大小未知)、访问 B 的成员、对 B* 做 delete(析构未知)、继承 B、用 sizeof(B)。需要这些操作的代码必须移进 .cpp 并包含完整头文件。

Q 4. Pimpl 技法把什么放进了 .cpp?它的主要收益与代价是什么?

✅ 答案

把类的全部数据成员与实现细节放进嵌套类 Impl,头文件只剩前向声明与 unique_ptr 成员。收益:头文件零依赖,调用方编译快,实现改动不影响使用方重编译。代价:每次访问多一层指针间接、对象在堆上、移动/析构特殊成员要在 .cpp 里定义,小对象性能敏感时慎用。

Q 5. C++20 模块相比头文件,解决了哪三类本质问题?

✅ 答案

可见性:未 export 的符号与宏对使用方不可见;成本:模块预编译一次、import 按语义读取,重复包含无开销,编译时间大幅下降;健壮性:无包含顺序问题、无循环依赖、无宏污染,模板定义也随模块分发。

实验:一个头文件组织的日志库

本实验写一个日志库,把本章全部知识点落地:Log.h 用 #pragma once、自包含、只放声明与 inline 函数;Log.cpp 放实现;main.cpp 使用它;再演示模板声明与定义分离、inline 变量、以及 Pimpl 的变体(日志实现细节藏在 .cpp 的内部匿名命名空间)。运行后应输出 INFO/ERROR 各一行到控制台与 log.txt。

实验:Log.h 头文件

```
// ---- Log.h ----
#pragma once                // 覆盖:include guard
#include <string>             // 自包含:自己包含自己需要的头

namespace tut {              // 命名空间收纳,避免污染全局

// inline 变量:多翻译单元共享一份存储
inline int g_verbose = 1;    // 覆盖:inline 变量

enum class Level { Info, Warn, Error };

// 模板:必须整体放头文件
template <typename T>
inline std::string fmt(const T& v) {
    return std::to_string(v); // 覆盖:模板定义在头文件
}

class Logger {               // 声明在头文件
public:
    Logger();                 // Pimpl 风格:构造在 cpp 完成
    ~Logger();
    Logger(const Logger&) = delete;
    Logger& operator=(const Logger&) = delete;
    void log(Level lv, const std::string& msg);
    void setFile(const std::string& path);
private:
    struct Impl;              // 覆盖:前置声明
    Impl* p_;                 // 实现细节在 cpp
};

} // namespace tut
```

实验:Log.cpp 实现

```
// ---- Log.cpp ----
#include "Log.h"
#include <fstream>
#include <iostream>
#include <mutex>

namespace tut {

struct Logger::Impl {          // 覆盖:定义藏在 cpp,不泄露
    std::ofstream file;
    std::mutex mtx;
};

Logger::Logger() : p_(new Impl()) {}    // 覆盖:构造分配实现
Logger::~Logger() { delete p_; }        // 覆盖:析构释放

void Logger::setFile(const std::string& path) {
    p_>file.open(path, std::ios::app);
}

void Logger::log(Level lv, const std::string& msg) {
    const char* tag = (lv == Level::Info) ? "INFO"
                    : (lv == Level::Warn) ? "WARN" : "ERROR";
    std::lock_guard<std::mutex> lock(p_>mtx); // 覆盖:并发安全
    std::string line = std::string(tag) + ": " + msg;
    std::cout << line << "\n";
    if (p_>file) p_>file << line << "\n";
}

} // namespace tut
```

实验:main.cpp 使用

```
// ---- main.cpp ----
#include "Log.h"
#include <string>

int main() {
    tut::Logger logger;                // 覆盖:分离编译的使用方
    logger.setFile("log.txt");

    logger.log(tut::Level::Info, "启动, 版本=" + tut::fmt(3));
    tut::g_verbose = 0;                // 覆盖:inline 变量跨单元共享
    logger.log(tut::Level::Error, "磁盘空间不足");

    // 模板在头文件,使用方实例化 std::string 版本
    std::string s = tut::fmt(std::string("字符串"));
    (void)s;
    return 0;
}

// 编译(Windows 命令行 / Linux 同理):
// g++ Log.cpp main.cpp -std=c++17 -o logger_app
// 运行后控制台与 log.txt 各出现 INFO 与 ERROR 两行
```

工程实践 1:头文件也是接口,评审它

头文件是 C++ 的公共接口层,评审代码时先看头文件:依赖是否最少、是否自包含、是否只暴露必要的 API、注释是否说明前置条件与线程安全。接口稳定的库,头文件一年不动,团队编译速度就赢了一半。

工程实践 2:增量迁移到模块

别等"全项目迁移"的完美时机:把最稳定、被包含最广的底层库先迁成模块,收益立刻显现;旧代码 import "老.h" 照常工作。同时把构建脚本的模块支持打开(CMake 3.28+、Bazel 6+),新文件一律用模块写,两年后回头看,头文件只剩一小撮。

本章总结

头文件是 C 时代的文本粘贴机制,理解翻译单元与链接才能写出正确的多文件工程:guard 防重复、自包含保稳定、声明与定义分居头/源、模板与 inline 靠 ODR 豁免留在头文件、前置声明与 Pimpl 控制依赖与编译成本。C++20 模块用 import/export 取代 #include,从语言层面解决可见性、编译时间与宏污染,是头文件体系的接班人。无论是维护老工程还是开新项目,模块系统的知识都直接决定编译体验与接口卫生。

🎯 本章目标

完整理解 C++ 的类型系统:隐式转换的规则与陷阱、四种 C++ 风格转换(static_cast、dynamic_cast、const_cast、reinterpret_cast)的适用场景、auto 与 decltype 的推导规则、type_traits 类型萃取与 if constexpr 的组合、以及把类型安全落地的工程纪律。

📦 前置知识

熟悉基本类型与指针(第1章)、运算符(第2章)、类与多态(第6章)、模板(第10章)。本章代码基于 C++17,type_traits 与 if constexpr 为 C++17 特性。

§14.1 类型全景	§14.2 隐式转换	§14.3 四种显式转换
§14.4 auto 与 decltype	§14.5 类型萃取	§14.6 类型安全实践

§14.1 类型全景:编译器知道得比你多

C++ 里每一个表达式都有静态类型,编译器在编译期就"知道"它是什么:是 int 还是 const char*,是引用还是指针,是基类还是派生类。这个知识是类型系统给你的一切能力的来源:重载决议靠它、模板实例化靠它、运算符语义靠它、编译错误报告也靠它。C++ 的类型检查不是运行时行为——编译通过的类型正确代码,类型错误在运行前就被拦截了。理解类型系统的第一课:类型不是注释,是约束。

类型谱系要先理清几个维度。按性质分:对象类型(算术、指针、数组、类、枚举)、引用类型(左值引用、右值引用)、函数类型与 void。按限定分:const 限定与 volatile 限定。按复合方式分:数组、指针、引用、成员指针。多态类型是含虚函数的类,它们是 dynamic_cast 与 typeid 的适用对象。这条谱系决定了每个类型能参与什么操作:数组名会衰减成指针、函数名会衰减成函数指针、const 对象不能调用非 const 成员——全是类型系统在工作。

还有价值类别(value category)一维:表达式分为左值(有名字、可取地址、可持久)与右值(临时值、即将消亡)。C++11 的右值又细分为纯右值(x + 1 的临时结果)与将亡值(std::move(x) 的结果)。价值类别决定移动语义能不能生效:move 返回将亡值,所以能触发移动构造。写代码时不用背全套理论,记住一句:左值有身份,右值是临时的;std::move 不移动任何东西,它只是把表达式转成将亡值,让重载决议选到移动路径。

§14.2 隐式转换:编译器替你做的类型体操

隐式转换(标准转换)是编译器自动执行、不需要写任何语法的转换。最日常的一档是整型提升:short、char、bool 参与运算时提升为 int,这是"从小到大更大且不丢值"的安全转换。第二档是整型转换:int 转 long、long 转 int,方向任意,可能丢值,但编译器默默允许——只在花括号初始化里被禁止(窄化检查)。第三档是浮点转换:float 转 double 安全,double 转 float 可能丢精度。第四档是算术转换:两个不同算术类型做运算时,编译器找"公共类型"(如 int 与 double 运算,结果类型是 double),规则表复杂且常有直觉意外。

清单 14-1 隐式转换的现场

```

#include <iostream>
#include <cstdint>

int main() {
    short s = 1000;
    int i = s;           // 整型提升:short -> int,安全
    long long ll = i;    // 整型转换:安全方向

    unsigned u = 4000000000u;    // 4 十亿
    int n = static_cast<int>(u);  // 有符号/无符号混用前先想清楚
    std::cout << "u=" << u << ", n=" << n << "\n"; // n 溢出为负

    double d = 3.14159;
    int truncated = d;           // double -> int 截断,丢弃小数
    std::cout << "truncated=" << truncated << "\n";

    int big = 2000000000;
    // 窄化警告(花括号初始化禁止):
    // char c = {big};           // 错误:窄化转换
    char c = static_cast<char>(big);
    std::cout << "char 包装: " << (int)c << "\n";

    // 经典陷阱:无符号与有符号比较
    int neg = -1;
    unsigned pos = 1;
    if (neg < pos) {             // 负转成巨大无符号数,比较结果反直觉
        std::cout << "neg < pos 为真\n";
    } else {
        std::cout << "neg < pos 为假(陷阱触发!)\n";
    }
    return 0;
}

```

隐式转换的第二大类是类类型转换:单参数构造函数充当"从别的类型构造本类型"的转换通道,转换运算符(operator double() 之类)充当"本类型转成别的类型"的通道。两者叠加,语言允许一串隐式转换自动发生——这正是陷阱所在:意外转换、歧义重载、难以追踪的类型体操。C++11 的 explicit 关键字就是给这些转换通道上锁:explicit 构造函数与 explicit 转换运算符只允许显式转换(static_cast、直接初始化),禁止隐式发生。

工程纪律:数值边界转换要么显式写出 static_cast,要么开 -Wconversion 让隐式窄化报警;有符号与无符号的混用比较几乎全是 bug 温床,比较前统一转型;类类型转换一律 explicit,需要隐式行为时深思熟虑再放开。隐式转换是便利,更是地雷,默认关掉它。

C 的写法	C++ 的写法	说明
隐式转换一律静默发生	花括号初始化 + -Wconversion 拦截窄化	C++ 给"丢值"的转换加了编译期刹车
enum 常量隐式转 int,泄漏进全局	enum class 不隐式转换,作用域封闭	C++11 起枚举是真正的强类型
单参构造函数与转换无开关	explicit 锁住隐式类转换	隐式转换通道默认关闭,意图明确

⚠ 易错点 1:无符号下标的递减陷阱

for (std::size_t i = v.size() - 1; i >= 0; --i) 是教科书级陷阱:i 减到 0 后再减一次,绕回 size_t 的最大值,条件 i >= 0 永远为真,循环永不停,v 为空时 v.size() - 1 直接就是最大值,更是隐蔽。修法:用反向迭代器(rbegin/rend)、用后置递减惯用法 for (size_t i = n; i-- > 0;),或显式转成有符号再递减。

§14.3 四种显式转换:各司其职的 cast

显式转换(强制转换)是程序员主动要求的类型变换。C++ 提供四种风格化 cast,取代了 C 的万能转型(类型)表达式。static_cast 是"合理的显式转换":数值类型互转、基类指针/引用与派生类指针/引用互转(向下转换不做运行时检查,是契约型)、void* 与具体指针互转。它是默认选择,能做 90% 的活。reinterpret_cast 是"位级重解释":把一种指针的位模式直接当另一种指针用,不做任何检查与调整,是最后手段——真实用途只有底层编解码、与 C 库对接等极少数场景。

清单 14-2 四种 cast 的分工

```
#include <iostream>
#include <cstring>
#include <cstdint>

int main() {
    // static_cast:数值与类型的合理转换
    double d = 3.75;
    int n = static_cast<int>(d);          // 截断 3
    std::cout << n << "\n";

    // reinterpret_cast:位重解释(危险,极少用)
    std::uint64_t bits = 0x3ff000000000000ull; // 1.0 的 IEEE 754 位模式
    double x = *reinterpret_cast<double*>(&bits);
    std::cout << "位模式解释为 double: " << x << "\n";
    // 安全做法(别名规则友好):
    double y;
    std::memcpy(&y, &bits, sizeof(bits));
    std::cout << "memcpy 解释: " << y << "\n";

    // const_cast:去掉 const(只用于修正"误标 const"的接口)
    const int ci = 42;
    int* p = const_cast<int*>(&ci);
    // *p = 7;                      // 危险:写真 const 对象是未定义行为

    // 类型别名双关用 memcpy,别用 reinterpret_cast(严格别名规则)
    float f = 1.5f;
    std::uint32_t fb;
    std::memcpy(&fb, &f, sizeof(f));
    std::cout << "float 位模式: 0x" << std::hex << fb << std::dec << "\n";
    return 0;
}
```

const_cast 是四个 cast 里最需要警惕的:它去掉对象的 const 限定。如果被去 const 的对象本身是"真 const"(声明为 const 的变量),写入它是未定义行为;如果它只是"经 const 接口传递"的普通对象,去掉 const 写入则是合法的。它的正当用途只有两种:调用一个错误标成非 const 的旧库函数,以及 const 成员函数内共享可变状态的底层细节(这种情况现代代码应该用 mutable 成员)。日常业务代码里出现 const_cast,先假定是设计问题。

`dynamic_cast` 是唯一做运行时检查的 `cast`:沿虚继承链把基类指针/引用转回派生类,转不动返回 `nullptr`(指针版本)或抛 `std::bad_cast`(引用版本)。它依赖 RTTI,只能用于多态类型。工厂函数返回基类指针、调用方需要特定派生接口时,它是正解;但如果代码里 `dynamic_cast` 满屏都是,说明设计在退化——应该靠虚函数与访问者模式消化分支。C 语言完全没有对应物,这正是 C++ 运行时多态语言特性的延伸。

清单 14-3 `dynamic_cast` 的运行时检查

```
#include <iostream>
#include <memory>
#include <string>
#include <vector>

class Shape {                                // 多态基类:有虚函数
public:
    virtual ~Shape() = default;
    virtual double area() const = 0;
};

class Circle : public Shape {
public:
    double r;
    explicit Circle(double radius) : r(radius) {}
    double area() const override { return 3.14159 * r * r; }
};

class Rect : public Shape {
public:
    double w, h;
    Rect(double width, double height) : w(width), h(height) {}
    double area() const override { return w * h; }
};

int main() {
    std::vector<std::unique_ptr<Shape>> shapes;
    shapes.push_back(std::make_unique<Circle>(1.0));
    shapes.push_back(std::make_unique<Rect>(2.0, 3.0));

    for (const auto& s : shapes) {
        // 运行时检查:真是 Circle 才转成功
        if (const auto* c = dynamic_cast<const Circle*>(s.get())) {
            std::cout << "圆, 半径 " << c->r
                        << ", 面积 " << s->area() << "\n";
        } else {
            std::cout << "其他图形, 面积 " << s->area() << "\n";
        }
    }
    return 0;
}
```

C 的写法	C++ 的写法	说明
(int)x 万能转型,无检查	static_cast<int>(x) 语义明确	C++ 四种 cast 各管一摊,转型意图可见
无运行时类型检查	dynamic_cast 运行时验证	多态向下转换 C 做不到,只能手工字段判定
类型双关靠指针强转	reinterpret_cast + memcpy	C 的强转常违反严格别名规则,是 UB 源

⚠ 易错点 2:reinterpret_cast 的类型双关是未定义行为

把 float 的位模式 reinterpret_cast 成 int 读,"看着能跑",实则是严格别名规则的违反:优化开启后编译器按"类型不同就不会互相影响"重排代码,结果未定义。安全做法是 std::memcpy 或 C++20 的 std::bit_cast。同理,把较小对象强转成较大指针再解引用是对齐错误,ARM 上直接崩溃。reinterpret_cast 前后都要想清楚"位布局依据"是什么。

§14.4 auto 与 decltype:让编译器写类型

auto 让编译器从初始化表达式推导变量类型。规则第一条:auto 推导值类型,const 与引用被剥掉(auto x = 常量引用对象,x 是副本);想要它们,写 const auto、auto&、const auto&、auto&&。第二条:auto&& 对左值实参推导为左值引用,对右值实参推导为右值引用——这是万能引用的基础(第10章提过)。第三条:auto 的推导发生在编译期,和模板推导同一套规则,因此 auto 变量必须有初始化器,初始化器决定一切。auto 的好处不是"少打字",而是"类型永不漂移":改接口返回类型,用 auto 的地方自动跟随。

decltype 是另一个方向:直接取出表达式或实体的声明类型。规则要记两条:decltype(实体名) 得到实体本身的声明类型(带引用与 const);decltype(表达式) 得到表达式结果类型,而表达式是左值时额外加引用(decltype(x)) 与 decltype(x) 不同——括号让 x 变成表达式)。decltype(auto) 让函数返回类型的推导完全按 decltype 规则进行,把"完美转发"写进返回类型(第10章的转发函数常以此结尾)。C++14 起函数返回类型可以裸写 auto 让编译器推导,但对外接口建议显式写出返回类型:接口要自文档化,推导适合内部辅助函数。

清单 14-4 auto 与 decltype 推导规则

```

#include <iostream>
#include <vector>
#include <string>

template <typename T>
decltype(auto) first(std::vector<T>& v) {
    return v.front();           // 返回 T&,不是副本
}

int main() {
    std::vector<std::string> names{"张三", "李四"};

    auto a = names[0];           // a 是 std::string 副本
    auto& ar = names[0];         // ar 是引用,改动影响容器
    const auto& cr = names[0];   // 常引用,只读不拷贝
    ar = "王五";
    std::cout << names[0] << "\n"; // 王五

    decltype(names[0]) d = names[1]; // d 与 names[0] 同型:引用
    d = "赵六";                      // 改的是 names[1]
    std::cout << names[1] << "\n";

    auto& f = first(names);         // decltype(auto) 返回引用
    f = "孙七";
    std::cout << names[0] << "\n";

    // decltype((x)) 是表达式,带引用;decltype(x) 是实体名,原样
    int x = 5;
    static_assert(std::is_same_v<decltype(x), int>);
    static_assert(std::is_same_v<decltype((x)), int&>);
    return 0;
}

```

结构化绑定(C++17)是 `decltype` 思路的日常应用:`auto [k, v] = map` 把 `pair`/元组/数组的结构拆开绑定,让"取类型"的样板消失。注意绑定的生命周期与副本语义:`auto [k, v]` 默认复制,`auto& [k, v]` 才是引用。有了 `auto`、`decltype`、结构化绑定,显式写类型只在需要控制存储与接口时出现——这正是现代 C++ 的"类型由编译器保证,人只写意图"的写法。

⚠ 易错点 3:auto 剥引用,改动"失效"

`auto x = 某引用`; 拿到的是副本,之后对 `x` 的修改不反映到原对象——比显式写类型时更隐蔽,读代码的人看不出这是副本。数组更是重灾区:`auto a = arr`; 直接把数组退化成指针,失去数组语义与 `sizeof` 信息。修法:想写原对象就用 `auto&` 或 `const auto&`,想保留数组语义用 `auto& a = arr` 绑定引用。

§14.5 类型萃取:把类型当数据用

`type_traits` 头文件提供了一整套编译期类型查询与变换工具,它们全部在编译期求值,零运行时成本。查询类:`is_same`、`is_integral`、`is_floating_point`、`is_class`、`is_pointer`、`is_reference`、`is_const`、`is_convertible`、`is_base_of`。变换类:`remove_reference`、`remove_const`、`decay`(剥掉引用与 `const`、数组转指针、函数转函数指针)、`conditional`(编译期三元)、`common_type`(找公共类型)。C++17 起每个 trait 都有 `_v` 后缀变量模板(`is_same_v<T,U>`),写法更短;旧代码用 `::value` 加 `typename` 也常见。

清单 14-5 类型萃取实用组合

```
#include <iostream>
#include <string>
#include <type_traits>
#include <vector>

// 编译期断言:类型关系是"硬契约"
static_assert(std::is_integral_v<int>, "int 必须是整型");
static_assert(!std::is_integral_v<double>, "double 不是整型");
static_assert(std::is_base_of_v<std::string, std::string>);

// decay:剥掉引用/const,数组转指针
template <typename T>
void showType() {
    using U = std::decay_t<T>;
    if (std::is_integral_v<U>)        std::cout << "整型\n";
    else if (std::is_floating_point_v<U>) std::cout << "浮点\n";
    else if (std::is_pointer_v<U>)    std::cout << "指针\n";
    else                             std::cout << "其他\n";
}

// conditional:编译期二选一
template <typename T>
using Storage = std::conditional_t<
    (sizeof(T) <= 8),                // 小对象用值,大对象用指针
    T,
    const T*>;

int main() {
    int arr[10];
    showType<decltype(arr)>();        // 数组 -> decay 成 int* -> 指针
    showType<const int&>();           // 引用/const 剥掉 -> 整型
    showType<double>();              // 浮点

    Storage<int> s1 = 3;              // 值存储
    std::string big = "大对象";
    Storage<std::string> s2 = &big; // 指针存储
    std::cout << s1 << " " << *s2 << "\n";
    return 0;
}
```

萃取与 if constexpr 的组合是 C++17 之后的"编译期派发"主流:if constexpr (std::is_integral_v<T>) { 整数版本 } else { 通用版本 },不满足的分支连编译都不参与,替代了第10章的 enable_if/SFINAE 大部分场景。constexpr if 让类型分支的代码可读性直线上升,是元编程从"技巧"走向"日常"的关键一步。C++20 的 concepts 更进一步,把"类型必须满足什么约束"写成名断言:template <std::integral T> 直接表达意图,编译错误信息也清晰得多。这三个工具(萃取、if constexpr、concepts)构成了现代 C++ 的类型驱动编程语言。

C 的写法	C++ 的写法	说明
sizeof 与 _Generic(C11) 有限判断	is_integral、is_same 等全套萃取	C++ 的编译期类型查询是图灵完备的
类型分支靠宏与运行时 if	if constexpr + type_traits	不满足的分支不生成代码,零成本
数组/函数类型几乎不可操作	decay、remove_reference 精确变换	类型变换是可组合的编译期函数

§14.6 类型安全实践:让错误在编译期显形

类型系统的终极价值是让"不可能的状态"无法表达。工程上把它落实到四点。第一,用 `enum class` 代替裸 `enum`:作用域限定的枚举不会泄漏常量名,不能隐式转成 `int`,比较与传递都受类型约束——C 的裸枚举和 `#define` 常量在 C++ 里没有位置。第二,用 `std::variant` 代替裸 `union`:`variant` 记住当前持有什么类型,用 `std::visit` 安全访问,而 `union` 是"谁写谁负责"的裸内存区,极易踩错字段。第三,转换纪律:数值边界转换显式化、窄化报警开启、类型双关用 `memcpy`、`const_cast` 只在修接口、`reinterpret_cast` 写注释说明位布局依据。

清单 14-6 `enum class` 与 `variant` 的类型安全

```
#include <iostream>
#include <string>
#include <variant>

// 作用域枚举:不泄漏名字,不隐式转 int
enum class Color { Red, Green, Blue };
enum class Priority { Low, High }; // 同名常量互不干扰

int toInt(Color c) {
    return static_cast<int>(c); // 显式转换才拿到数值
}

// variant:类型安全的多选一
struct Circle { double r; };
struct Rect { double w, h; };

using Shape = std::variant<Circle, Rect>;

double area(const Shape& s) {
    return std::visit([](const auto& sh) -> double {
        using T = std::decay_t<decltype(sh)>;
        if constexpr (std::is_same_v<T, Circle>) {
            return 3.14159 * sh.r * sh.r;
        } else {
            return sh.w * sh.h;
        }
    }, s);
}

int main() {
    Color c = Color::Red;
    std::cout << "红色序号: " << toInt(c) << "\n";

    Shape s1 = Circle{2.0};
    Shape s2 = Rect{3.0, 4.0};
    std::cout << area(s1) << " " << area(s2) << "\n";
    return 0;
}
```

第四,让类型关系成为契约:`static_assert` 写在模板入口,把"调用方必须满足的前置条件"变成编译期硬失败;公共 API 的返回类型写清楚(别用 `auto` 糊接口);需要运行时多态时用 `dynamic_cast` 并显式处理失败分支,而不是赌"一定是那个类型"。类型安全不是消灭所有转换——那是乌托邦——而是让每个转换都有名字、有位置、有检查,错误发生时,报错在编译期而不是线上。

清单 14-7 转换纪律检查清单

```
#include <iostream>
#include <cstdint>

// 纪律 1:边界转换显式化,禁止隐式窄化
long toBytes(std::size_t pages) {
    return static_cast<long>(pages) * 4096;    // 先扩大再乘,不丢位
}

// 纪律 2:索引遍历要想到无符号语义
void dump(const int* xs, std::size_t n) {
    for (std::size_t i = 0; i < n; ++i) {      // 正向无风险
        std::cout << xs[i] << " ";
    }
    for (std::size_t i = n; i-- > 0; ) {      // 反向:后置递减惯用法
        std::cout << xs[i] << " ";
    }
    std::cout << "\n";
}

int main() {
    int arr[] = {1, 2, 3};
    dump(arr, 3);
    std::cout << toBytes(4) << "\n";          // 16384
    return 0;
}
```

快速参考:类型系统速查表

场景	工具
安全显式转换(数值、类型间)	static_cast
运行时检查的多态向下转换	dynamic_cast(失败返 nullptr / 抛 bad_cast)
位级重解释(最后手段)	reinterpret_cast + 注释依据
去掉 const(只修接口)	const_cast(写入真 const 对象是 UB)
类型双关安全做法	std::memcpy
隐式窄化保护	花括号初始化 + -Wconversion
禁止隐式类转换	explicit 构造函数/转换运算符
推导变量类型	auto / auto& / const auto& / auto&&
取出声明类型	decltype(实体名) / decltype((表达式))
完美转发返回类型	decltype(auto)
编译期类型查询	is_same_v、is_integral_v、is_base_of_v
编译期类型变换	decay_t、remove_reference_t、conditional_t
编译期类型分支	if constexpr(C++20: concepts)
类型安全枚举/多选一	enum class / std::variant + std::visit

自测题

Q 1. 隐式转换分哪几档?为什么有符号与无符号混用比较是陷阱?

✓ 答案

整型提升、整型转换、浮点转换、算术转换(找公共类型)。有符号转无符号按模运算重解释:-1 转 unsigned 成为巨大的正数,与正数比较时结果反直觉;混用比较前应显式统一类型。

Q 2. 四种显式转换分别适用什么场景?C 的万能转型 (T)x 为什么在 C++ 里是坏味道?

✓ 答案

static_cast 用于合理的显式转换(数值、类层次);dynamic_cast 用于运行时检查的多态向下转换;const_cast 用于修正误标 const 的接口;reinterpret_cast 用于位级重解释。C 的转型是一把锤子,不区分意图,容易掩盖错误转换,且 reinterpret 与 const 相关错误在 C 里无法表达与拦截。

Q 3. auto x = v[0] 与 auto& x = v[0] 有什么区别?decltype(x) 与 decltype(x)) 呢?

✓ 答案

前者复制一份,后者是引用(修改会影响容器)。decltype(x) 是实体名,得到声明的类型;decltype(x)) 把 x 当表达式求,左值表达式带引用,所以是 int&。

Q 4. if constexpr (std::is_integral_v<T>) 相比第10章的 enable_if 优势在哪?

✓ 答案

可读性:分支代码线性排列,不再需要层层 enable_if 签名技巧;不满足的分支不参与编译、不生成代码;配合 _v 后缀的 trait 变量模板,意图直接。enable_if 仍在库实现与 SFINAE 场景有用,但日常派发用它足矣。

Q 5. 写出三个"让类型错误在编译期显形"的具体做法。

✓ 答案

用 enum class 与 std::variant 替代裸 enum 与裸 union,让非法组合无法表达;模板入口写 static_assert 把前置条件变成编译失败;开启 -Wconversion 让隐式窄化报警;公共 API 返回类型显式写出并配合 nodiscard(第12章)。

实验:类型安全工具库

本实验把本章知识点组装成一个小的类型工具库:showType 用 type_traits 打印类型的类别,以 const char* 的参数统一字符串化(演示 const_cast 的正确用法);safeAdd 用 if constexpr 按整型/浮点派发并做溢出检查;Shape 用 variant 与 visit 计算面积(避免 dynamic_cast 的 ODR 顾虑);再演示 decltype(auto) 与 auto& 的引用语义。运行输出见注释。

| 实验:类型工具库 main.cpp

```

#include <iostream>
#include <string>
#include <variant>
#include <type_traits>
#include <limits>
#include <stdexcept>

// 覆盖:type_traits 查询
template <typename T>
const char* showType() {
    using U = std::decay_t<T>;          // 覆盖:decay 变换
    if constexpr (std::is_integral_v<U>)
        return "整型";
    else if constexpr (std::is_floating_point_v<U>)
        return "浮点";
    else if constexpr (std::is_pointer_v<U>)
        return "指针";
    else if constexpr (std::is_class_v<U>)
        return "类";
    else
        return "其他";
}

// 覆盖:if constexpr 编译期派发 + 溢出检查
template <typename T>
T safeAdd(T a, T b) {
    if constexpr (std::is_integral_v<T>) {
        if ((b > 0 && a > std::numeric_limits<T>::max() - b) ||
            (b < 0 && a < std::numeric_limits<T>::min() - b)) {
            throw std::overflow_error("整型溢出");
        }
        return a + b;
    } else {
        return a + b;                  // 浮点分支,无溢出检查
    }
}

// 覆盖:std::variant 类型安全多选一
struct Circle { double r; };
struct Rect { double w, h; };
using Shape = std::variant<Circle, Rect>;

double area(const Shape& s) {
    return std::visit([](const auto& sh) -> double {
        using T = std::decay_t<decltype(sh)>;
        if constexpr (std::is_same_v<T, Circle>) // 覆盖:is_same
            return 3.14159 * sh.r * sh.r;
        else
            return sh.w * sh.h;
    }, s);
}

// 覆盖:decltype(auto) 返回引用
template <typename T>
decltype(auto) first(T& v) { return v.front(); }

```

```

// 覆盖:const_cast 修正"误标 const"的旧接口(只读,不改写)
const char* legacyFormat(const std::string& s) {
    // 旧库签名是非 const,这里 const_cast 只去掉接口层 const
    return const_cast<std::string*>(&s)->c_str();
}

int main() {
    // 覆盖:auto / auto& / const auto& 引用语义
    std::string name = "类型系统";
    auto copy = name;           // 副本
    auto& ref = name;           // 引用
    const auto& cre = name;      // 常引用
    ref += "进阶";
    std::cout << copy << " | " << cre << "\n";    // 类型系统 | 类型系统进阶

    // 覆盖:类型打印
    std::cout << showType<int>() << " " << showType<double&>()
        << " " << showType<const char*>() << "\n";

    // 覆盖:safeAdd 整型分支(会抛异常)与浮点分支
    try {
        std::cout << safeAdd(2147483640, 10) << "\n";    // 溢出异常
    } catch (const std::overflow_error& e) {
        std::cout << "捕获: " << e.what() << "\n";
    }
    std::cout << safeAdd(2.5, 3.25) << "\n";            // 5.75

    // 覆盖:variant + visit
    Shape s = Rect{3.0, 4.0};
    std::cout << "面积: " << area(s) << "\n";

    // 覆盖:decltype(auto) 引用返回
    std::string arr[] = {"甲", "乙"};
    auto& f = first(arr);
    f = "丙";
    std::cout << arr[0] << " " << arr[1] << "\n";

    // 覆盖:const_cast 修正旧接口
    std::cout << legacyFormat(name) << "\n";
    return 0;
}

// 预期输出:
// 类型系统 | 类型系统进阶
// 整型 浮点 指针
// 捕获: 整型溢出
// 5.75
// 面积: 12
// 丙 乙
// 类型系统进阶

```

工程实践 1:转换审计

给代码评审加一条固定项:任何 `static_cast/reinterpret_cast/const_cast` 都要回答"为什么这里需要显式转换"。说不出理由的转换就是设计缺口——通常是接口类型设计错了。`reinterpret_cast` 出现处必须带注释说明位布局依据与兼容保证。

工程实践 2:类型作为文档

把"合法的状态"编码进类型:用 `enum class` 与 `std::variant` 表达互斥状态,用 `const` 表达只读,用值类型表达所有权。类型注释不写"该传什么",而是让错的根本传不进来。配合 `static_assert` 把前置条件钉在编译期,线上事故在编译期就变成错误报告。

本章总结

类型系统是 C++ 最重要的安全网:隐式转换提供便利也埋雷,显式转换用四种 `cast` 各司其职,`auto` 与 `decltype` 让类型推导自动化,`type_traits` 把类型变成可编程的编译期数据,`if constexpr` 与 `concepts` 把类型分支变成可读的日常代码。最终纪律只有一条:让错误尽可能发生在编译期——类型用得越准,运行期越安静。至此 14 章走完,从变量到内存、从对象到泛型、从网络到类型系统,你已经掌握了现代 C++ 的完整地图。